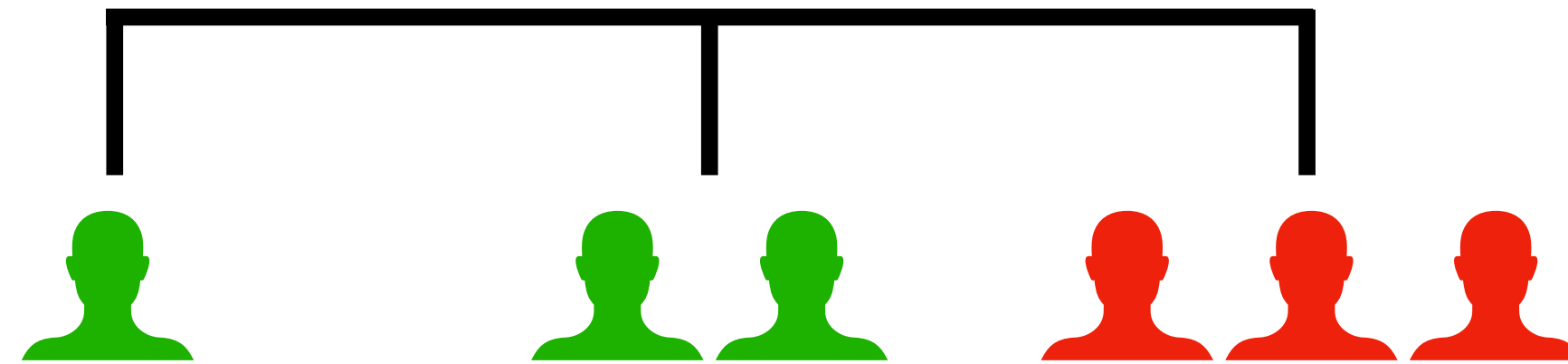


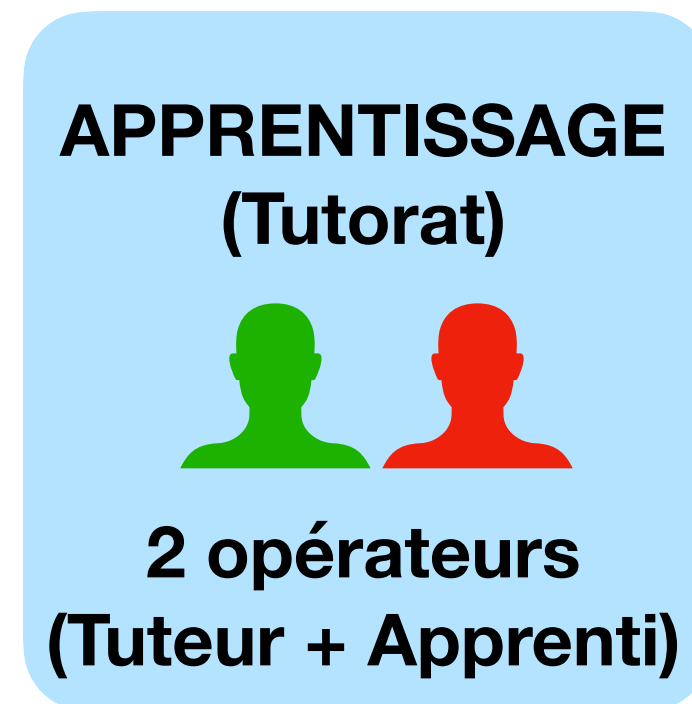
Visualisation des contraintes du model

Chaque opération est réalisée
par au plus deux opérateurs



Mode d'exécution et temps de traitement

 z_{ij}^0

 T_{ij}^{solo}
 z_{ij}^1

 $T_{ij}^{apprentissage}$
 z_{ij}^2

 $T_{ij}^{collaboration}$
 z_{ij}^3

 $T_{ij}^{solo_no_level} = T_{ij}^{solo} \cdot \gamma$
 $\gamma = 2$

Temps :

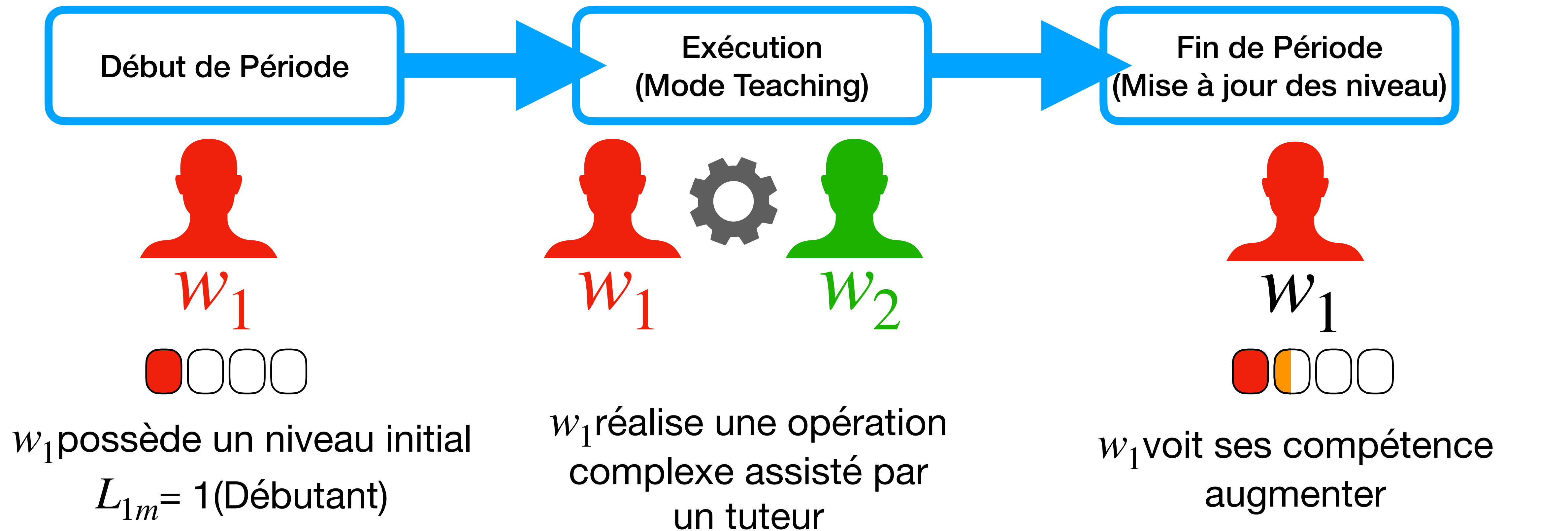
Standard

Allongé

Réduit

Allongé

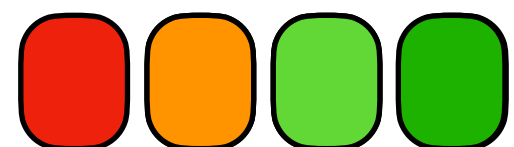
Modélisation du transfert de compétence



$$\text{Augmentation} = nb_operation \cdot \beta$$

β = coefficient d'apprentissage

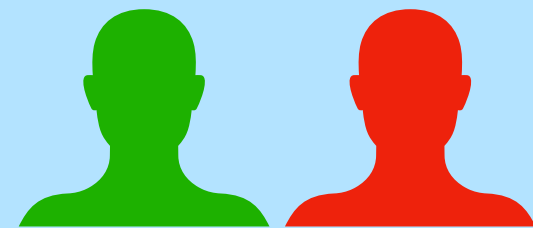
Au cours d'une période les opérateurs ne peuvent pas voir leurs compétence pour un corps de métier augmenter de plus de 1 unité



Cognitive Load

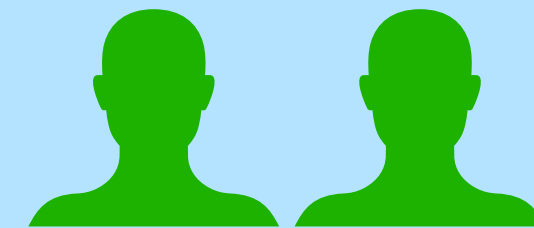
Toutes les opération fait en **duo** génère une fatigue mental du à la coordination avec l'autre opérateur

APPRENTISSAGE
(Tutorat)



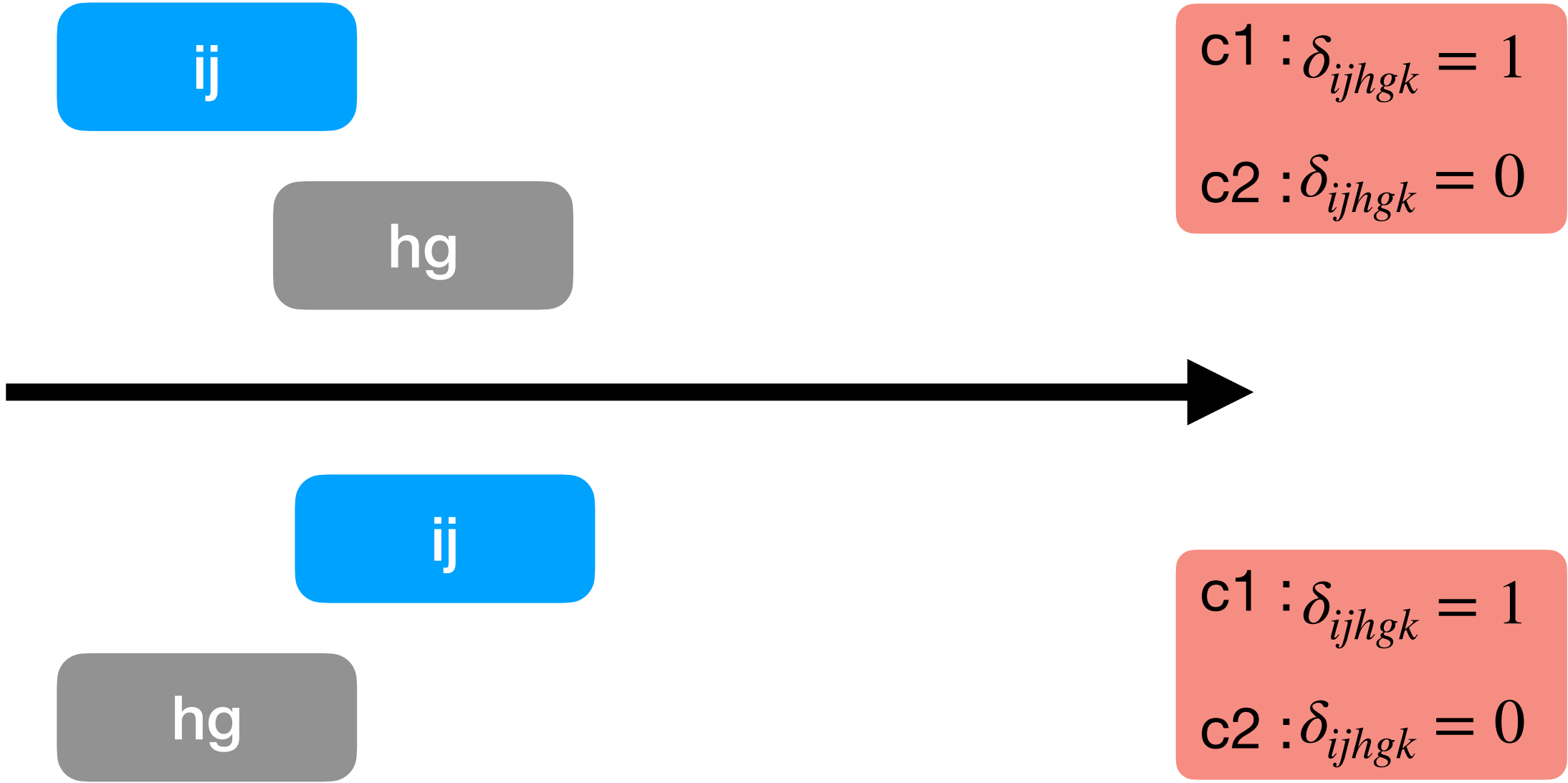
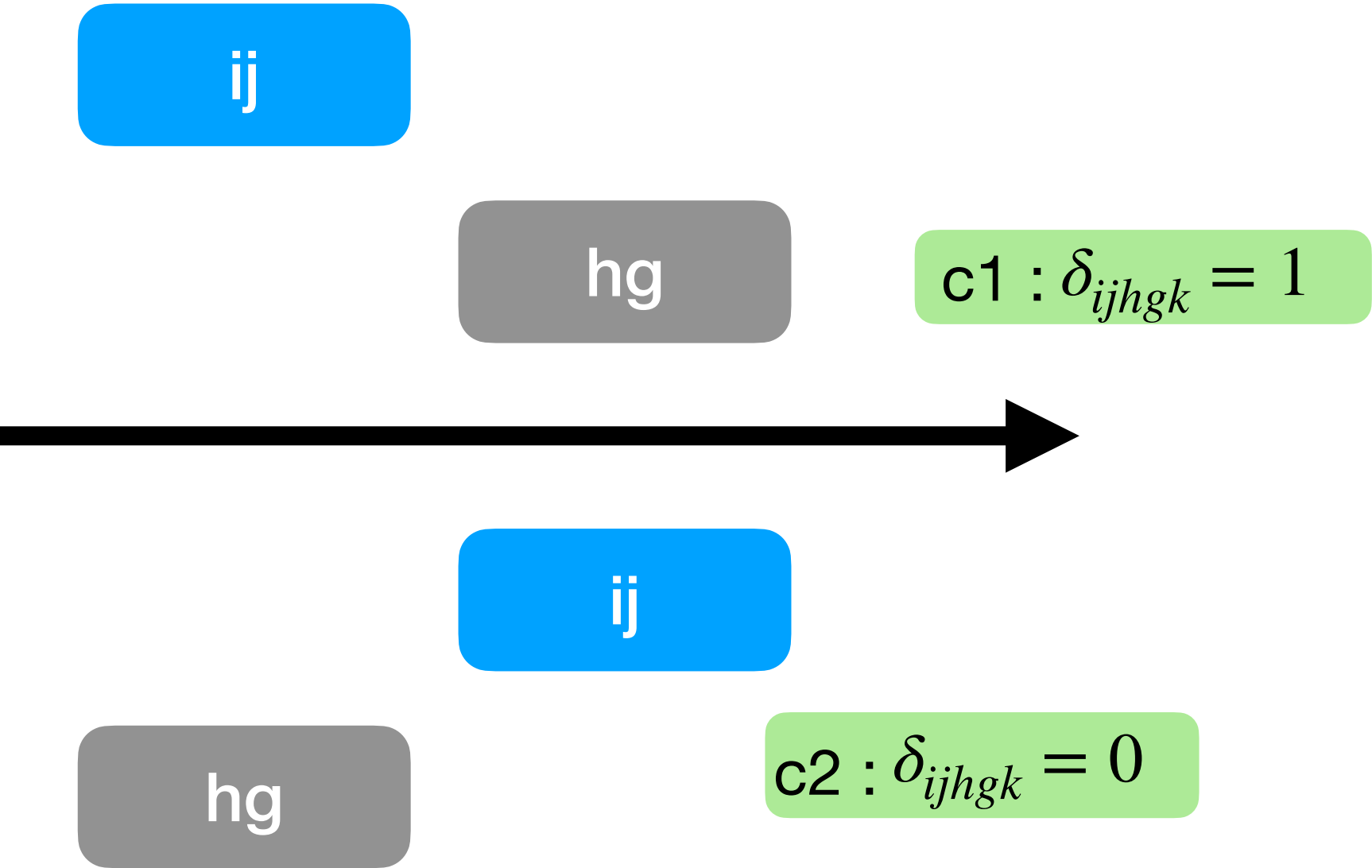
2 opérateurs
(Tuteur + Apprenti)

COLLABORATION
(compétent)



2 opérateurs
(Compétents)

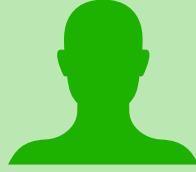
Overlap



Identification de l'opérateur le moins compétant
parmis ceux affecté pour chaque opération
(Linéarisation du min)

Opérateurs			
w1		Niv. 4	δ'_{ijk}
w2		Niv. 2	δ'_{ijk}
w3		Niv. 3	δ'_{ijk}
w4		Niv. 2	δ'_{ijk}

Level_{min} = 2

Opérateurs			
w1		Niv. 4	δ'_{ijk}
w2		Niv. 2	δ'_{ijk}
w3		Niv. 1	δ'_{ijk}
w4		Niv. 2	δ'_{ijk}

Level_{min} = 1

Précédence des opérations d'un même job

Job i



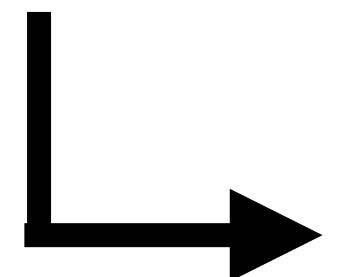
L'ordre dans lequel les opérations d'un jobs sont ordonnancer doit être respecté

Task Job et Job before limit

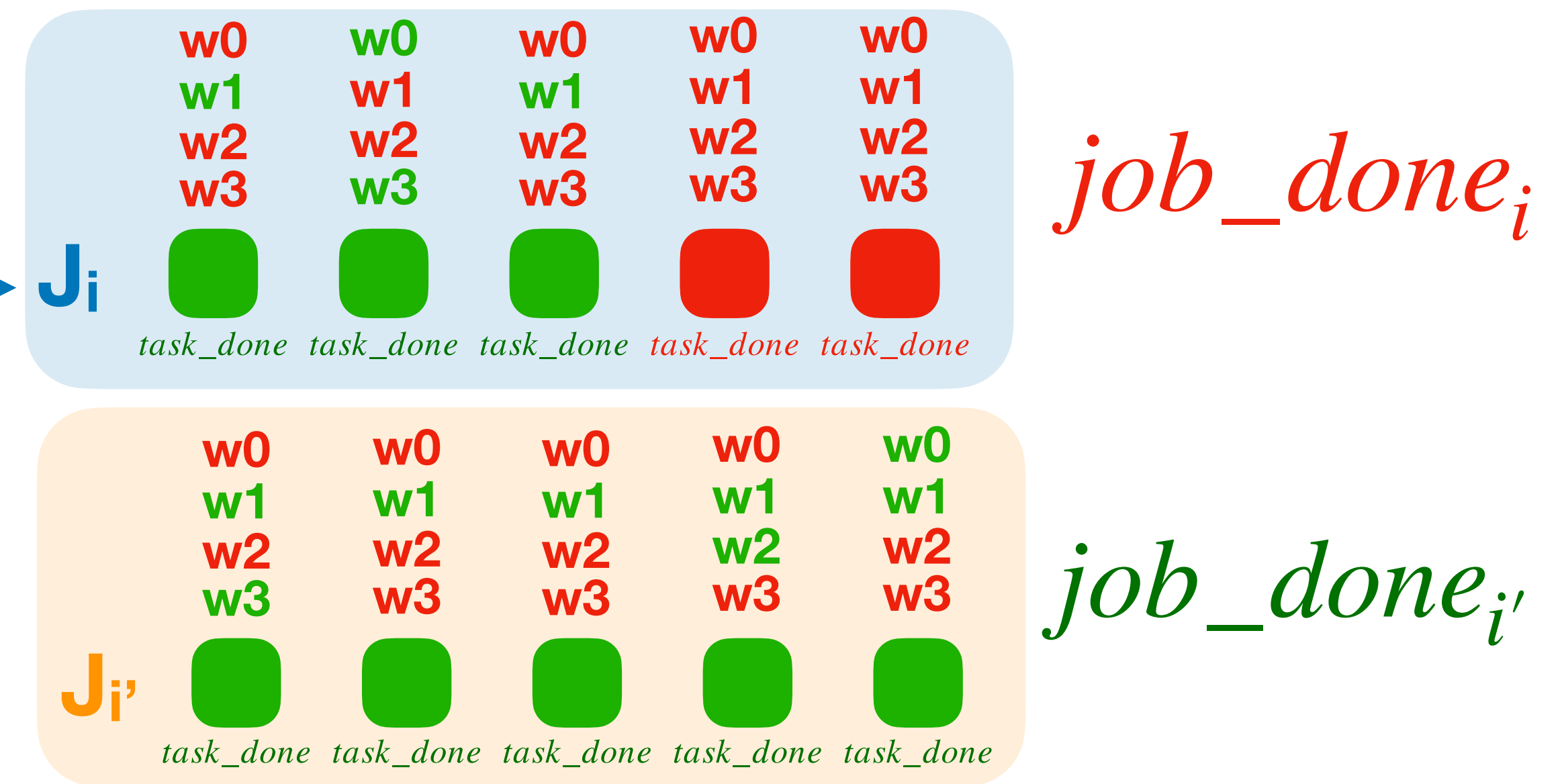
$$\forall i \in J, j \in J_i \quad task_done_{ij} \leq \sum_k x_{ijk}$$

$$\forall i \in J, j \in J_i \quad job_done_i \leq task_done_{ij}$$

$$\forall i \in J, j \in J_i, k \in W \quad x_{ijk} \leq job_done_i$$


 $\Rightarrow \forall i, j, k \quad x_{ijk} = 0$

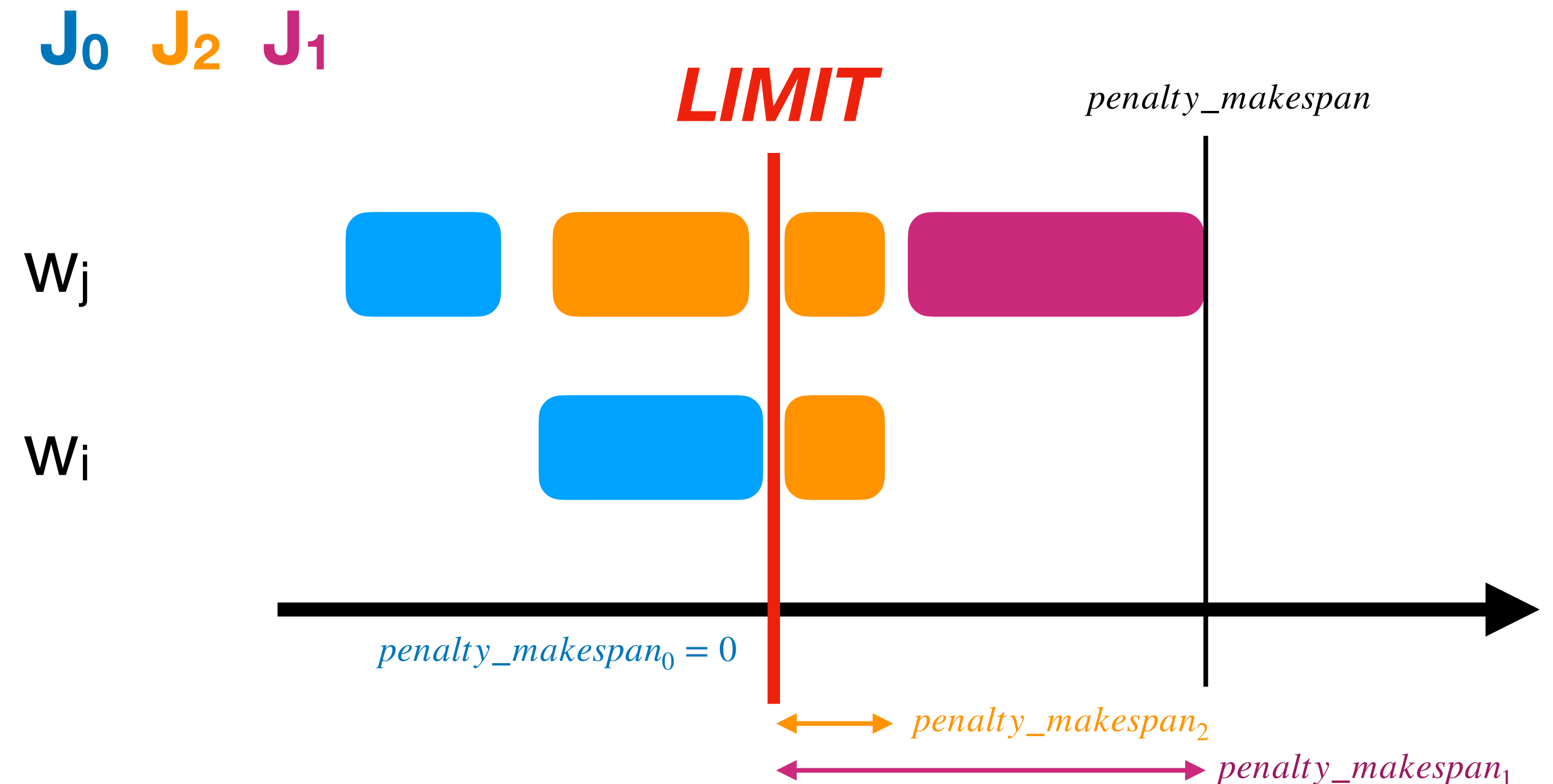
$$\forall i \in J \quad job_done_before_limit_i \leq job_done_i$$



CONSTRAINED MAKESPAN

L'utilisateur rentre une limite pour le makespan et le modele force les jobs affecté a ce finir avant cette limite

$$C_{max} \leq LIMIT + penalty_makespan$$
$$C_i \leq LIMIT + penalty_makespan_i$$

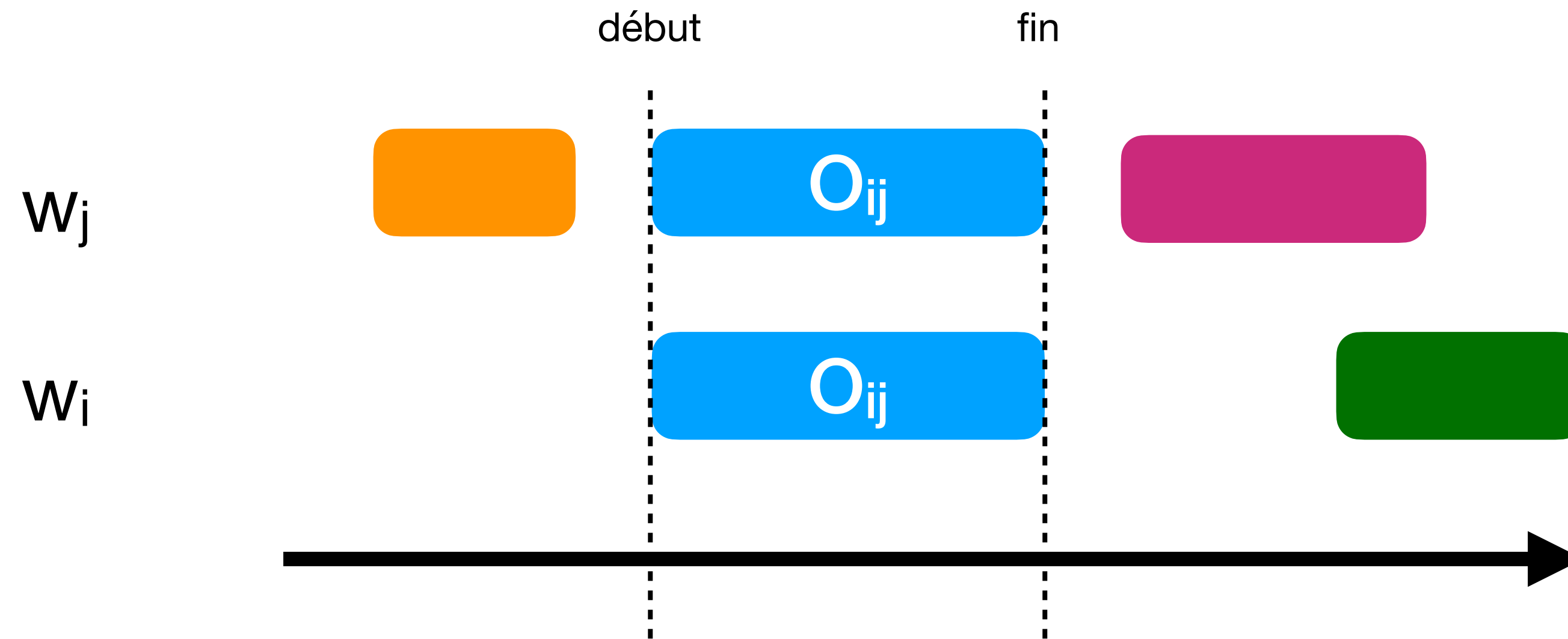


$$\forall i \in J \quad penalty_makespan_i \leq (1 - job_done_before_limit_i) \cdot M$$

Sous l'hypothèse de maximisation de *job_done_before_limit_i*

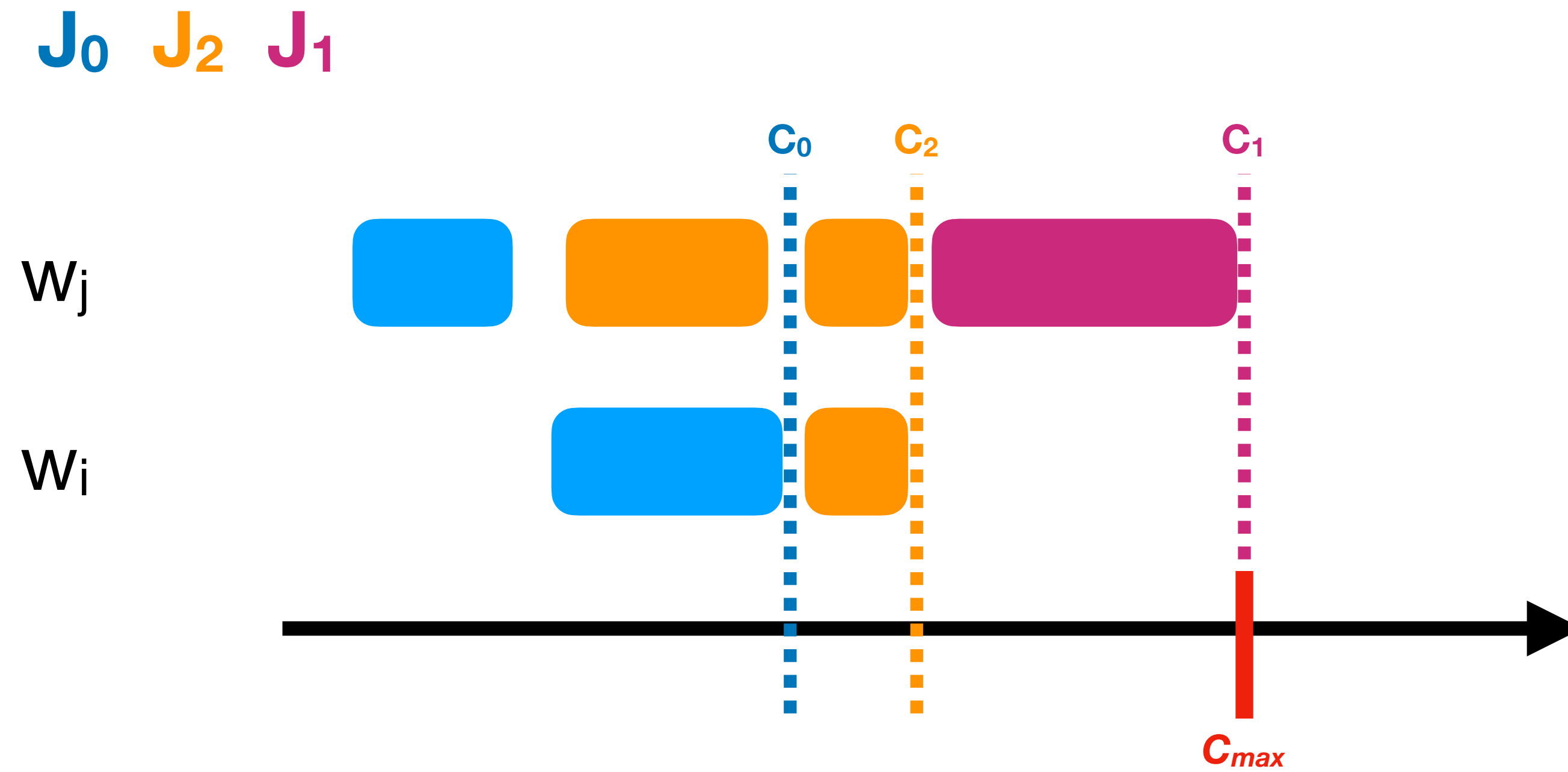
Job peut être done avant limite si ne produit pas de pénalité de makespan

Synchronisation des opérations fait en duo (Collaboration ou apprentissage)



Date de complétude de chaque job et du job se terminant le plus tard affecter

Contraintes par borne inferieur



Niveau de difference entre level du worker et niveau de difficulté de l'operation doit toujours être ≥ 1

w1		Niv. 4
w2		Niv. 1.9
w3		Niv. 1
w4		Niv. 2
w5		Niv. 2.3
w6		Niv. 3



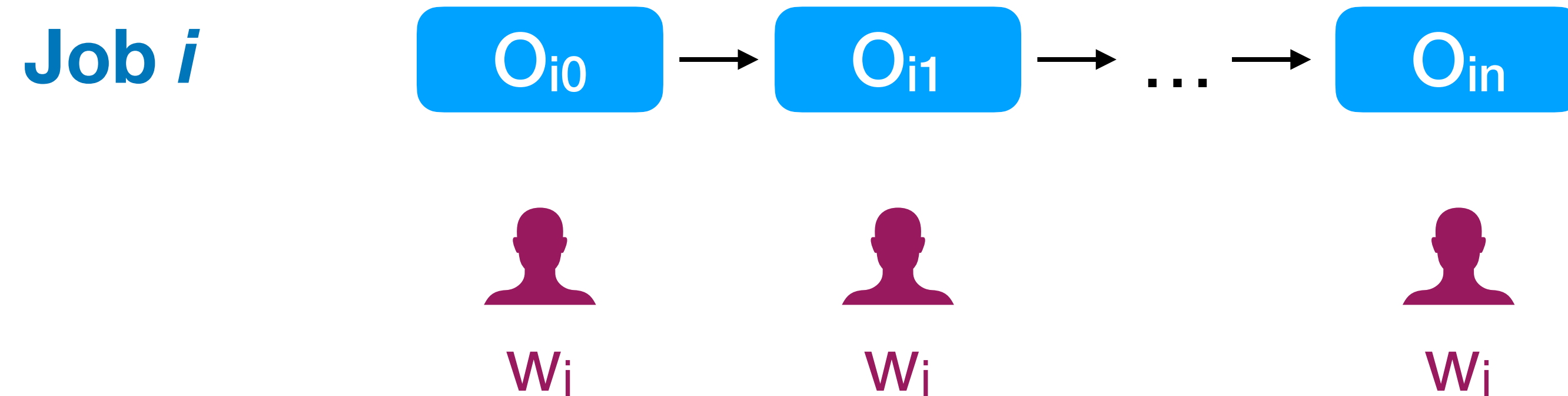
Niv. Difficulté = 3

w2 et **w3** ne peuvent
faire parti des workers
affecté à l'operation O_{ij}

$$x_{ij2} = x_{ij3} = 0$$

Opérateur de base par job

Pour chaque job, un worker de base doit être désigné et doit être présent sur toutes les opérations du job considéré



$y_{ik} = 1 \rightarrow x_{ijk} = 1$ pour toutes le opérations du Job i

HARD CONSTRAINT REQUIRED LEVEL

Si w_i n'a pas le level requis pour l'opération alors il doit être accompagné
L'opérateur qui l'accompagne doit avoir le niveau requis

Les opérateurs w₂ et w₄ ne peuvent faire l'opération seul	w1		Niv. 4
	w2		Niv. 2
	w3		Niv. 3
	w4		Niv. 2



Niv. Difficulté = 3

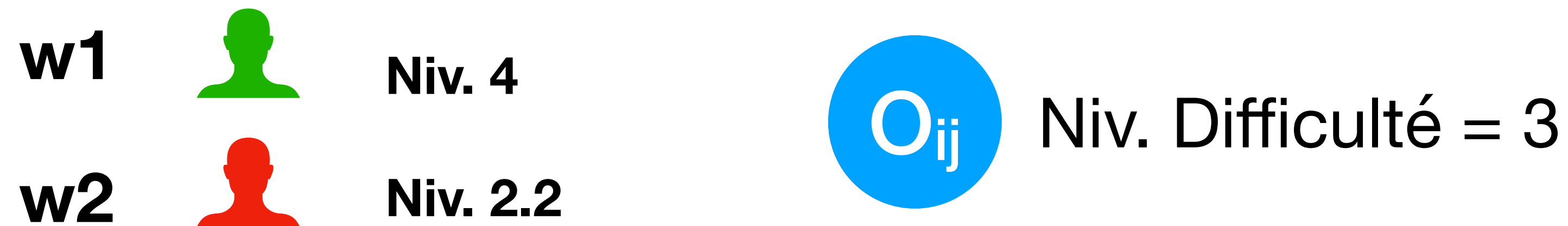
$$x_{ijk} + \sum_{k' \in W, k' \neq k} x_{ijk'} \geq M_2 \cdot (x_{ijk} - 1) + 2 \quad \text{Tel que worker } k \text{ n'a pas le niveau requis pour } O_{ij}$$

$$\sum_{k' \in W, k' \neq k} x_{ijk'} \cdot L_{km} \geq D_{ij} \cdot x_{ijk}$$

SOFT CONSTRAINT REQUIRED LEVEL

On interdit 2 workers sans niveau requis à faire opérations en duo

Si un opérateur sans niveau requis est affecté à une opération la variable $penalty_levels_{ij}$ vaut la différence de niveau entre l'opérateur et l'opération



Si w1 fait seul $penalty_levels_{ij} \geq 0$

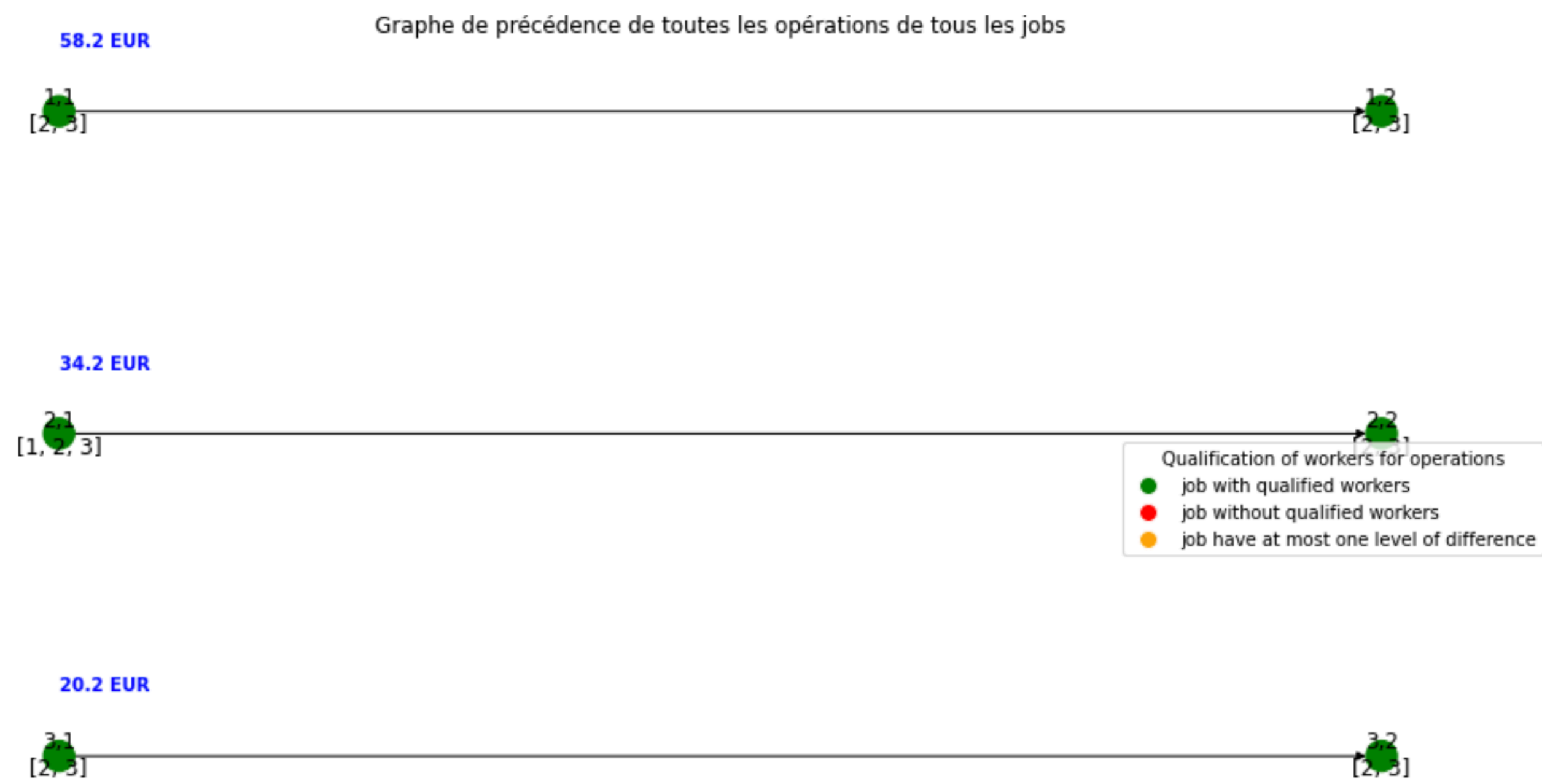
Si w2 fait seul $penalty_levels_{ij} \geq 0.8$

Si fait a 2 alors $penalty_levels_{ij}$ non contraint

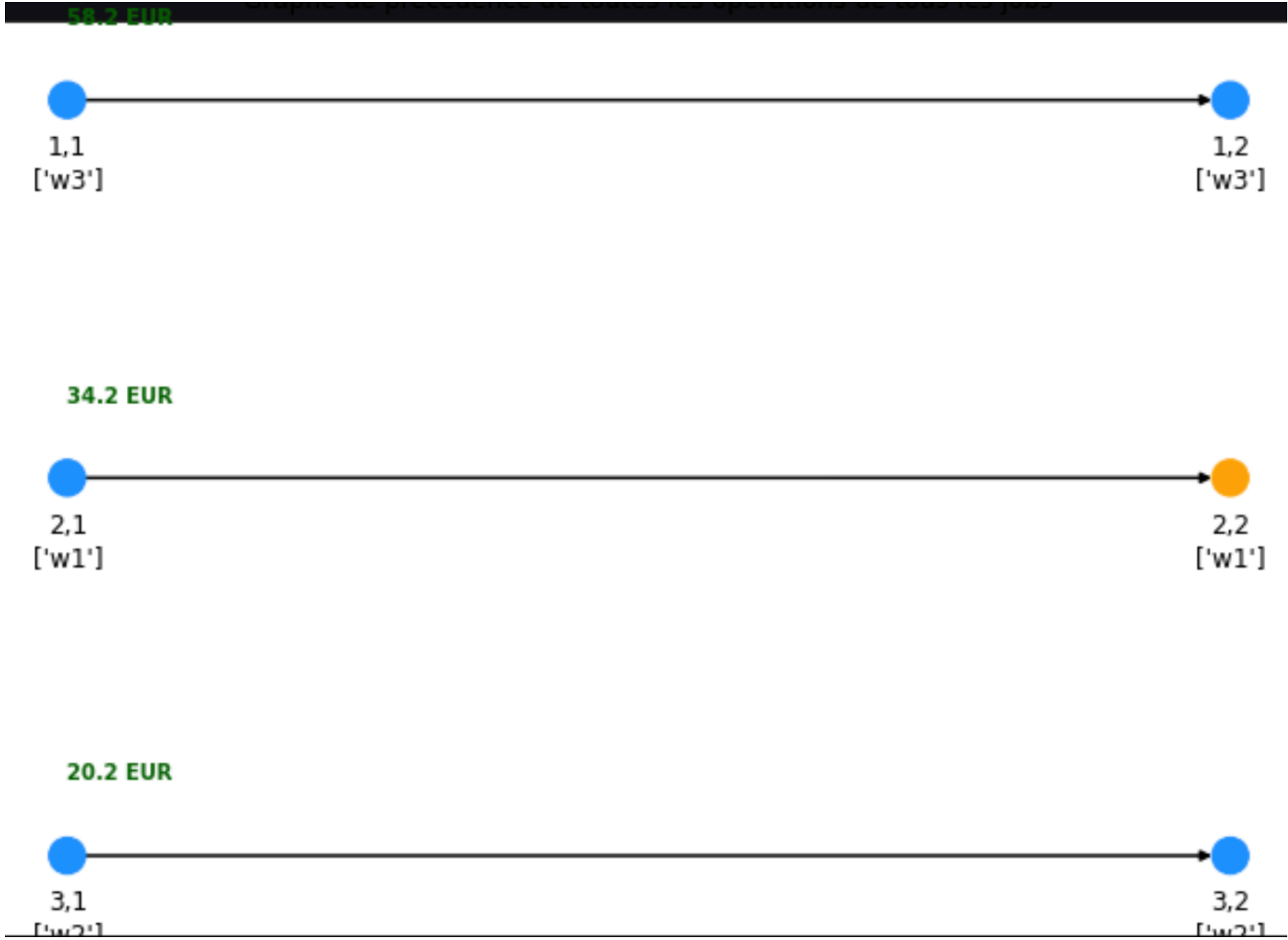
**1 Seul job est autorisé a avoir des opérations
fait par un worker non compétant**

Modélisation de la fonction objectif

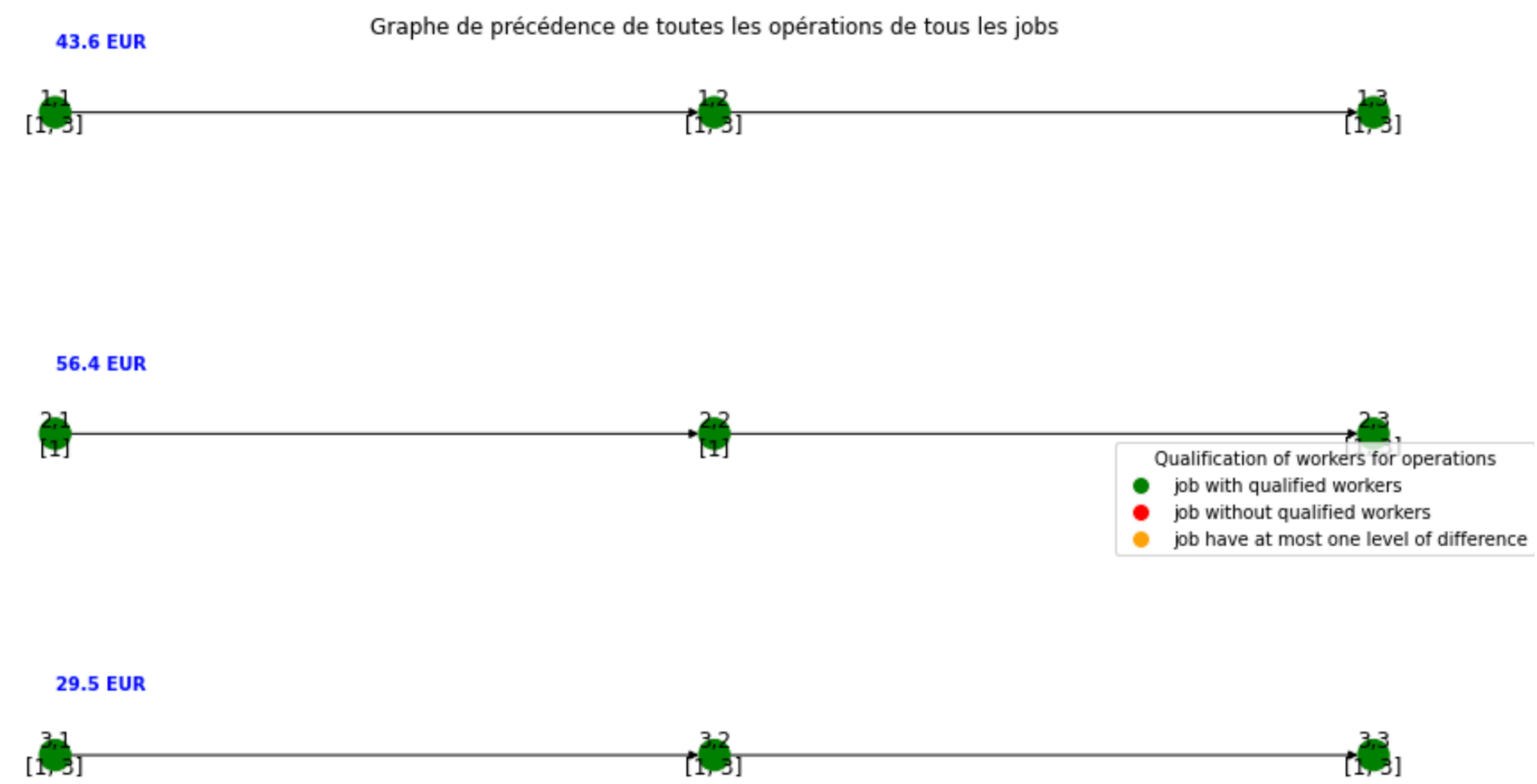
$$\sum \textit{Profit} \cdot 100 + \textit{Skills} \cdot 10 + \textit{Ergonomie} \cdot 1$$

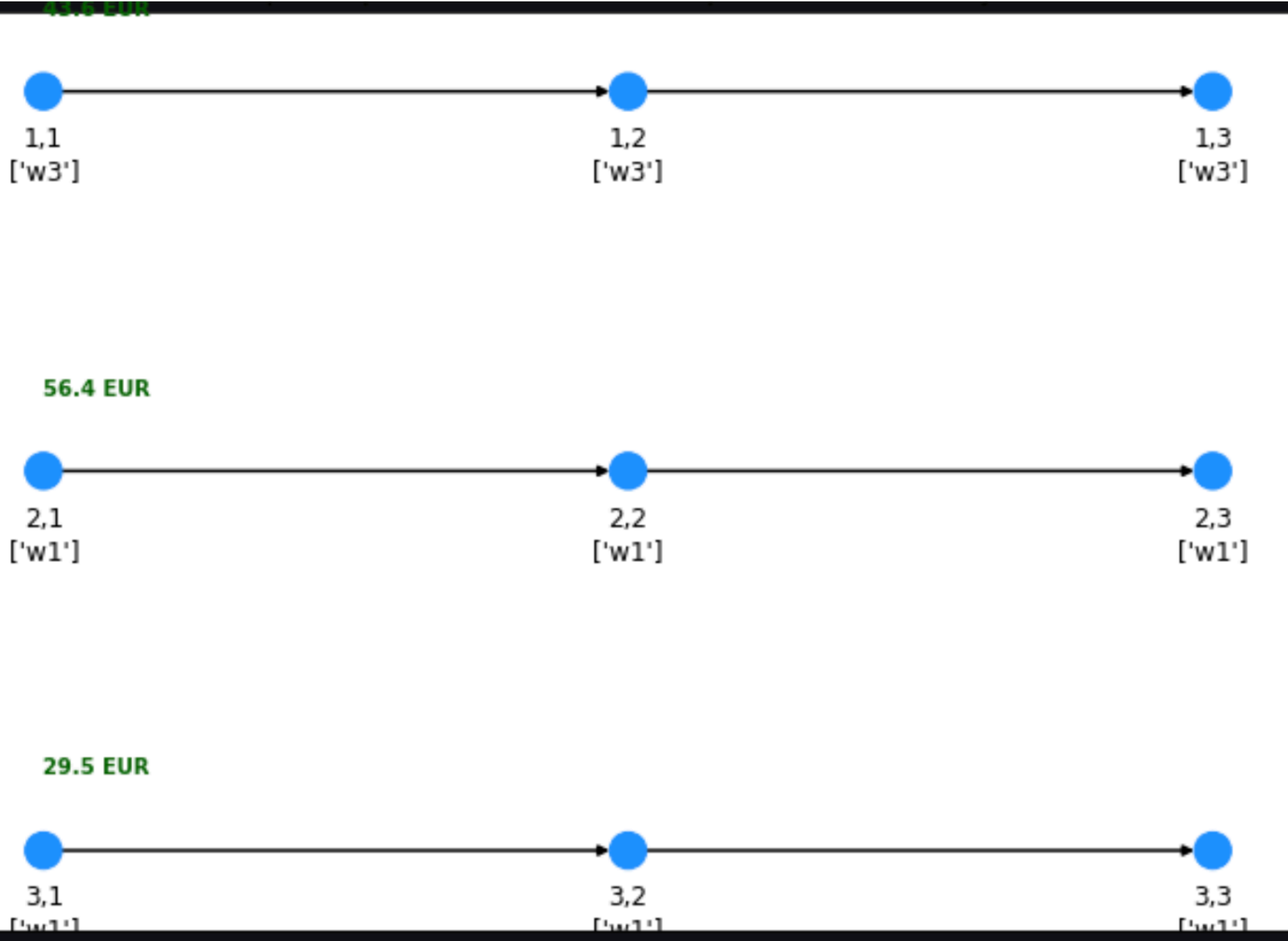


- Cmax

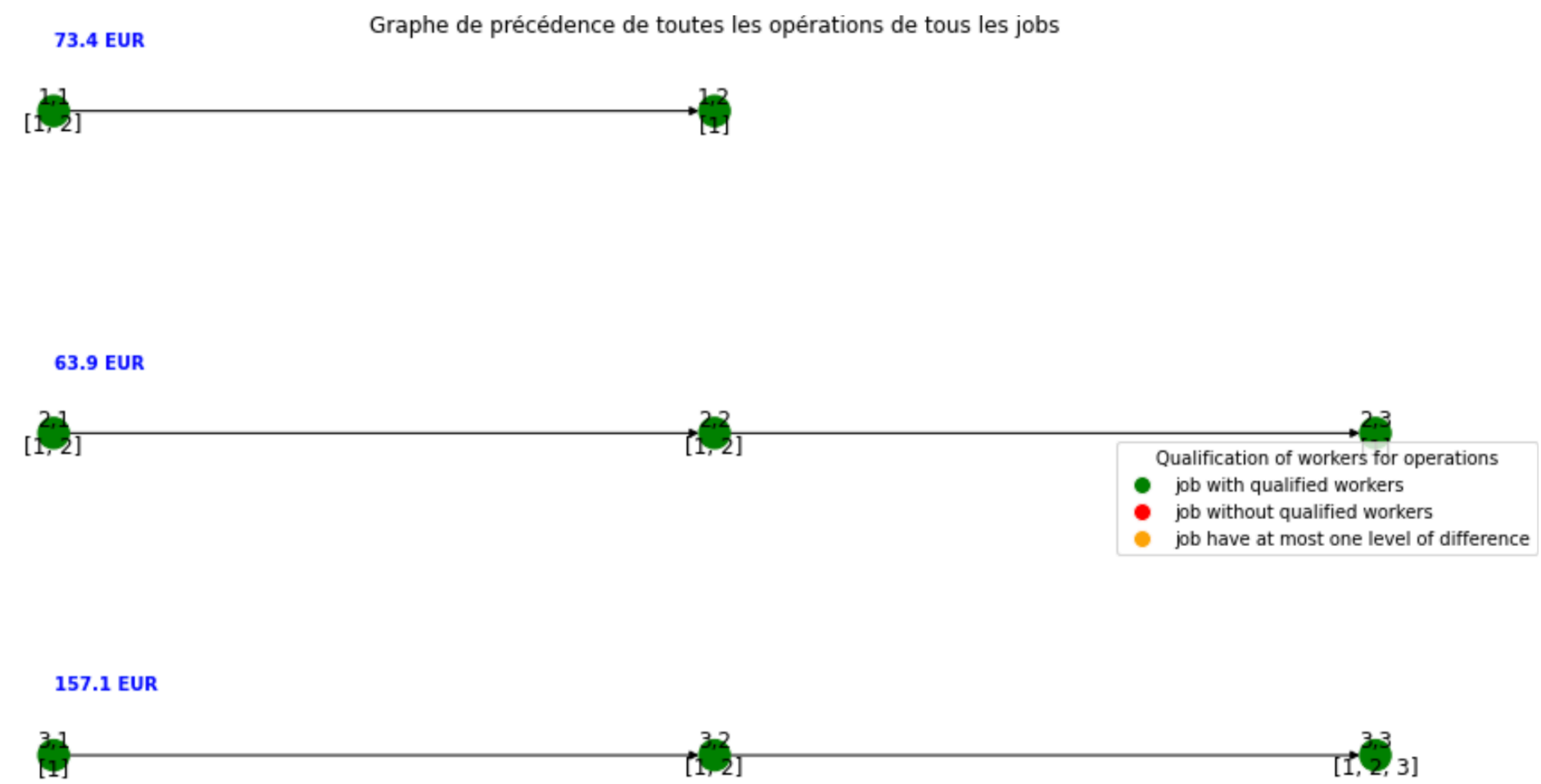


```
instance : 0
profit : 112.61712462747883
skills : 0.5
cognitive_load : 0.0
Cmax : 24.0
11242.712462747882
```



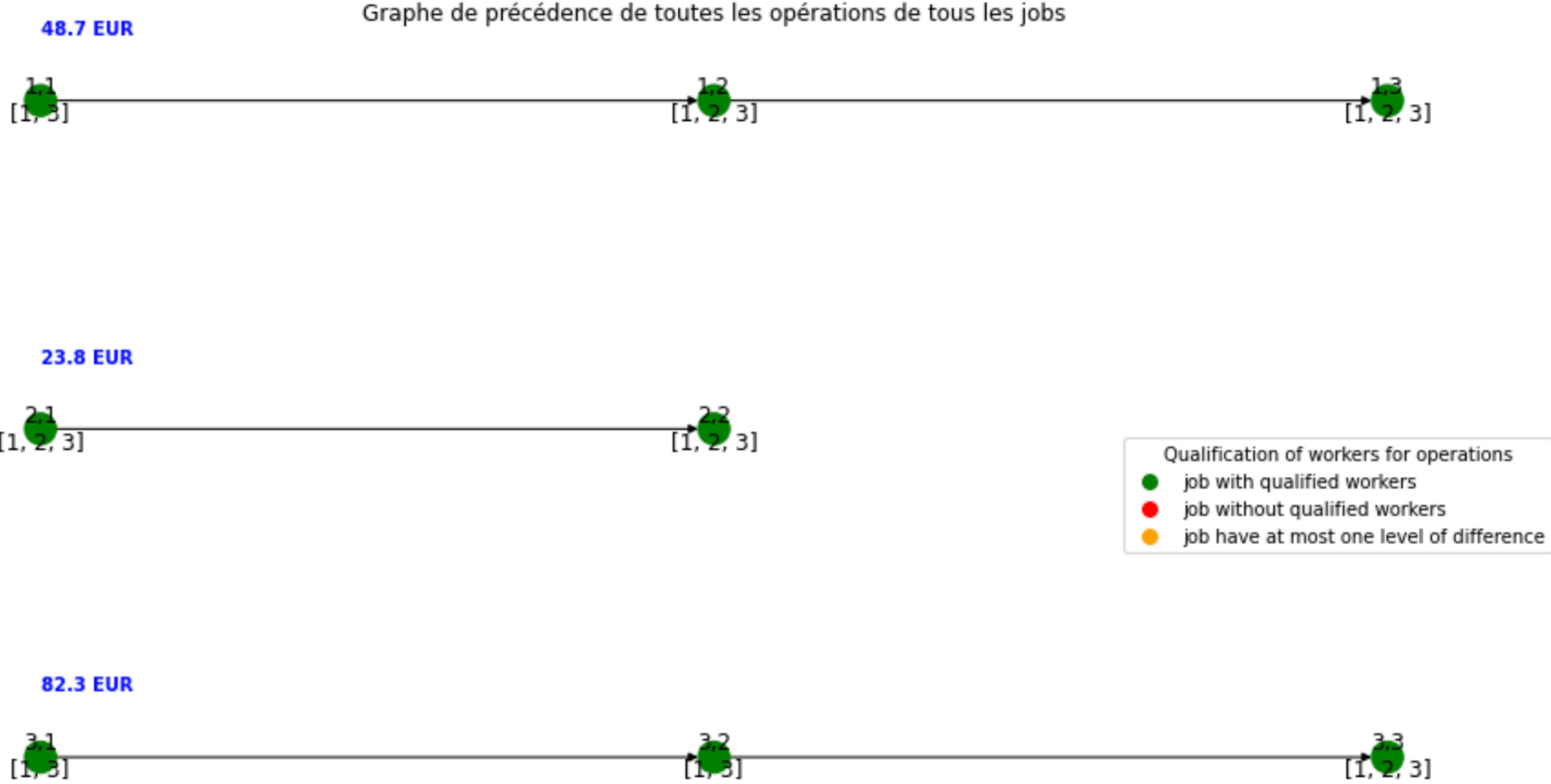


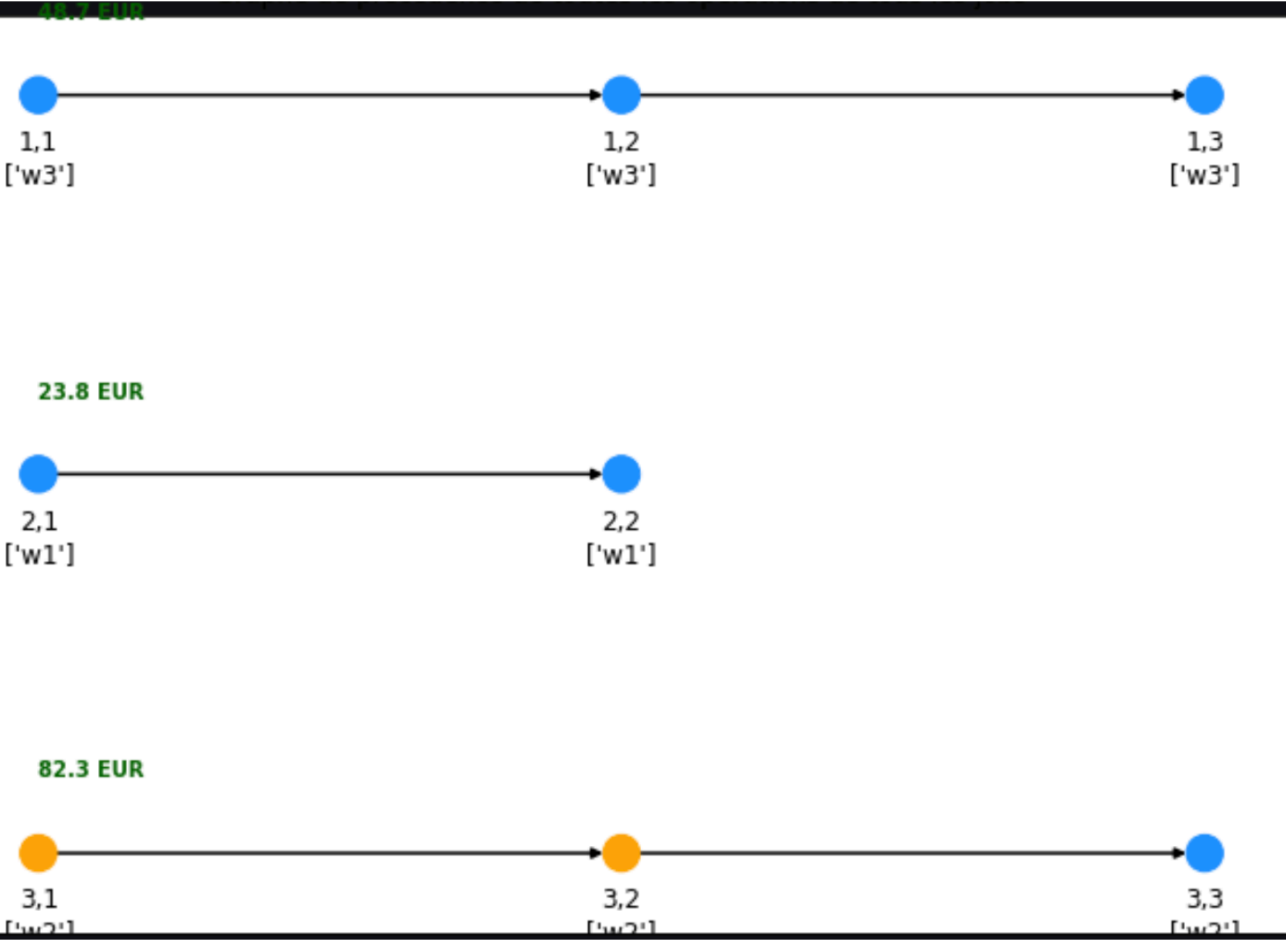
```
instance : 1
profit : 129.5178110061303
skills : 0.0
cognitive_load : 0.0
Cmax : 30.0
12921.78110061303
```



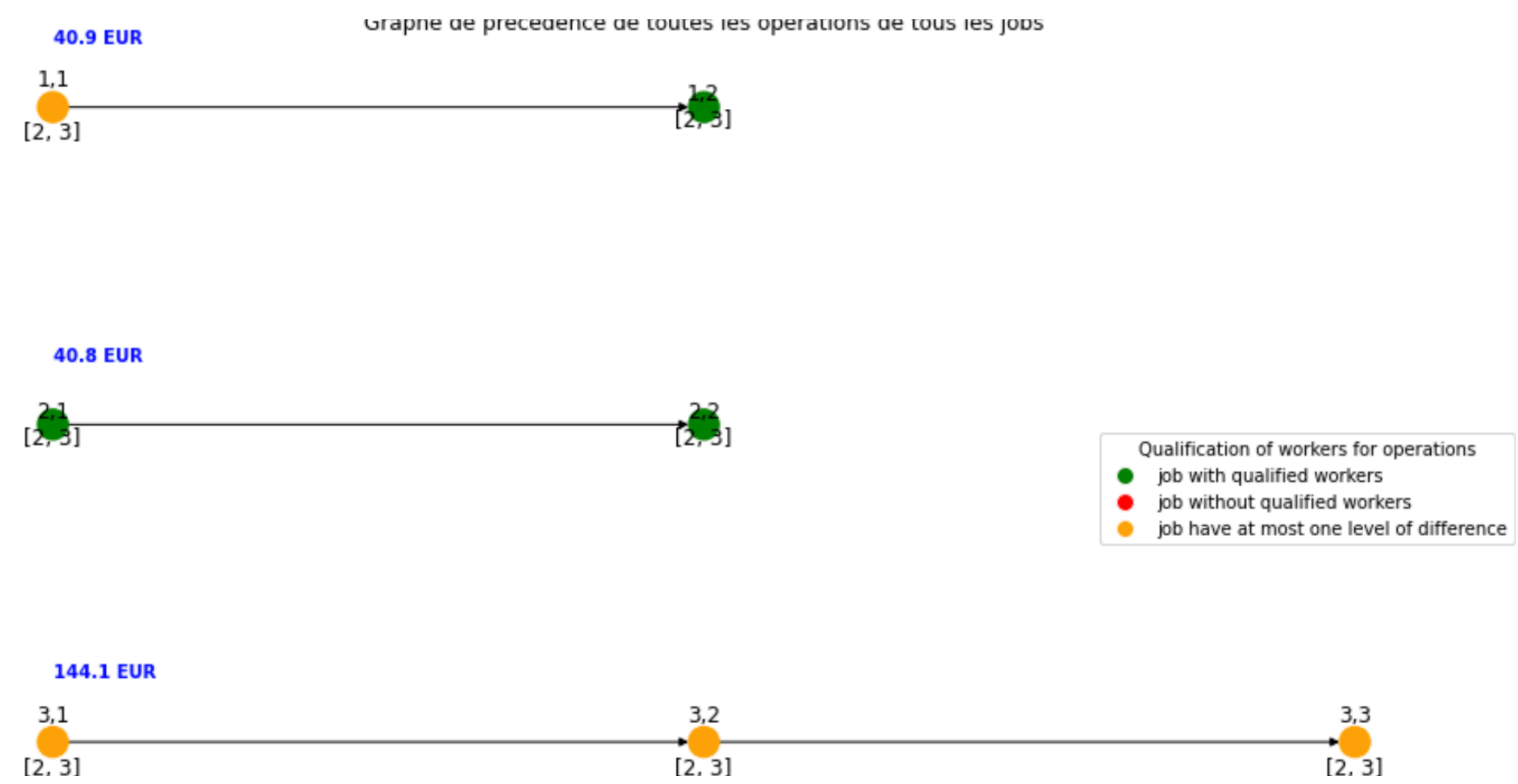


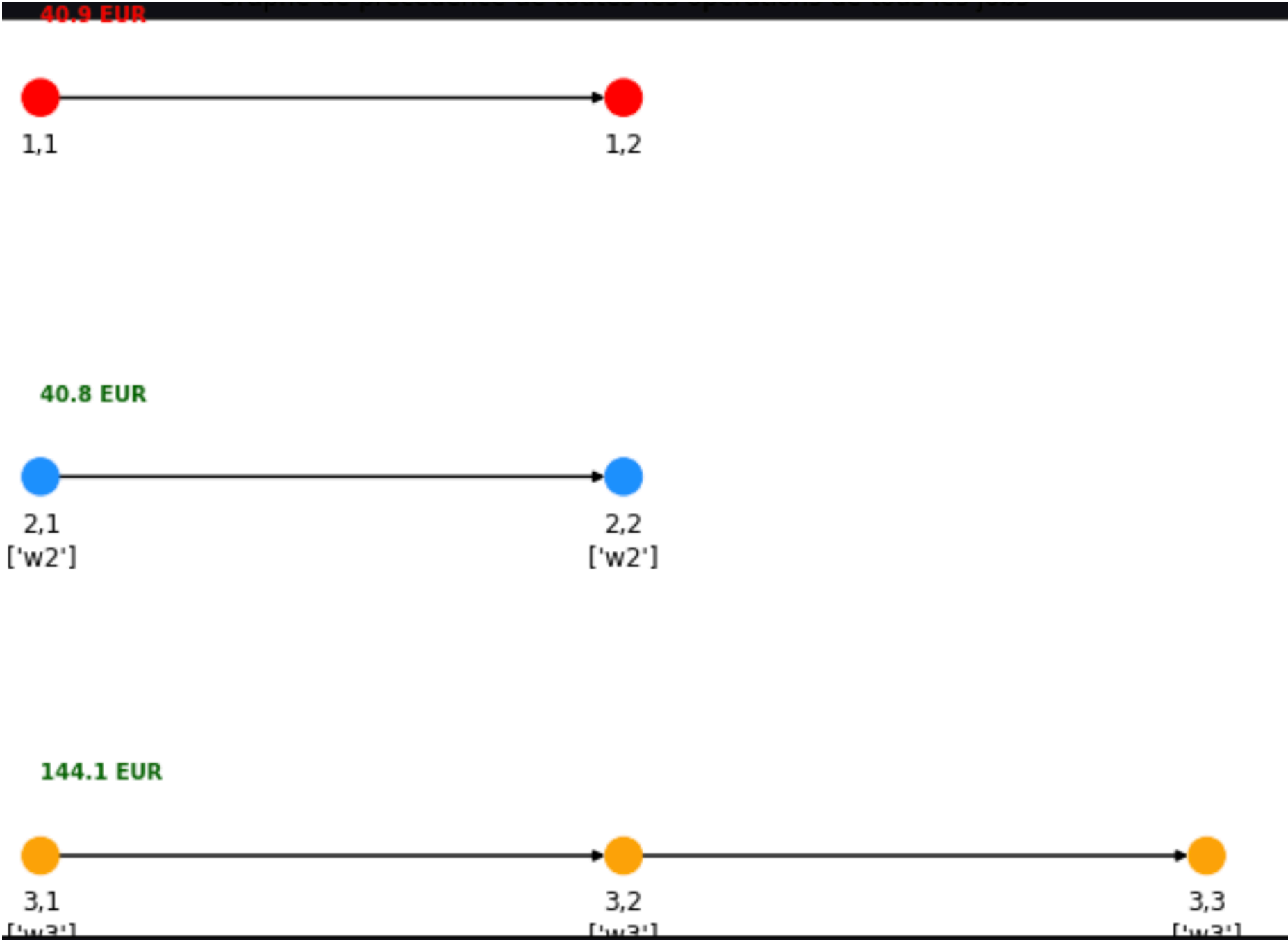
```
instance : 2
profit : 294.3630623249443
skills : 0.04613405158988826
cognitive_load : 0.0
Cmax : 25.0
29411.76757301033
```





```
instance : 3
profit : 154.90003658703284
skills : 1.0
cognitive_load : 0.0
Cmax : 15.0
15485.003658703285
```





```
instance : 4
profit : 184.91409749612126
skills : 0.8859634953188777
cognitive_load : 0.0
Cmax : 32.0
18468.269384565312
```


MULTIPLE TEST

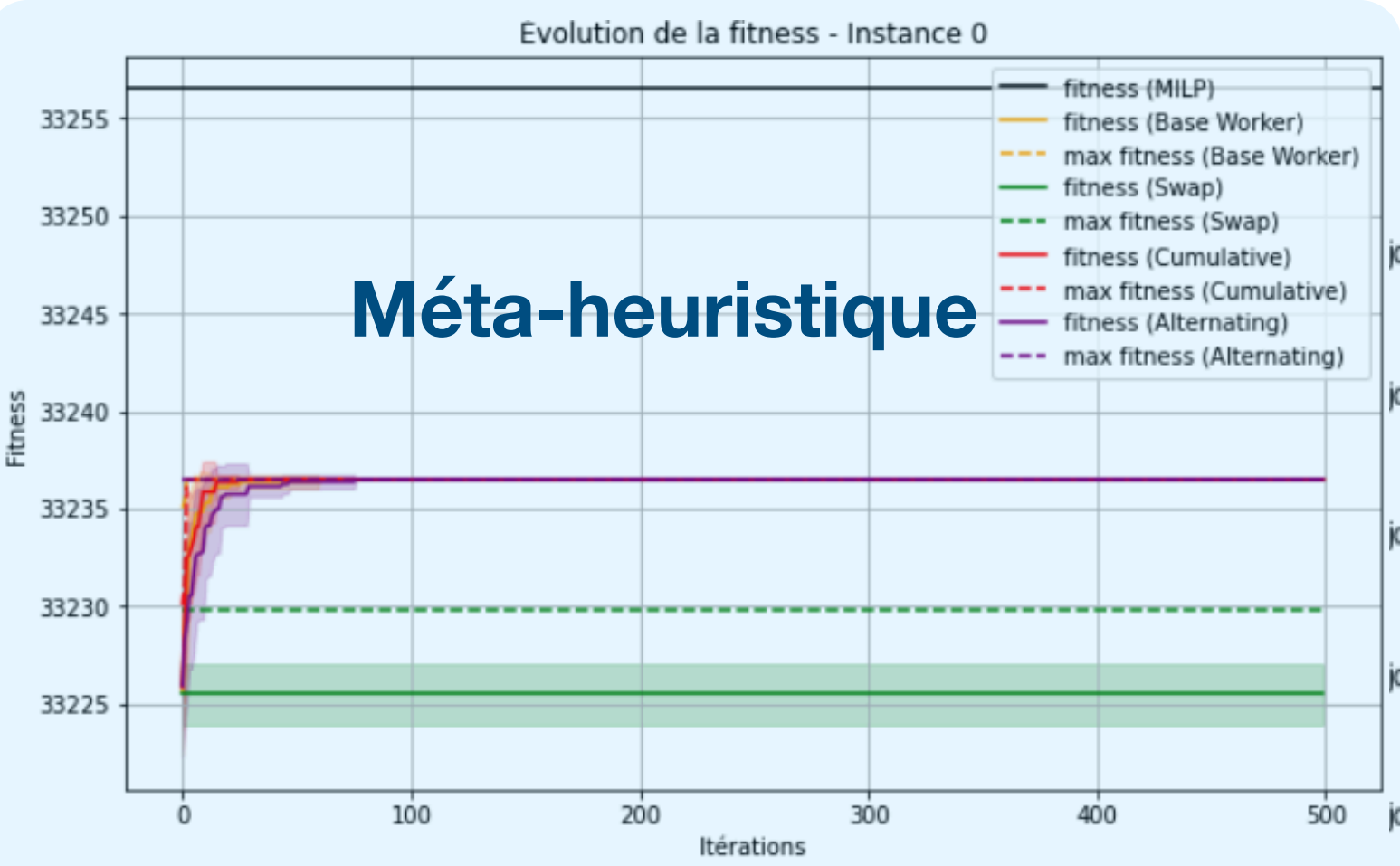
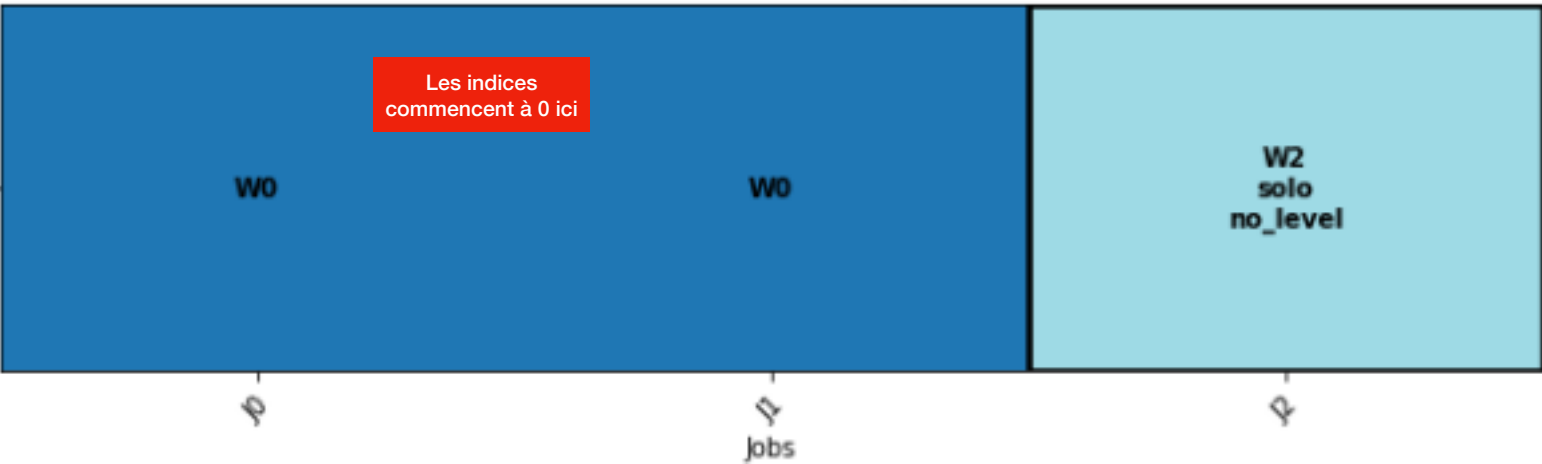
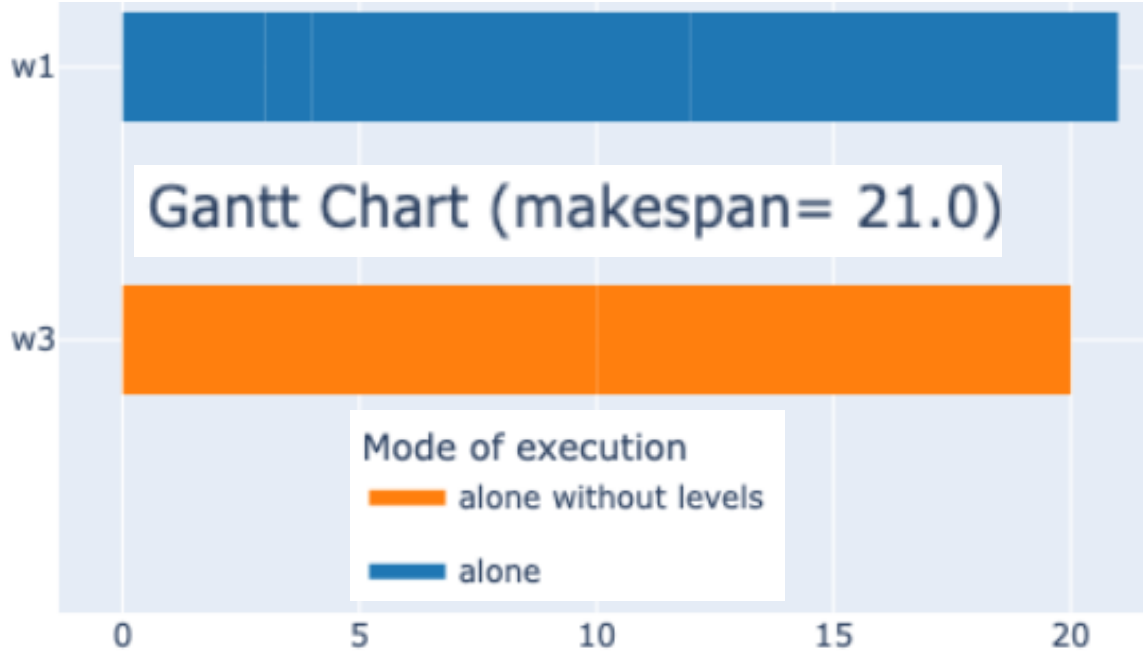
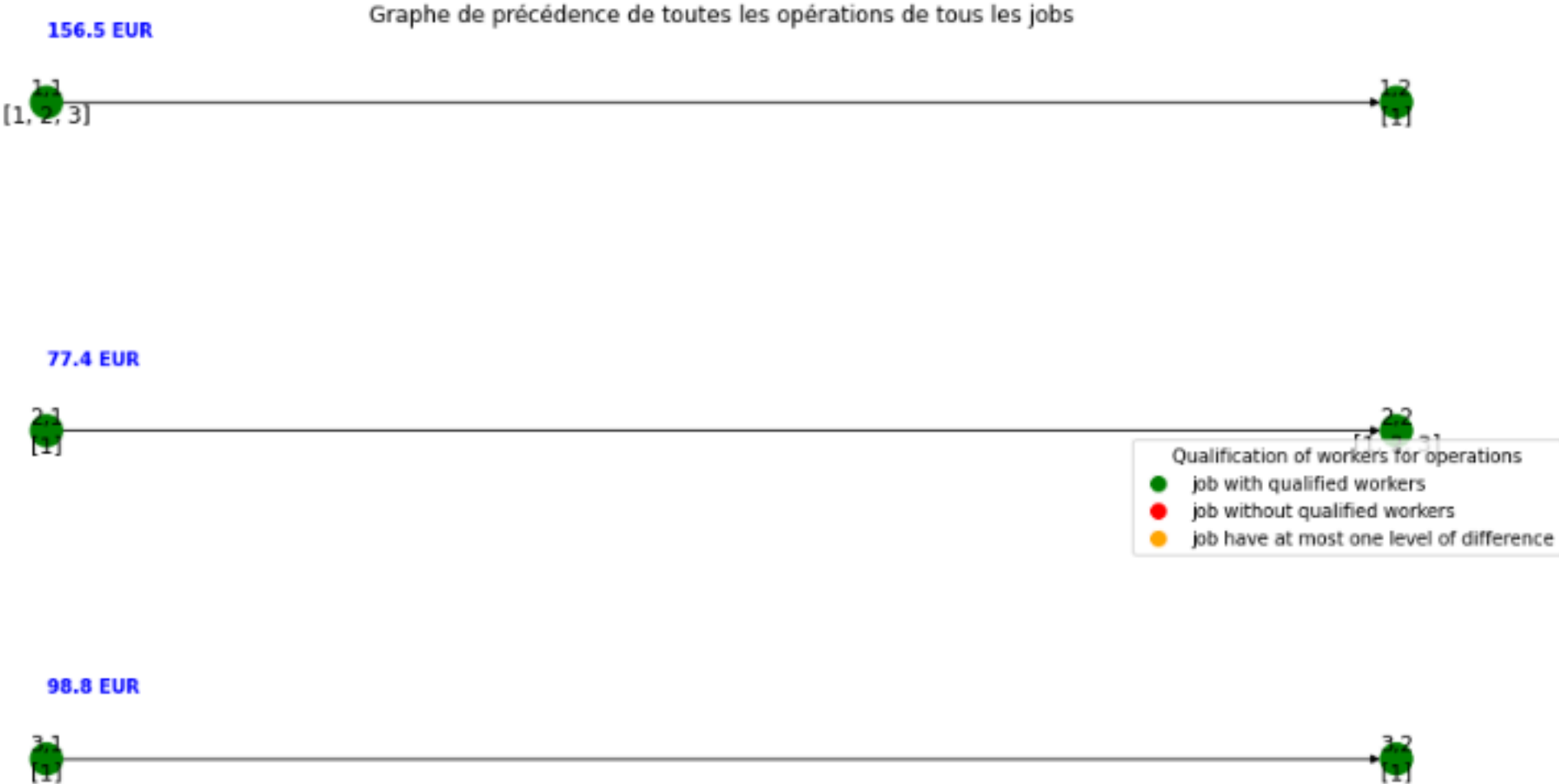
```
CONSTRAINTS_CONFIG = {  
    "job_with_no_skills": True,  
    "constrained_makespan": False,  
    "no_base_worker": False,  
    "no_solo_tasks": False,  
}  
  
WEIGHTS = {"profit": 100, "skills": 10, "cognitive_load": 1}  
  
OBJECTIVE_TERMS_CONFIG = {  
    "benefit_all_jobs": 0,  
    "penalty_levels": 0,  
    "penalty_makespan_job": 0,  
    "Cmax": 1,  
}
```

Note a moi même:
Faire commencer les indices de chaque graphique identiquement

MILP MODEL

instance : 0
profit : 332.67493007272844
skills : 1.0
cognitive_load : 0.0
Cmax : 21.0
33256.493007272846

Instance 0



Comparaison des meilleures valeurs obtenues - Instance 0

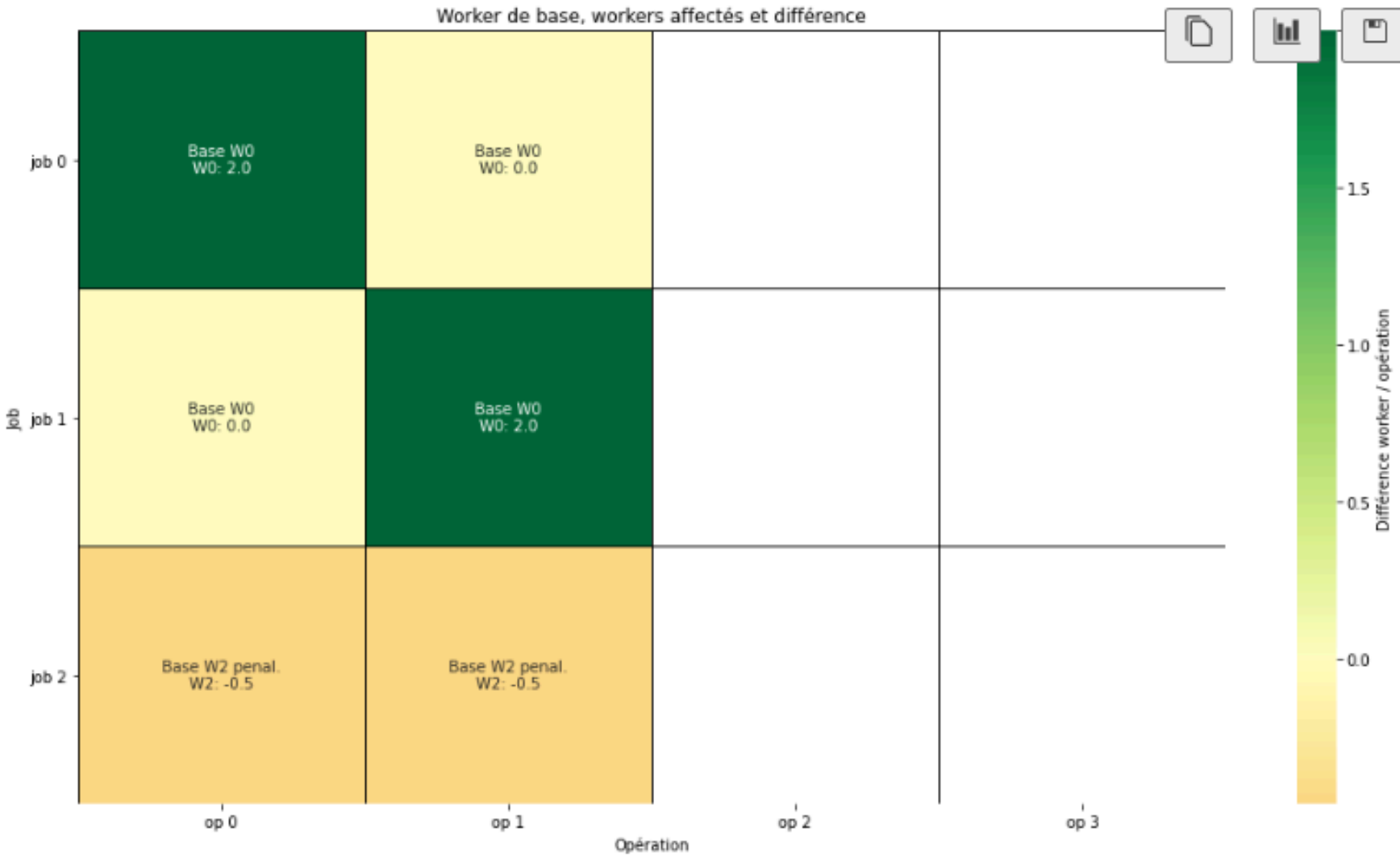
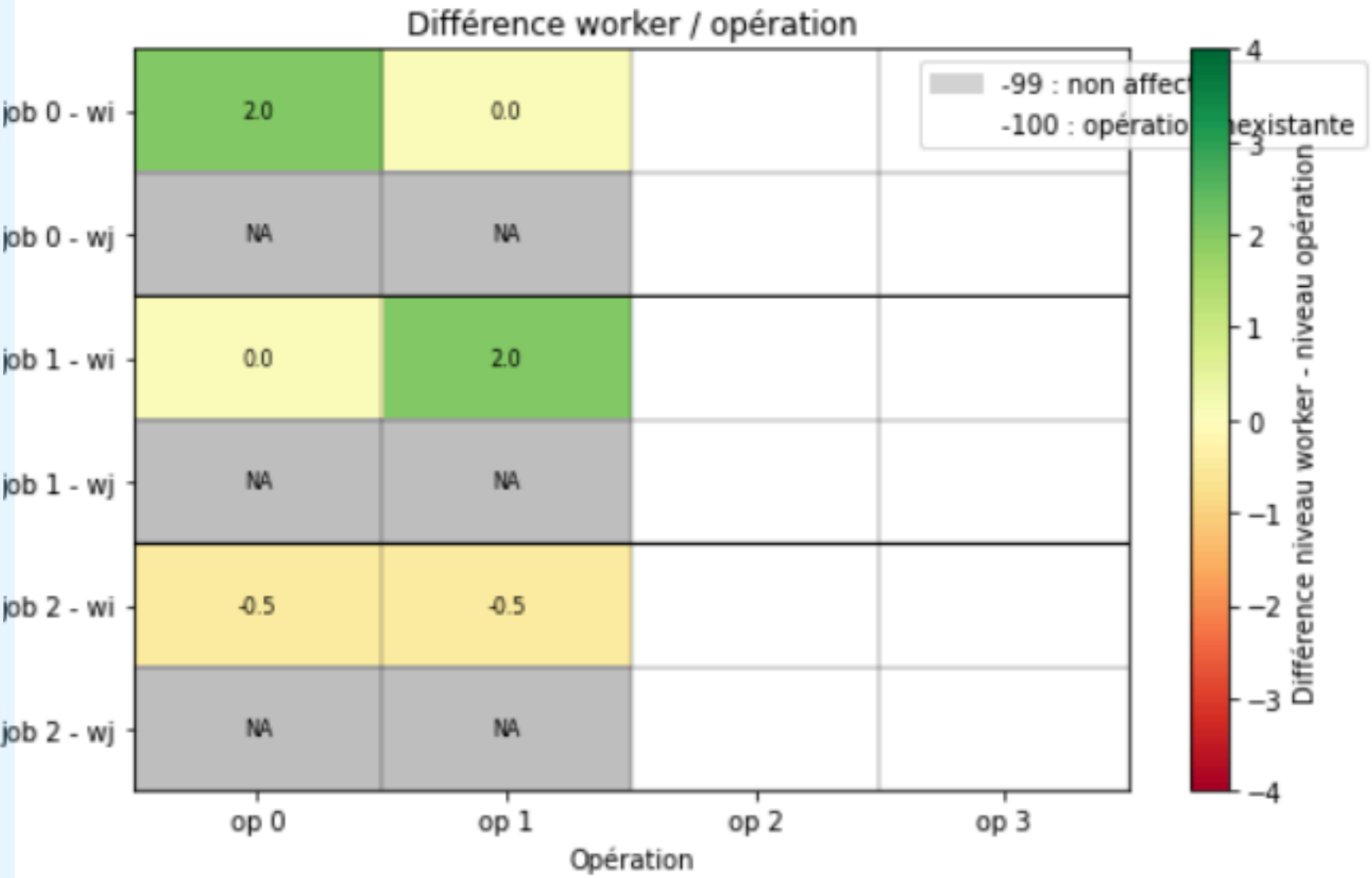
Solution exacte MILP
33256.49

Base worker
Max = 33236.49
Moy. fin = 33236.49
Gap max = 0.06 %
Gap moy fin = 0.06 %

Swap
Max = 33229.76
Moy. fin = 33225.49
Gap max = 0.08 %
Gap moy fin = 0.09 %

Cumulative
Max = 33236.49
Moy. fin = 33236.49
Gap max = 0.06 %
Gap moy fin = 0.06 %

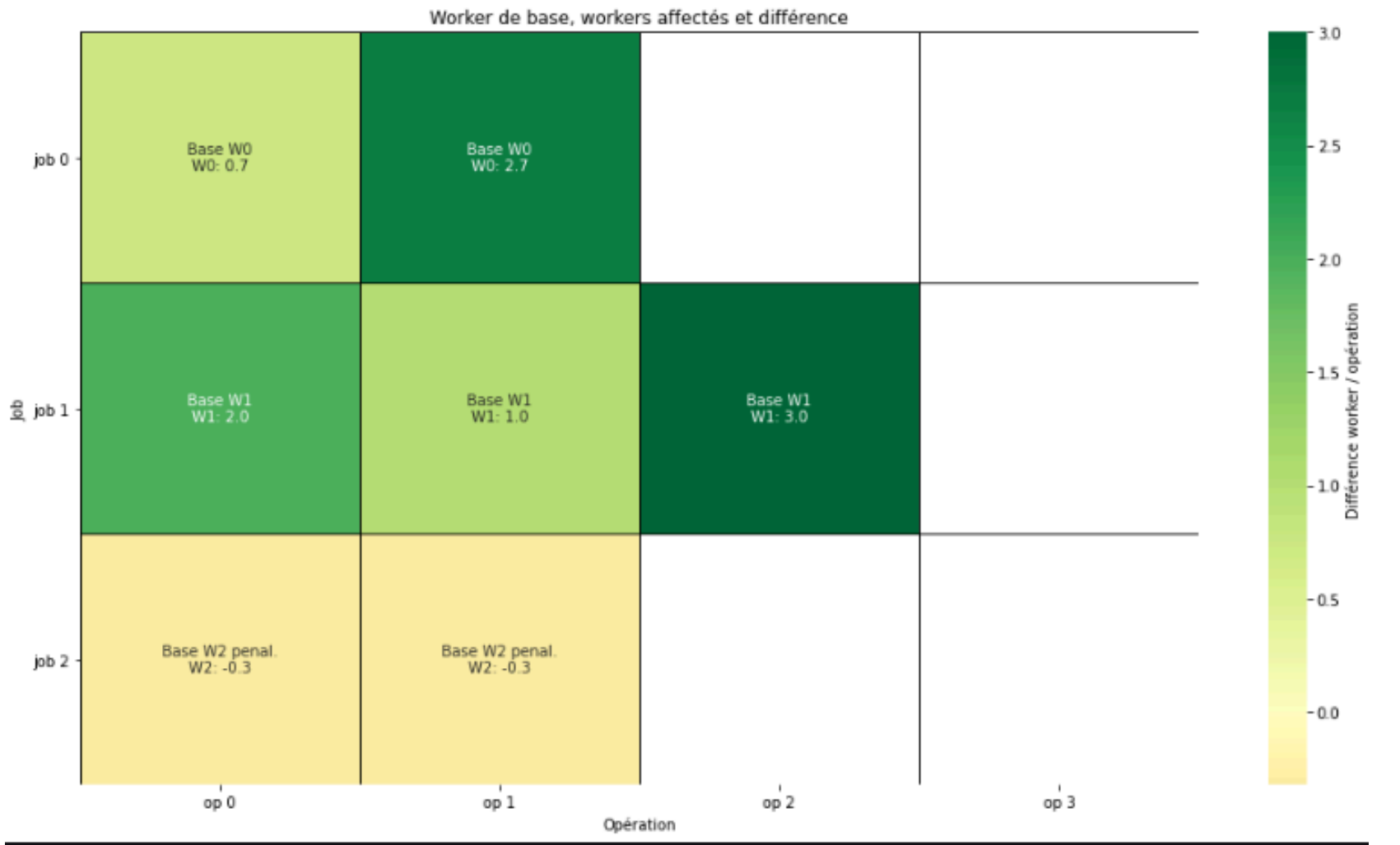
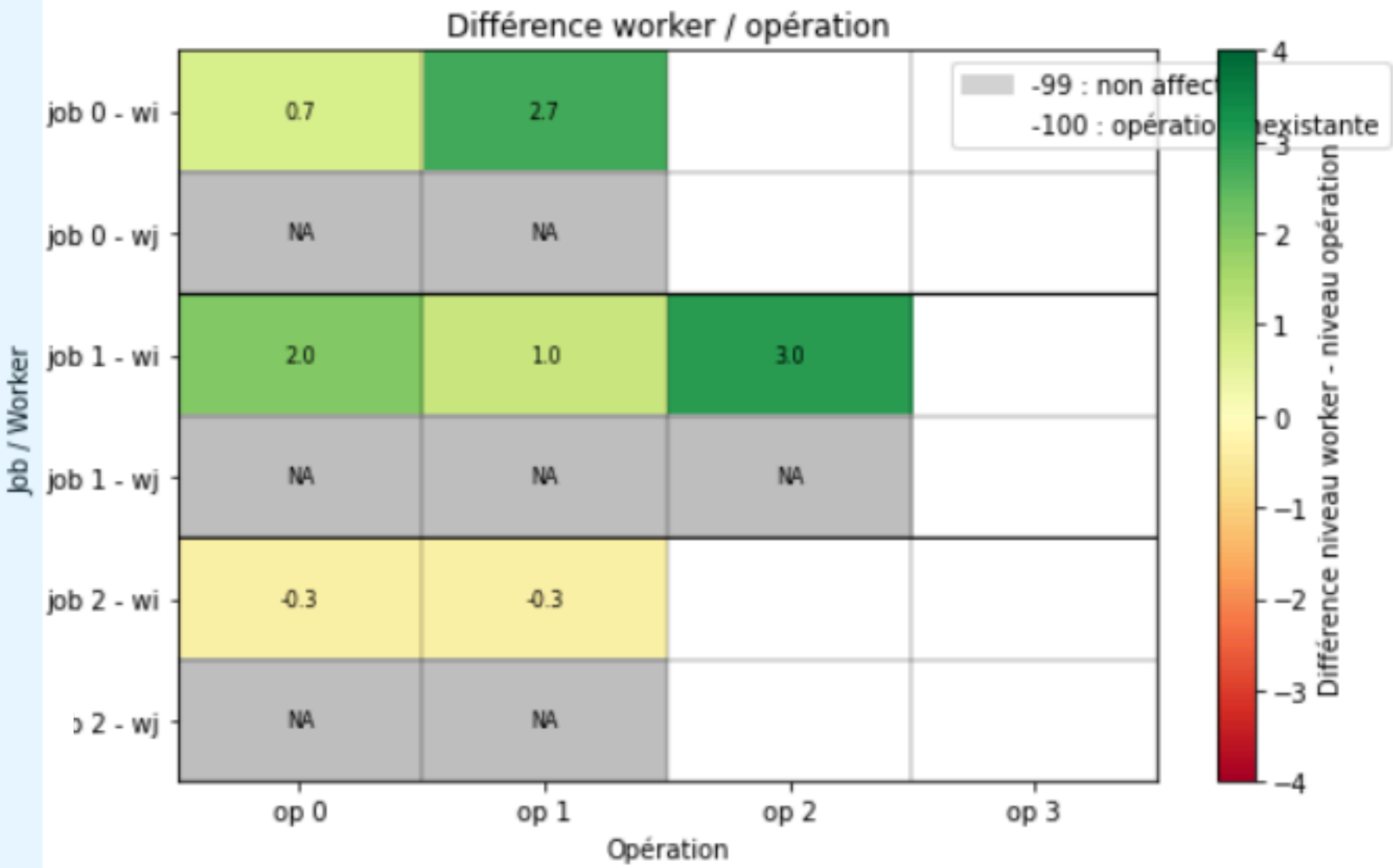
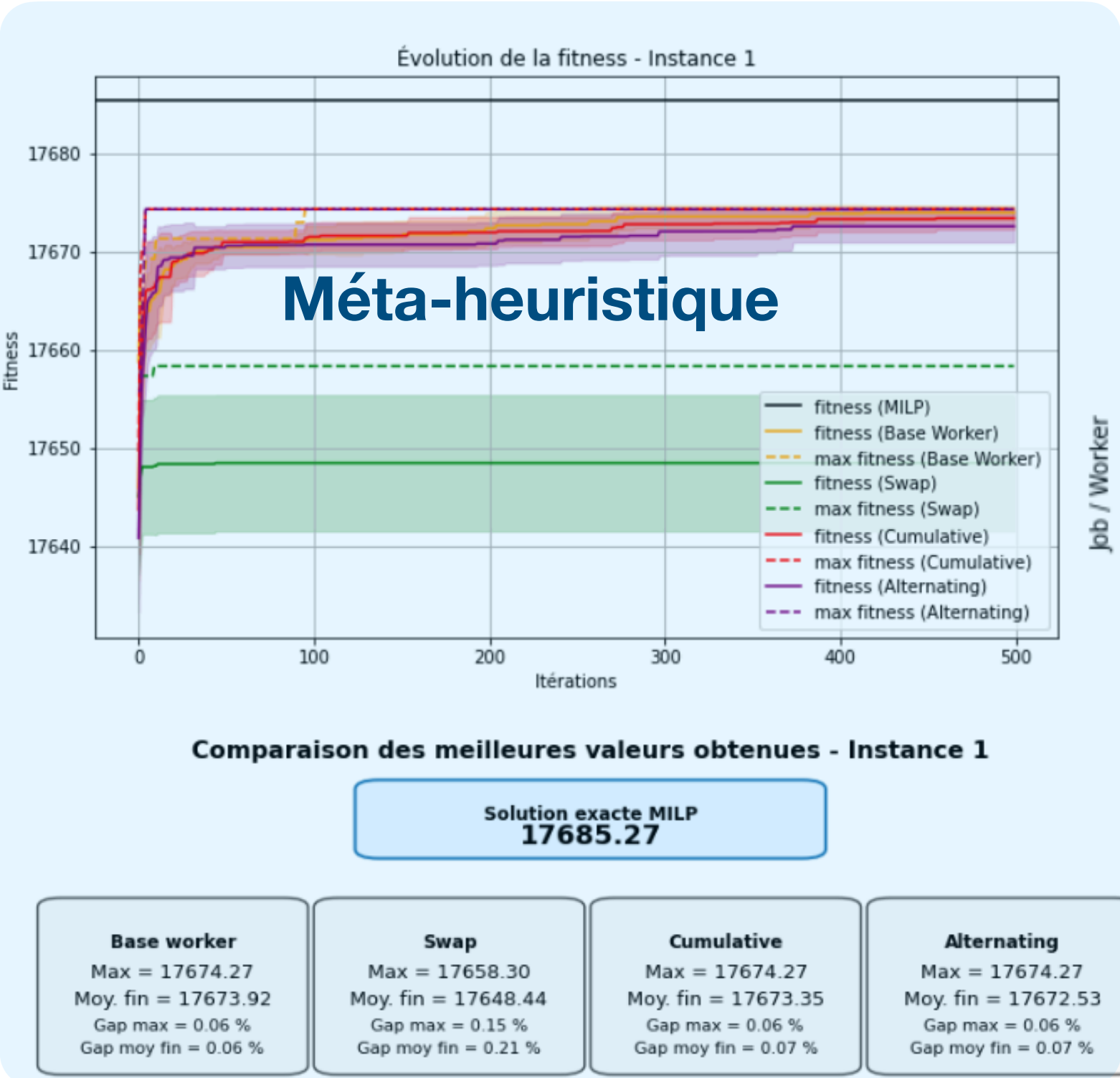
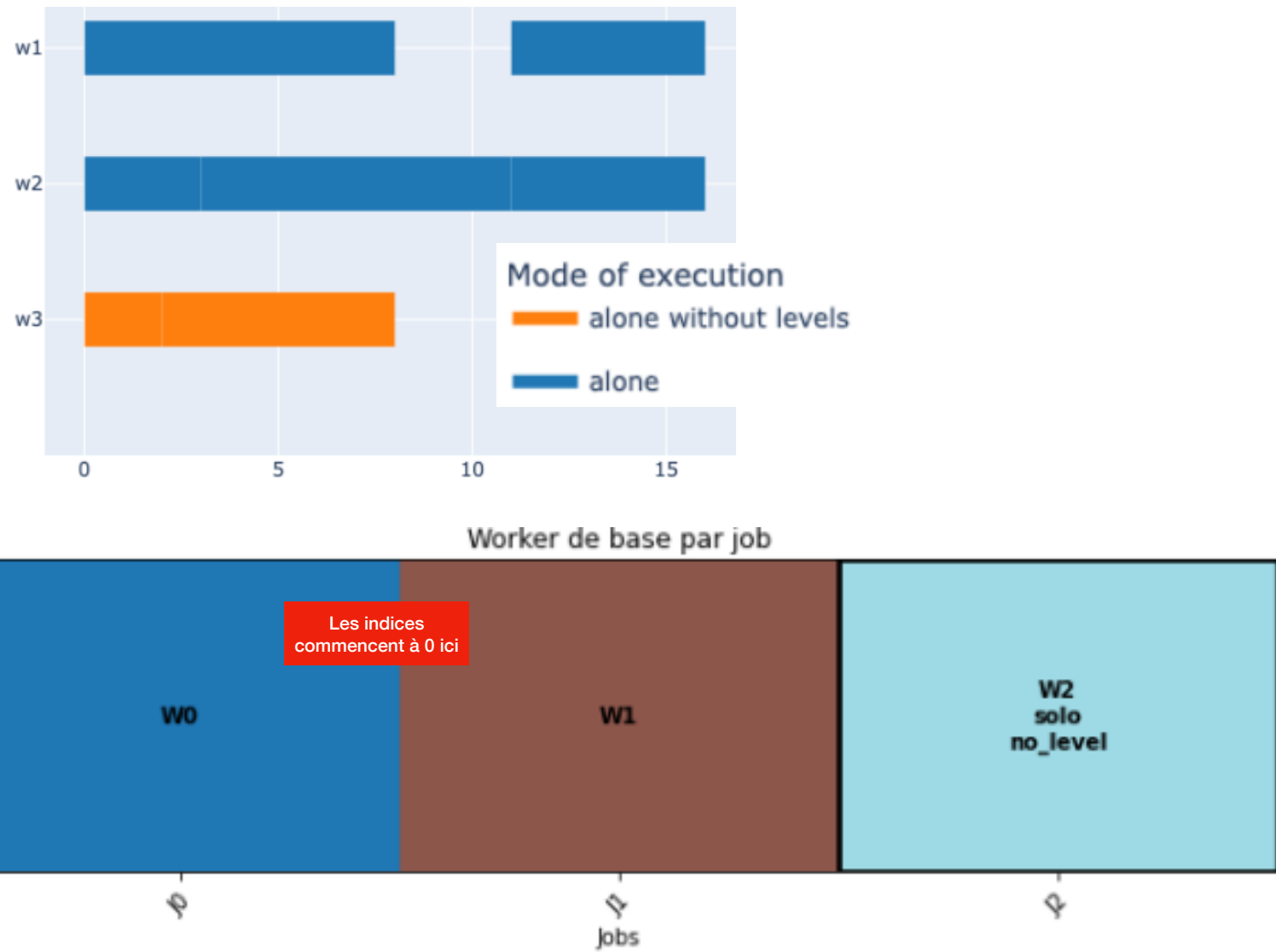
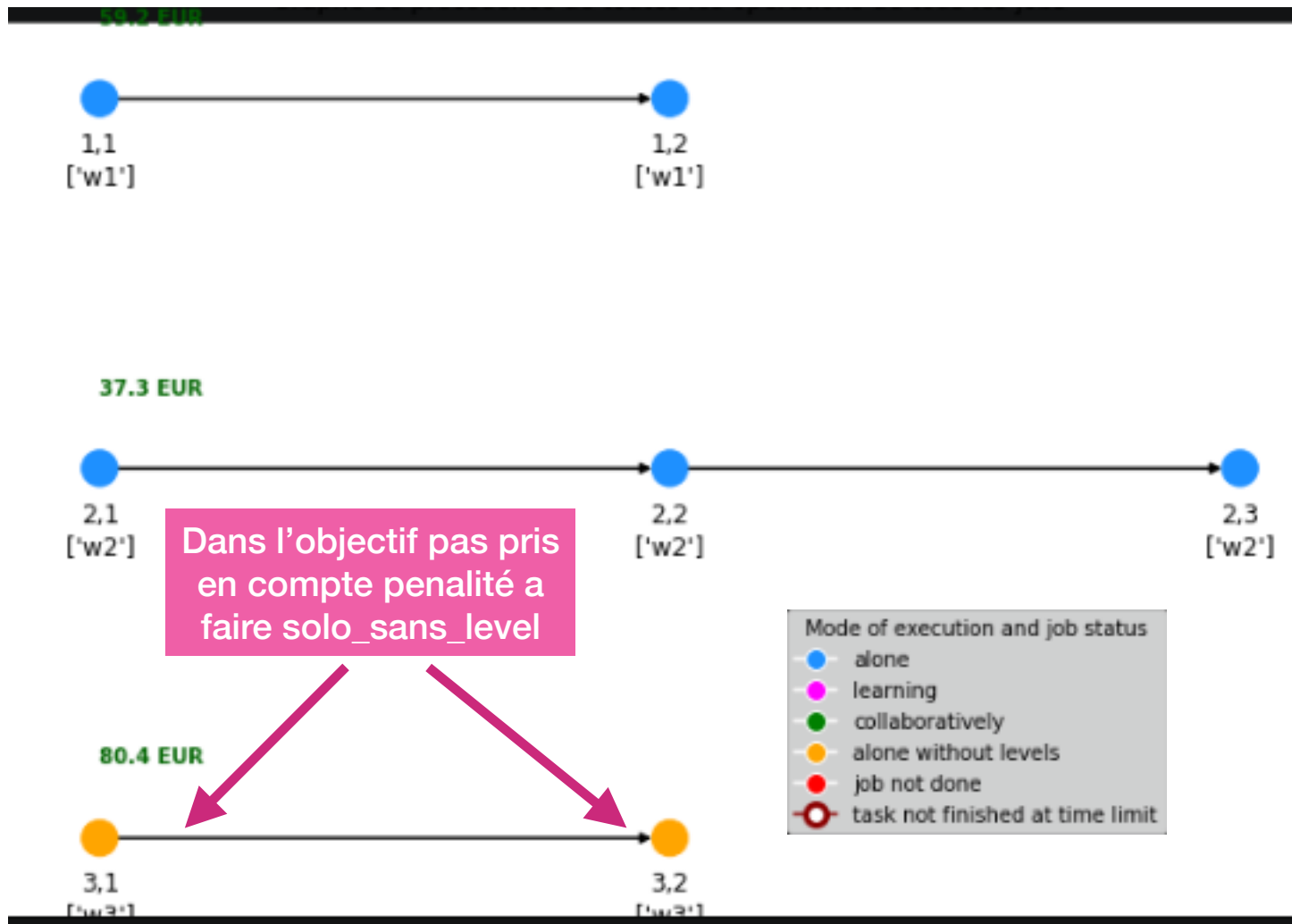
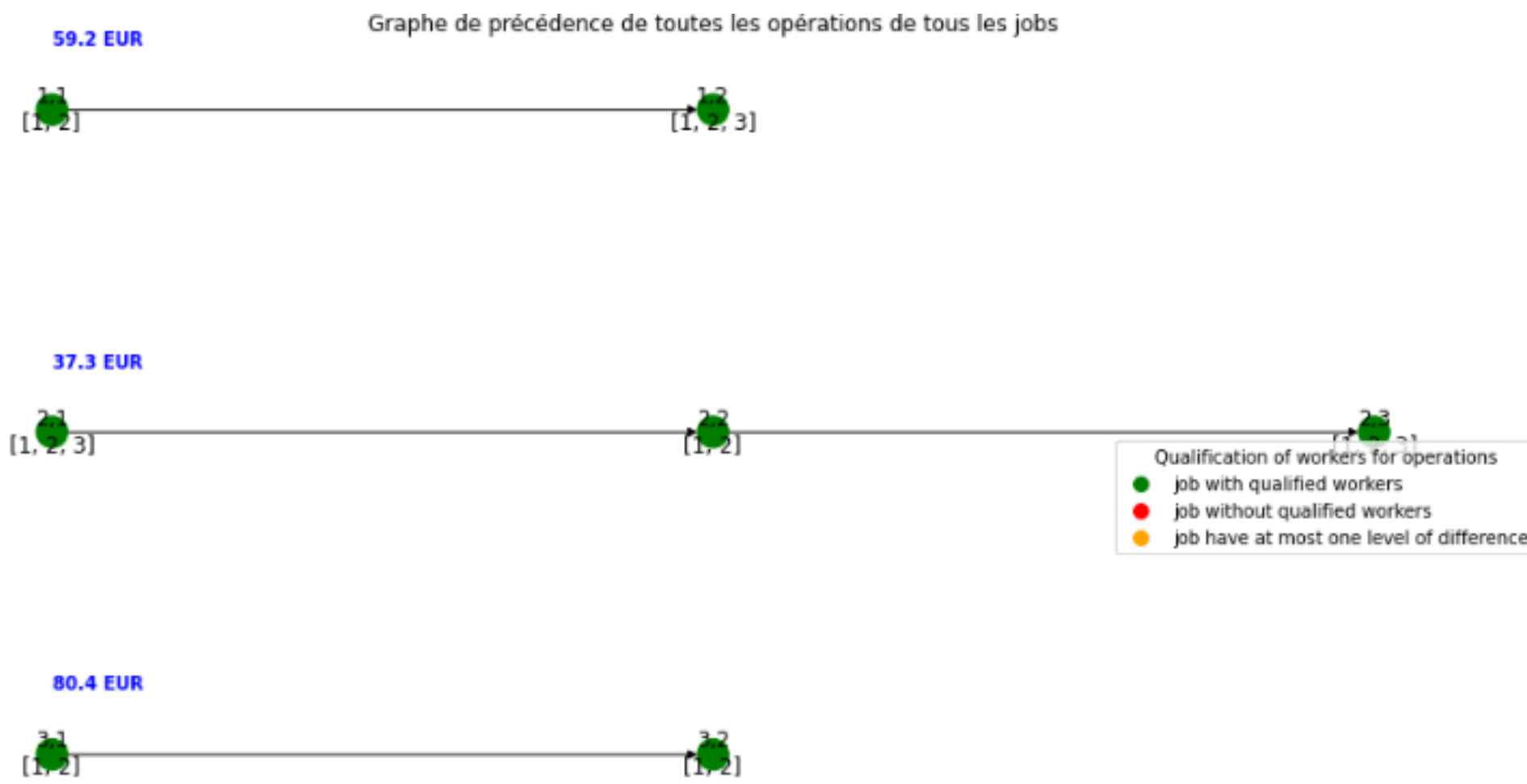
Alternating
Max = 33236.49
Moy. fin = 33236.49
Gap max = 0.06 %
Gap moy fin = 0.06 %



MILP MODEL

instance : 1
profit : 176.91265678978039
skills : 1.0
cognitive_load : 0.0
Cmax : 16.0
17685.265678978038

Instance 1



Comparaison des meilleures valeurs obtenues - Instance 1

Instance 1

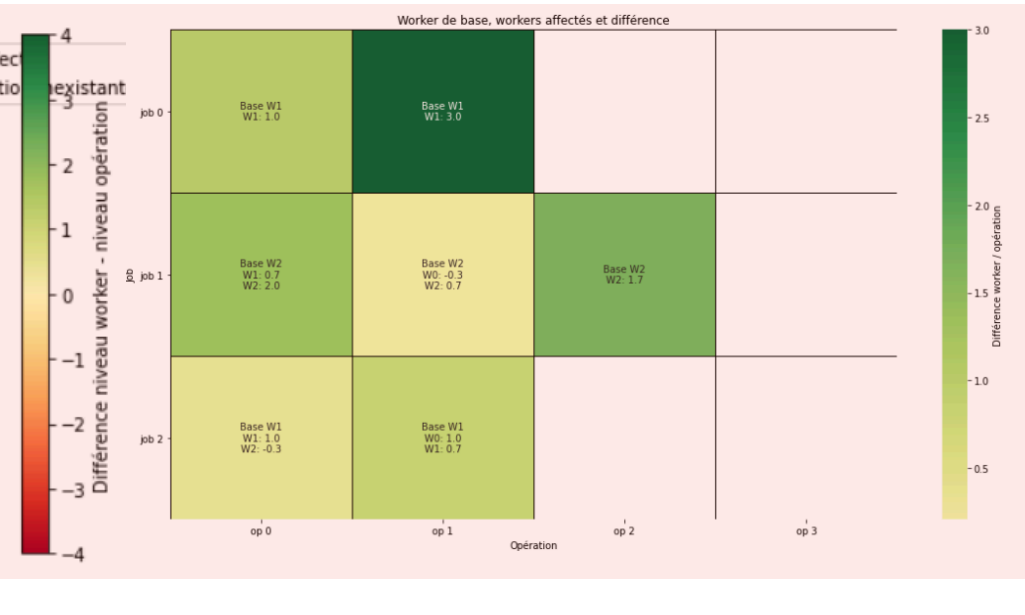
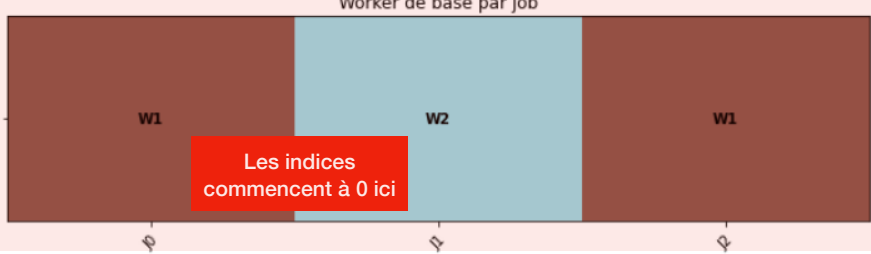
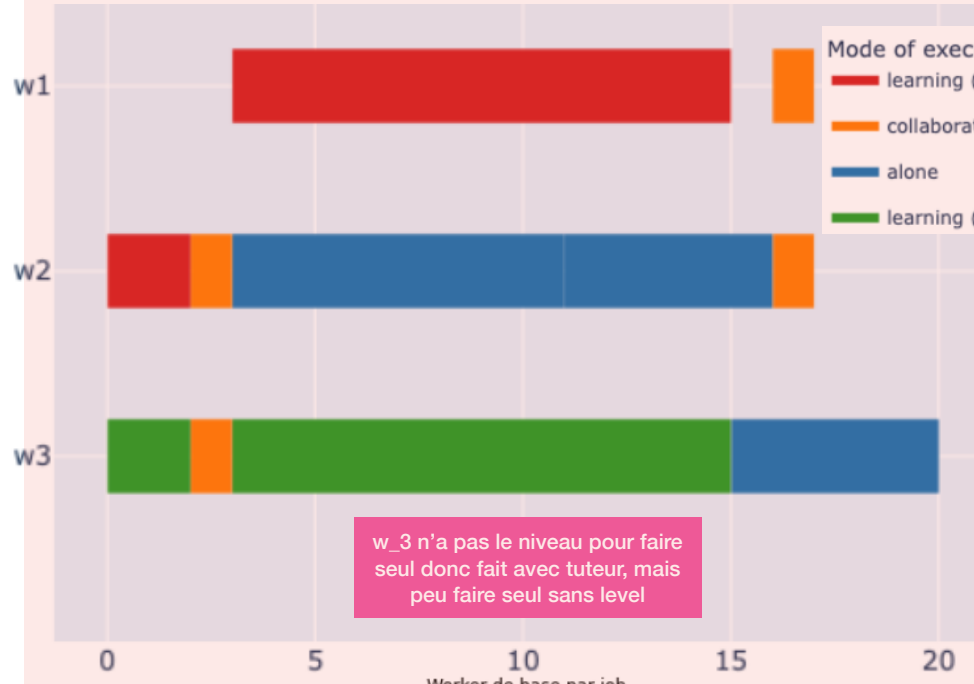
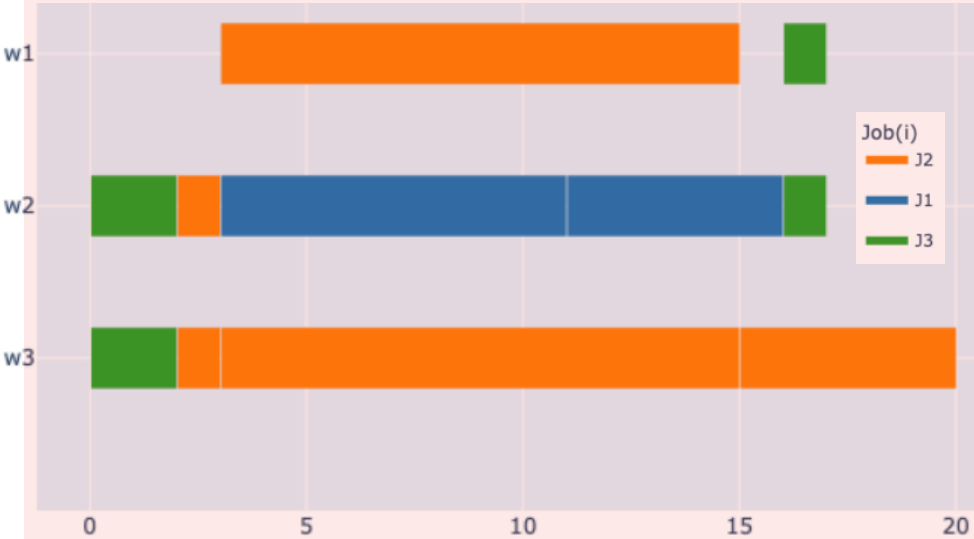
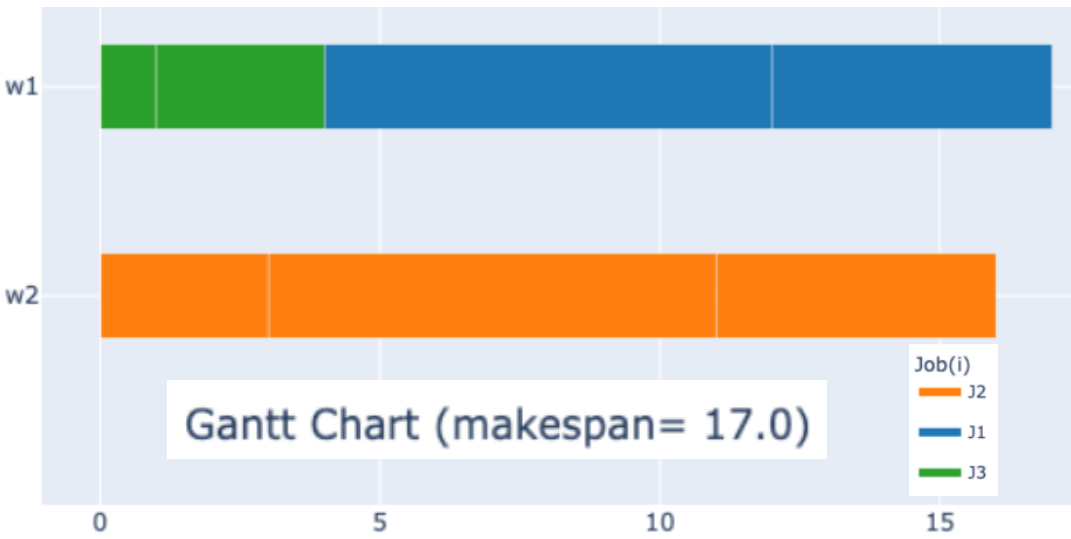
Solution exacte MILP
17685.27

Base worker
Max = 17674.27
Moy. fin = 17673.92
Gap max = 0.06 %
Gap moy fin = 0.06 %

Swap
Max = 17658.30
Moy. fin = 17648.44
Gap max = 0.15 %
Gap moy fin = 0.21 %

Cumulative
Max = 17674.27
Moy. fin = 17673.35
Gap max = 0.06 %
Gap moy fin = 0.07 %

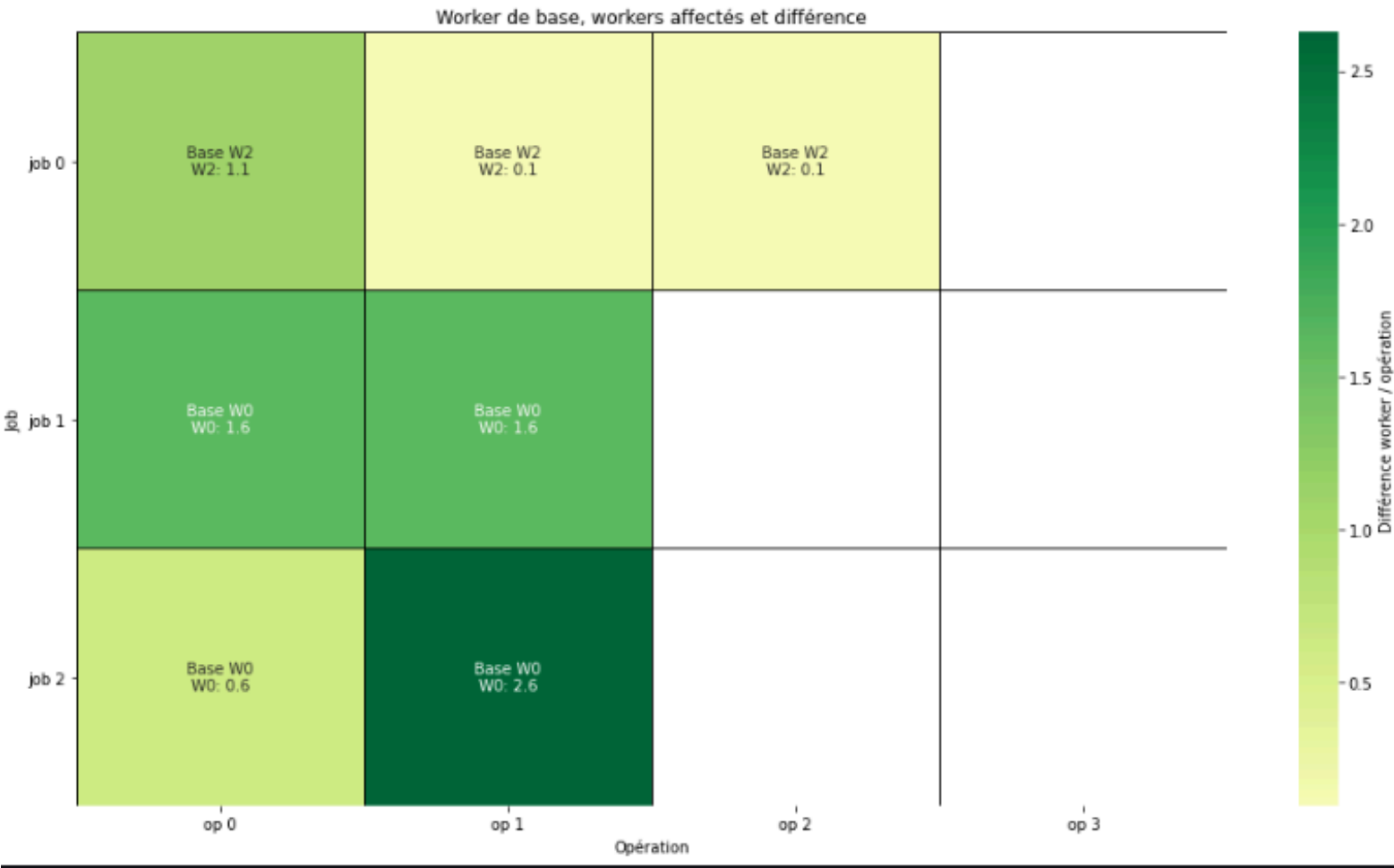
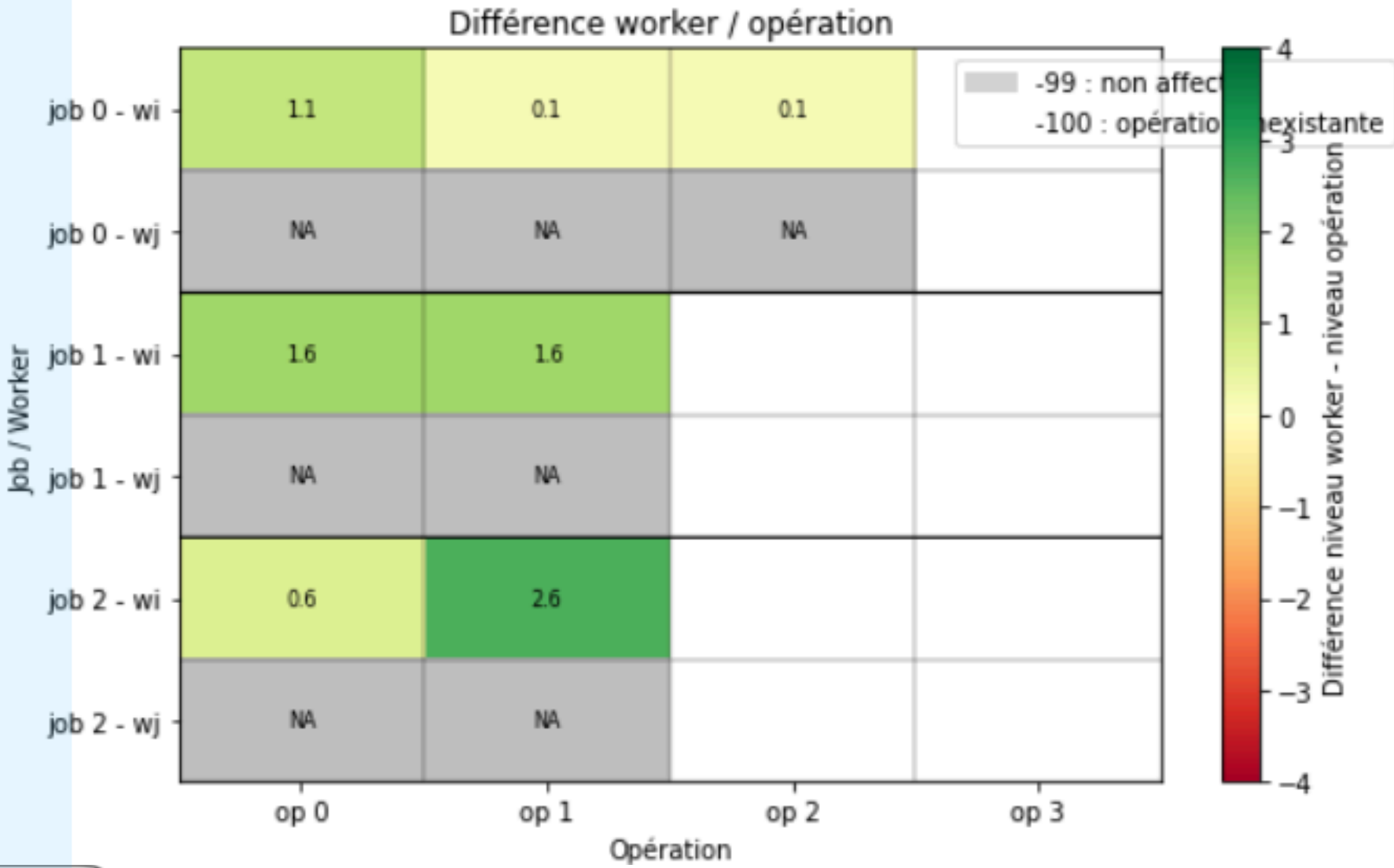
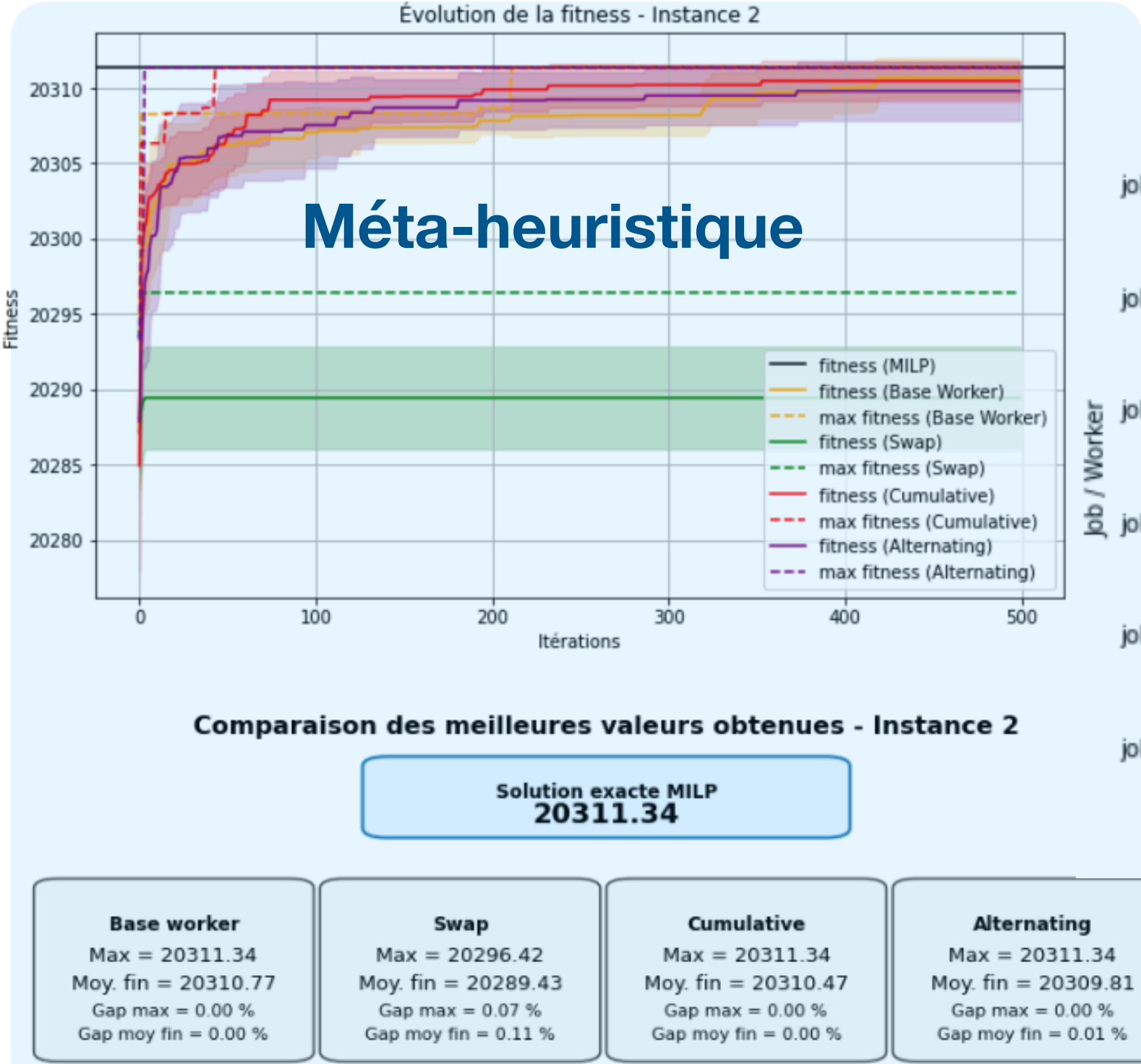
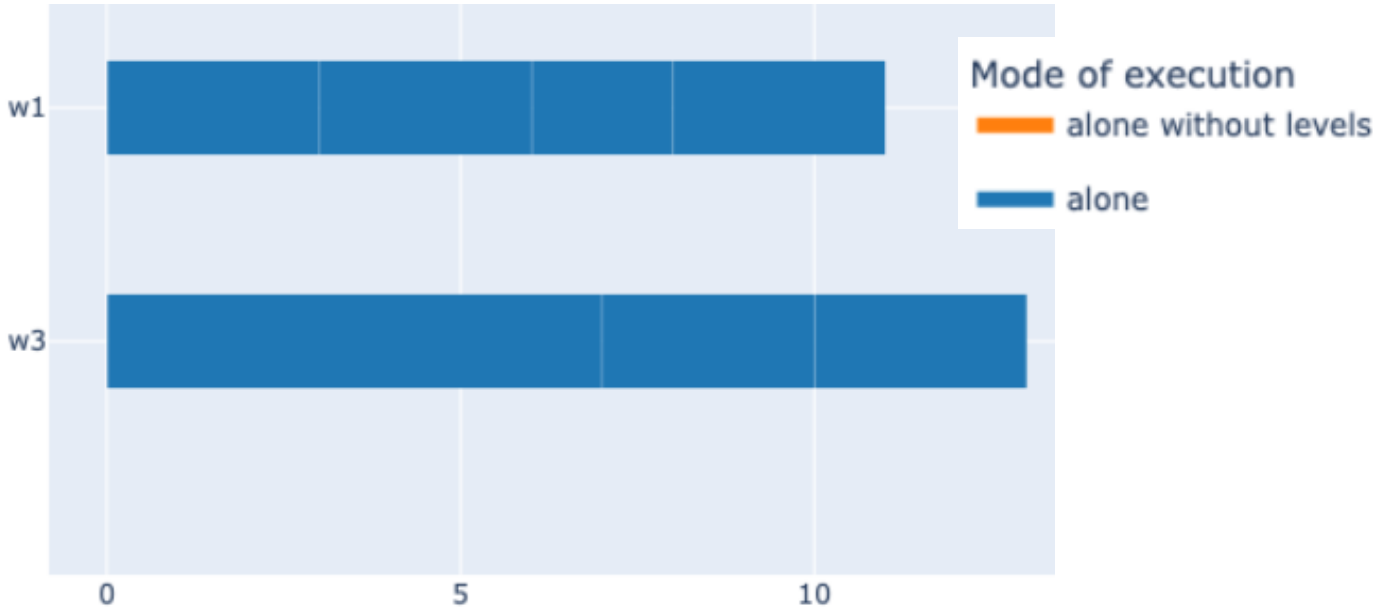
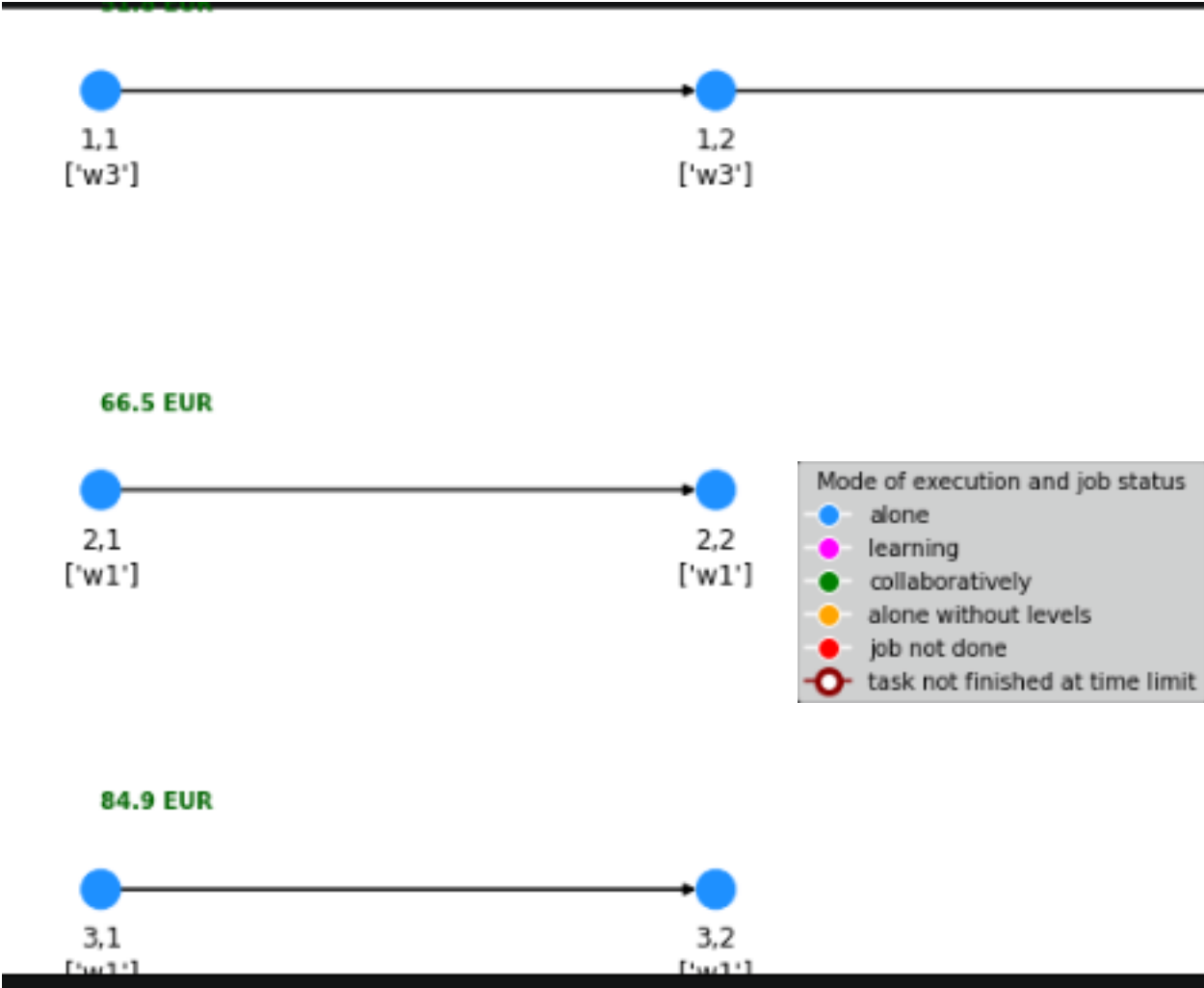
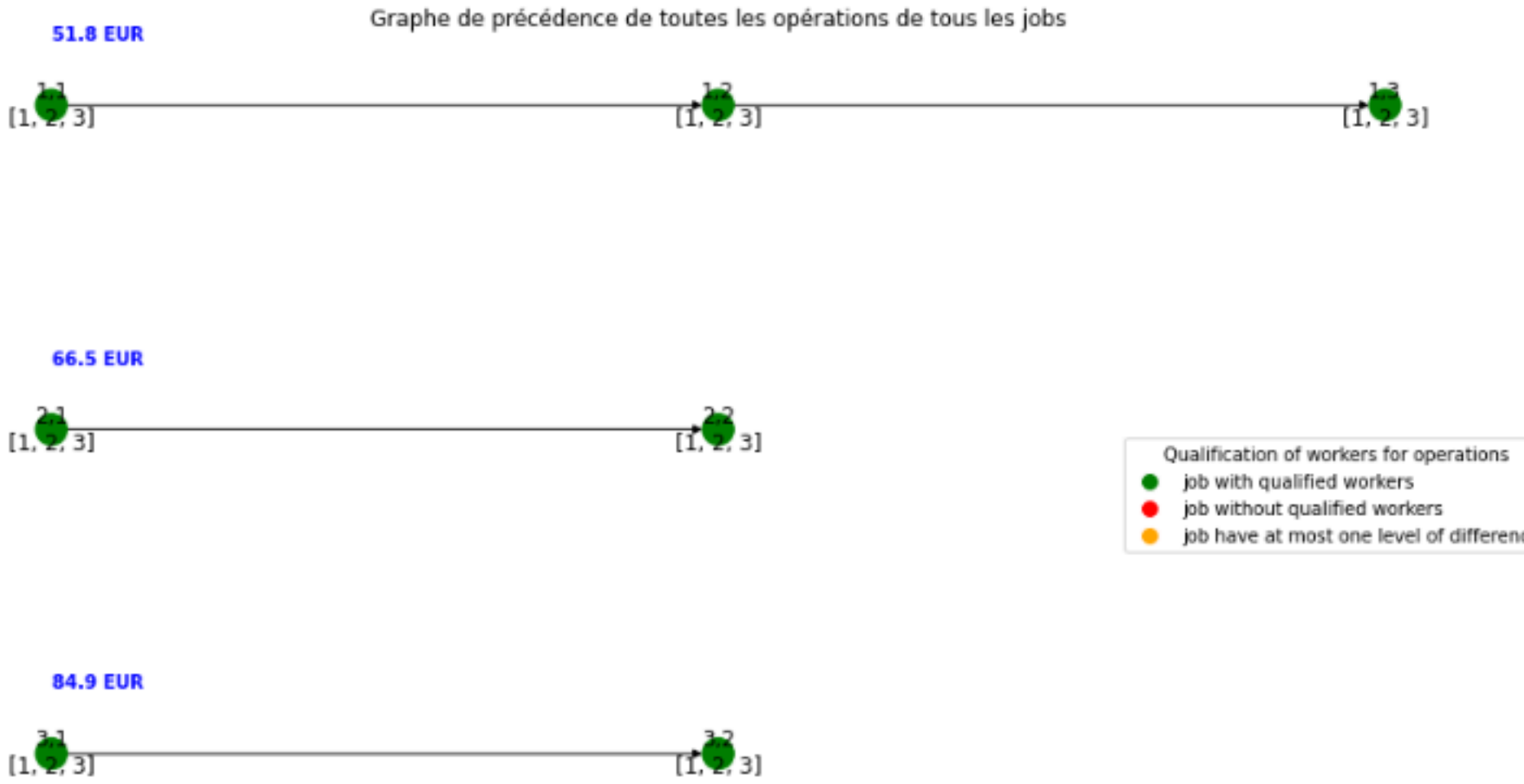
Alternating
Max = 17674.27
Moy. fin = 17672.53
Gap max = 0.06 %
Gap moy fin = 0.07 %



MILP MODEL

instance : 2
profit : 203.24341837462248
skills : 0.0
cognitive_load : 0.0
Cmax : 13.0
20311.34183746225

Instance 2



Comparaison des meilleures valeurs obtenues - Instance 2

Instance 2

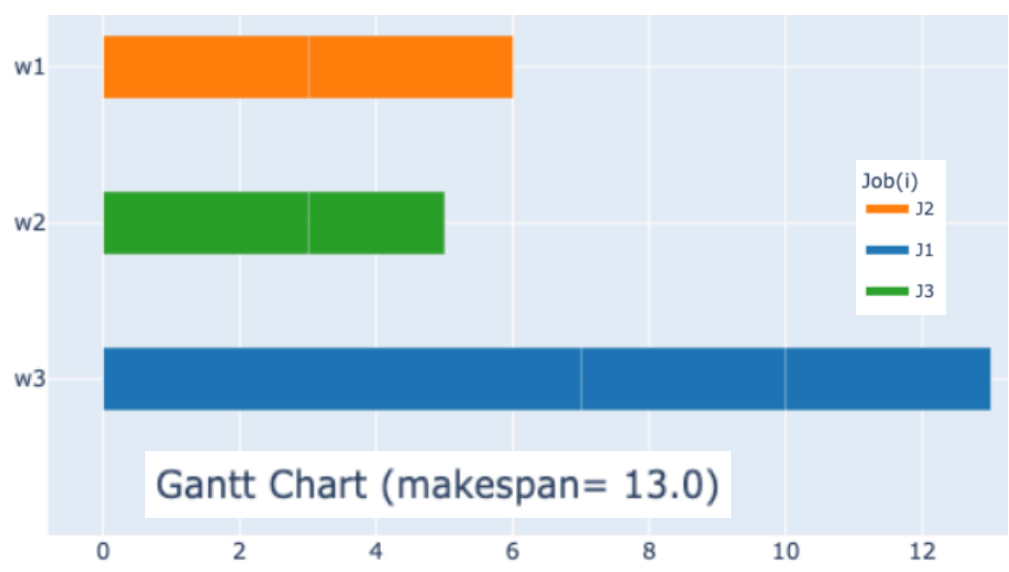
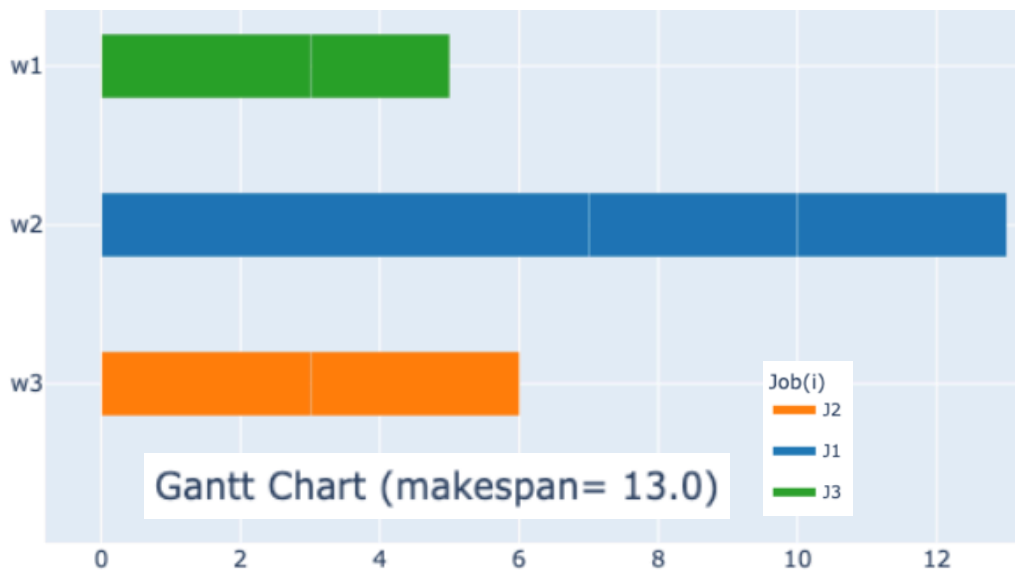
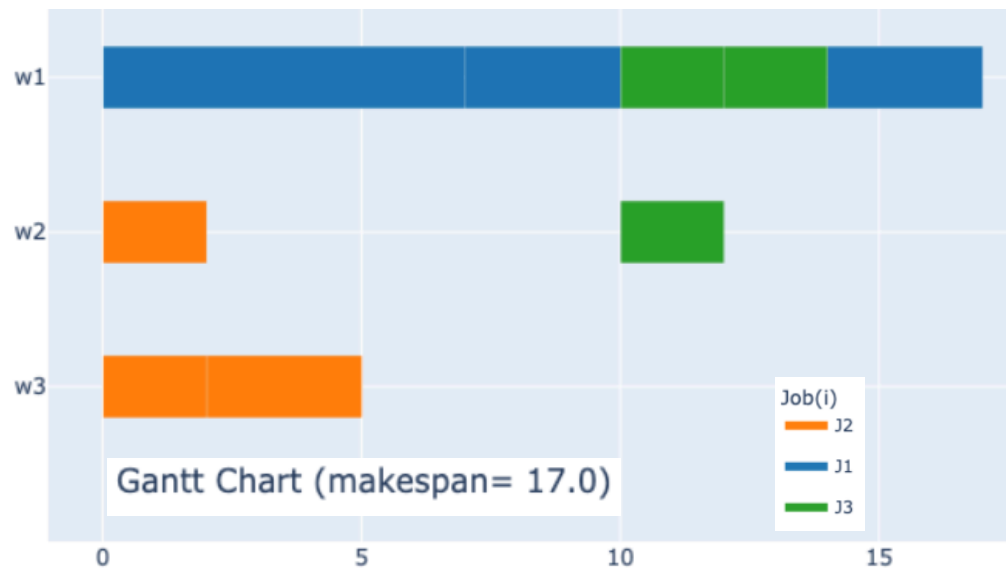
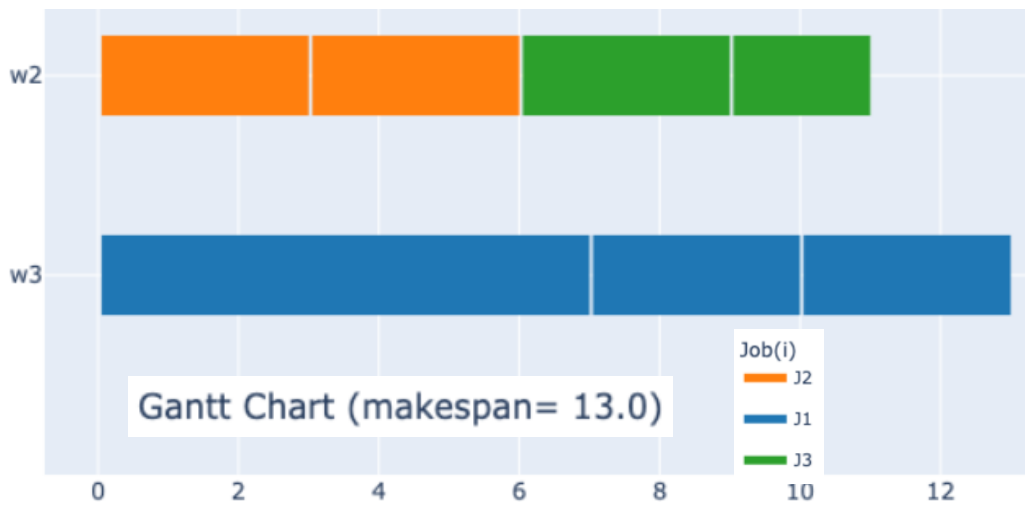
Solution exacte MILP
20311.34

Base worker
Max = 20311.34
Moy. fin = 20310.77
Gap max = 0.00 %
Gap moy fin = 0.00 %

Swap
Max = 20296.42
Moy. fin = 20289.43
Gap max = 0.07 %
Gap moy fin = 0.11 %

Cumulative
Max = 20311.34
Moy. fin = 20310.47
Gap max = 0.00 %
Gap moy fin = 0.00 %

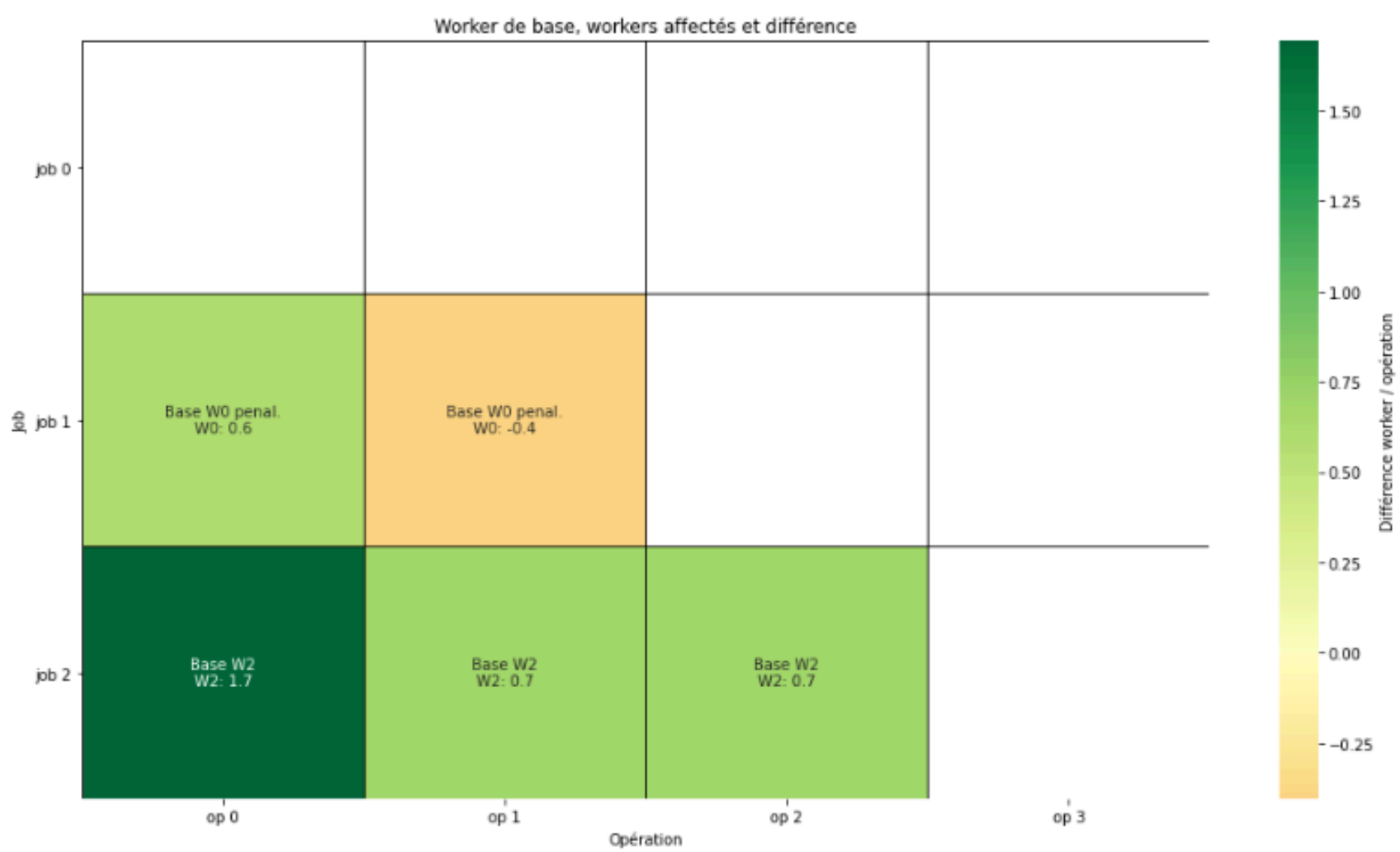
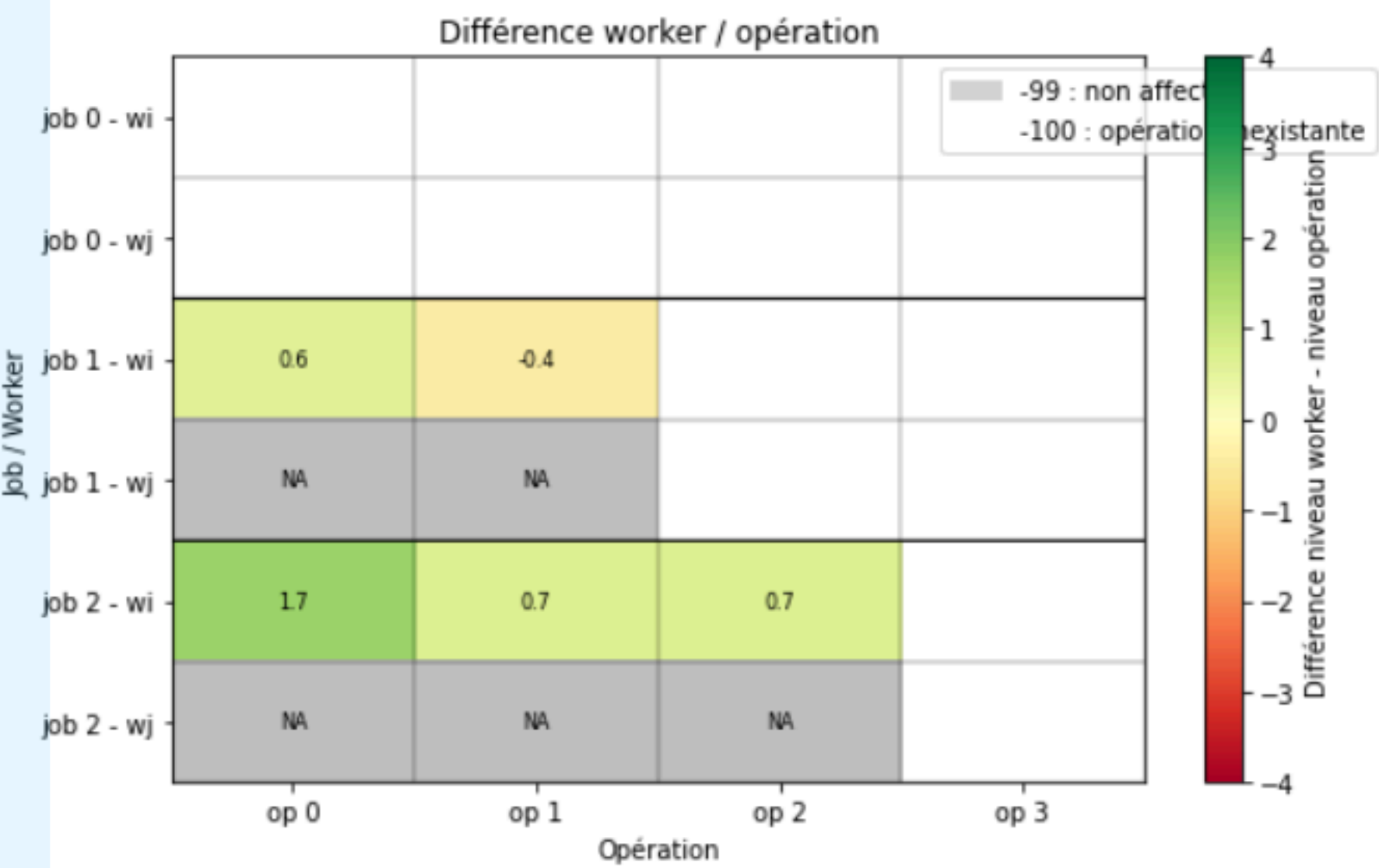
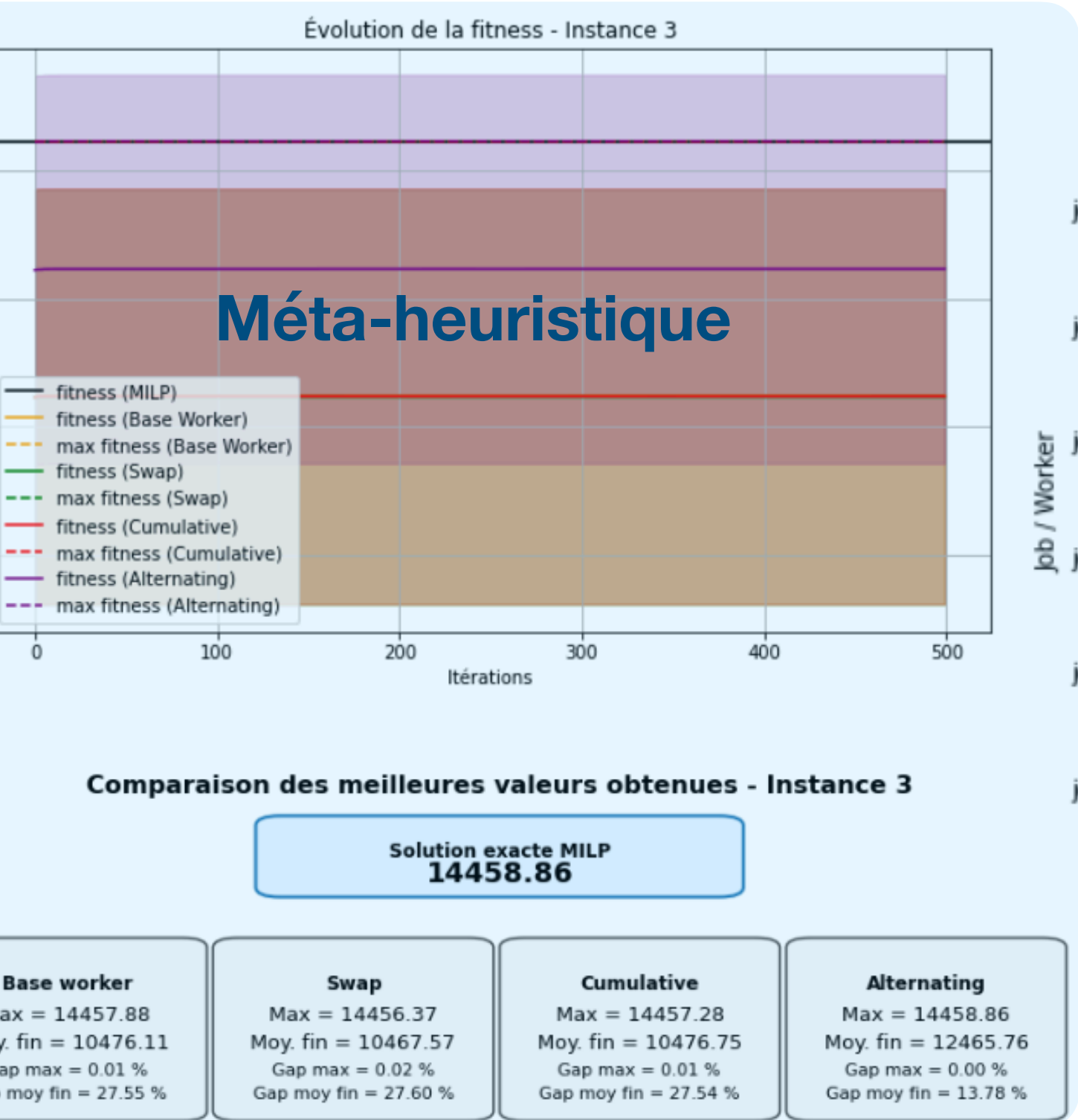
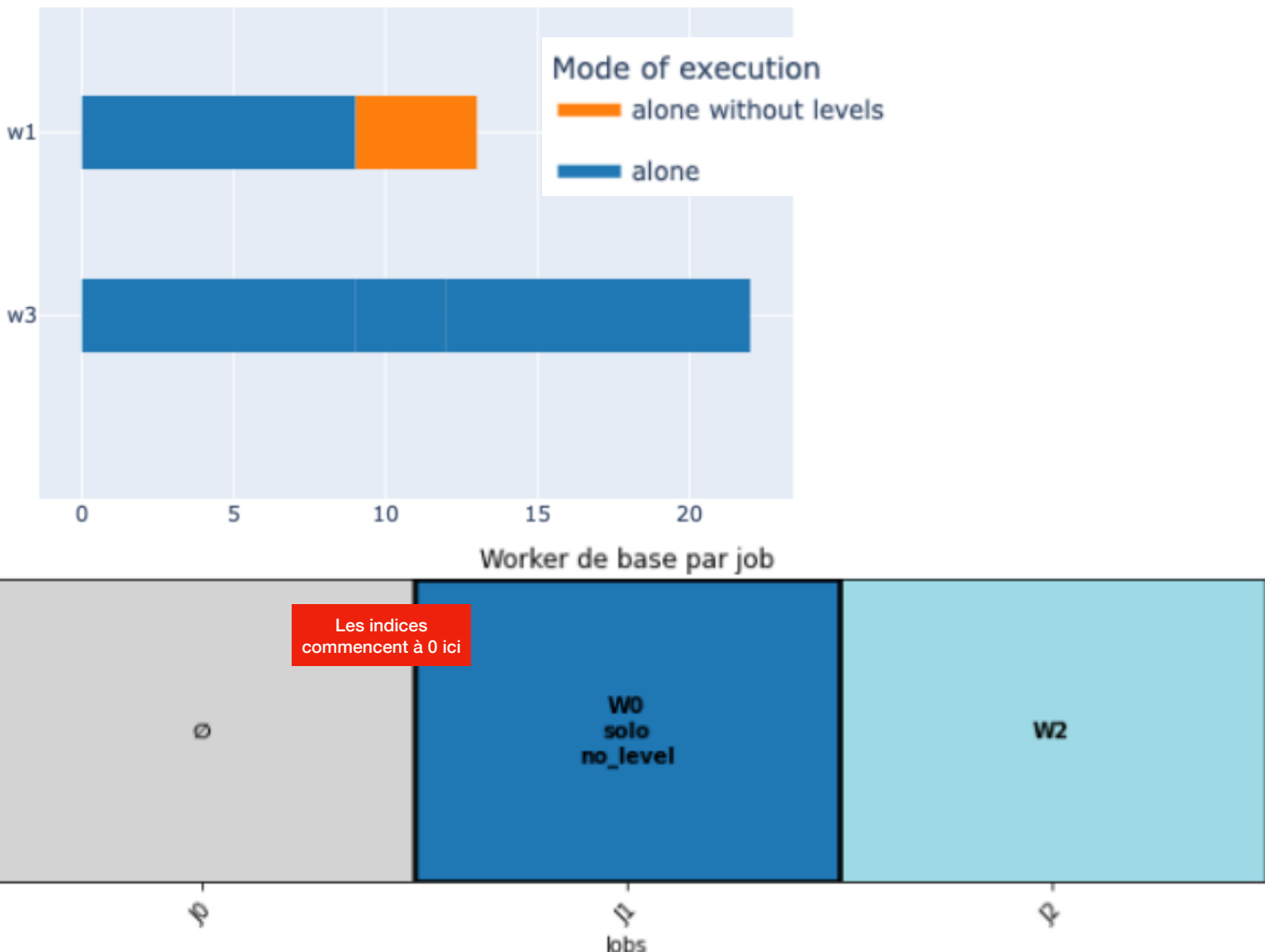
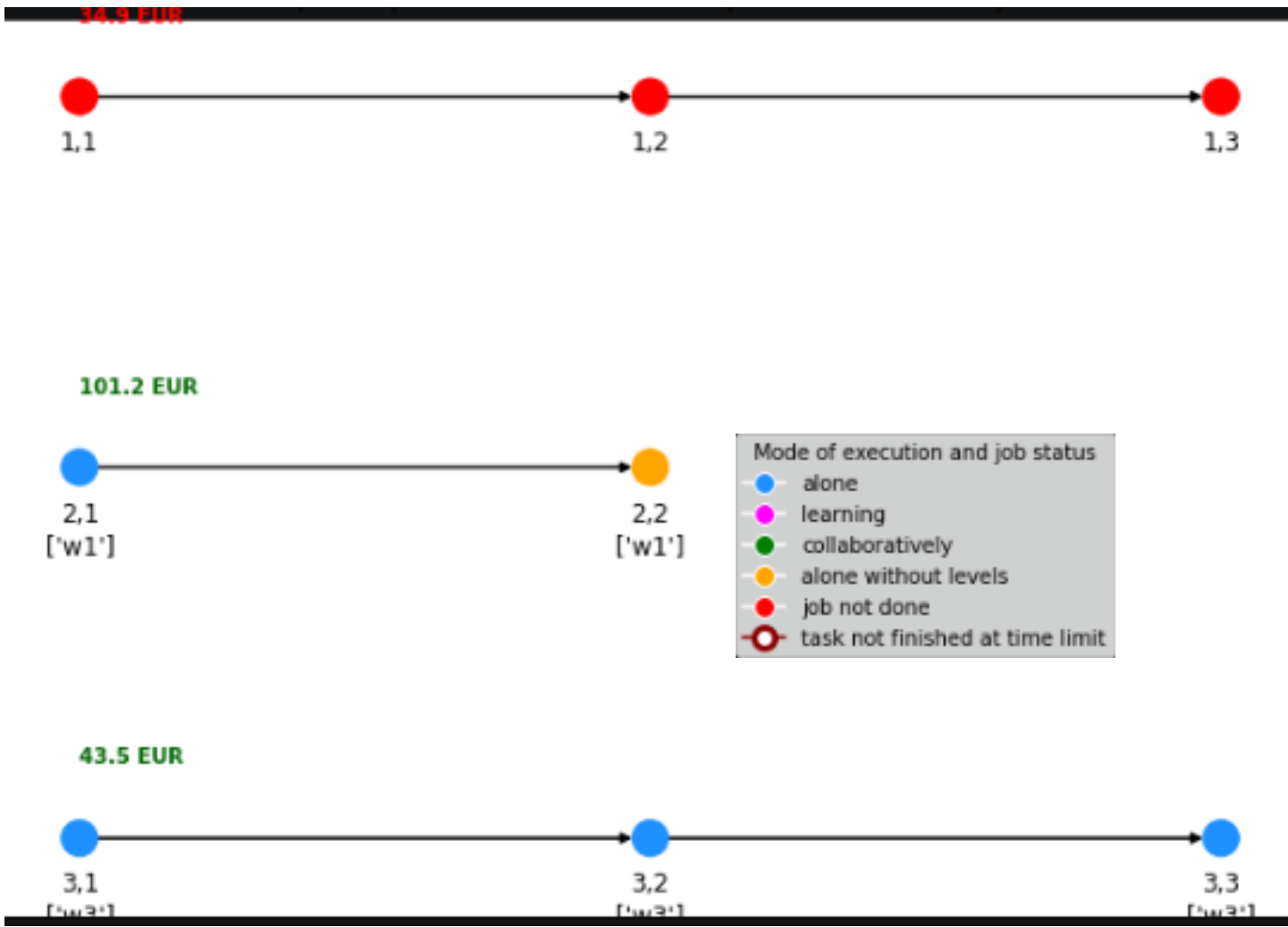
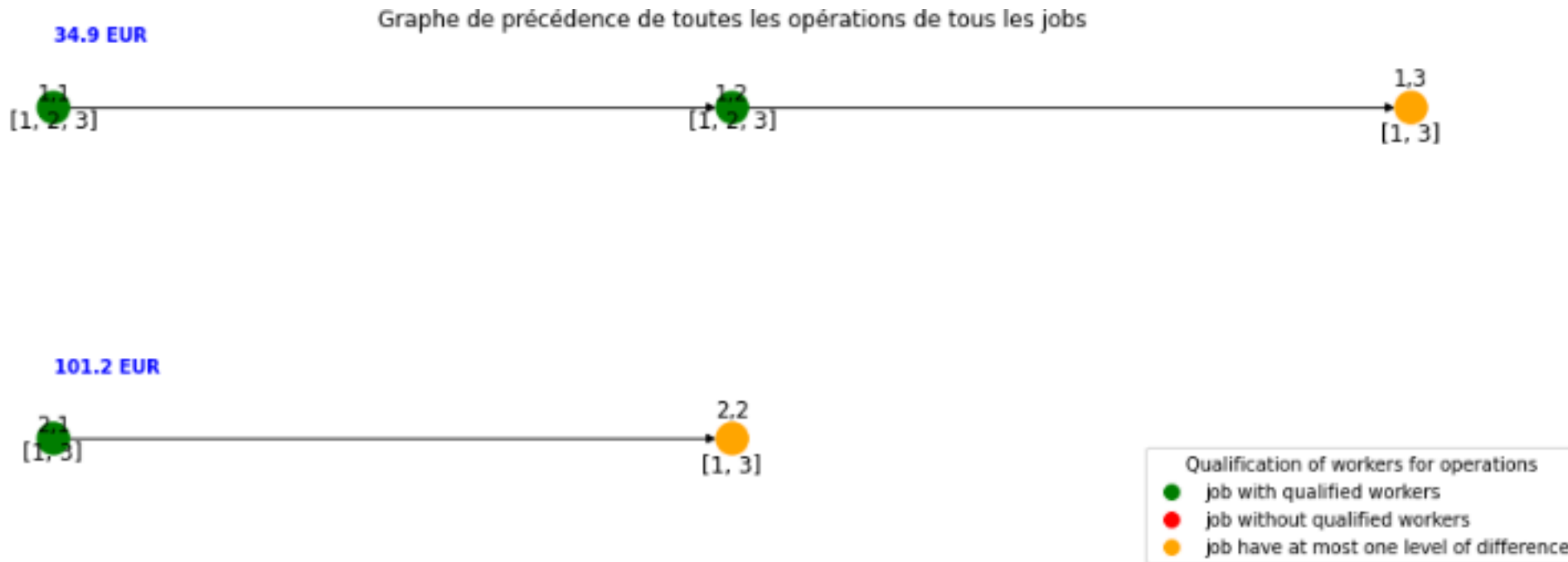
Alternating
Max = 20311.34
Moy. fin = 20309.81
Gap max = 0.00 %
Gap moy fin = 0.01 %



MILP MODEL

instance : 3
profit : 144.76806776826345
skills : 0.4053544369325994
cognitive_load : 0.0
Cmax : 22.0
14458.86032119567

Instance 3



Comparaison des meilleures valeurs obtenues - Instance 3

Solution exacte MILP
14458.86

Base worker

Max = 14457.88
Moy. fin = 10476.11
Gap max = 0.01 %
Gap moy fin = 27.55 %

Swap

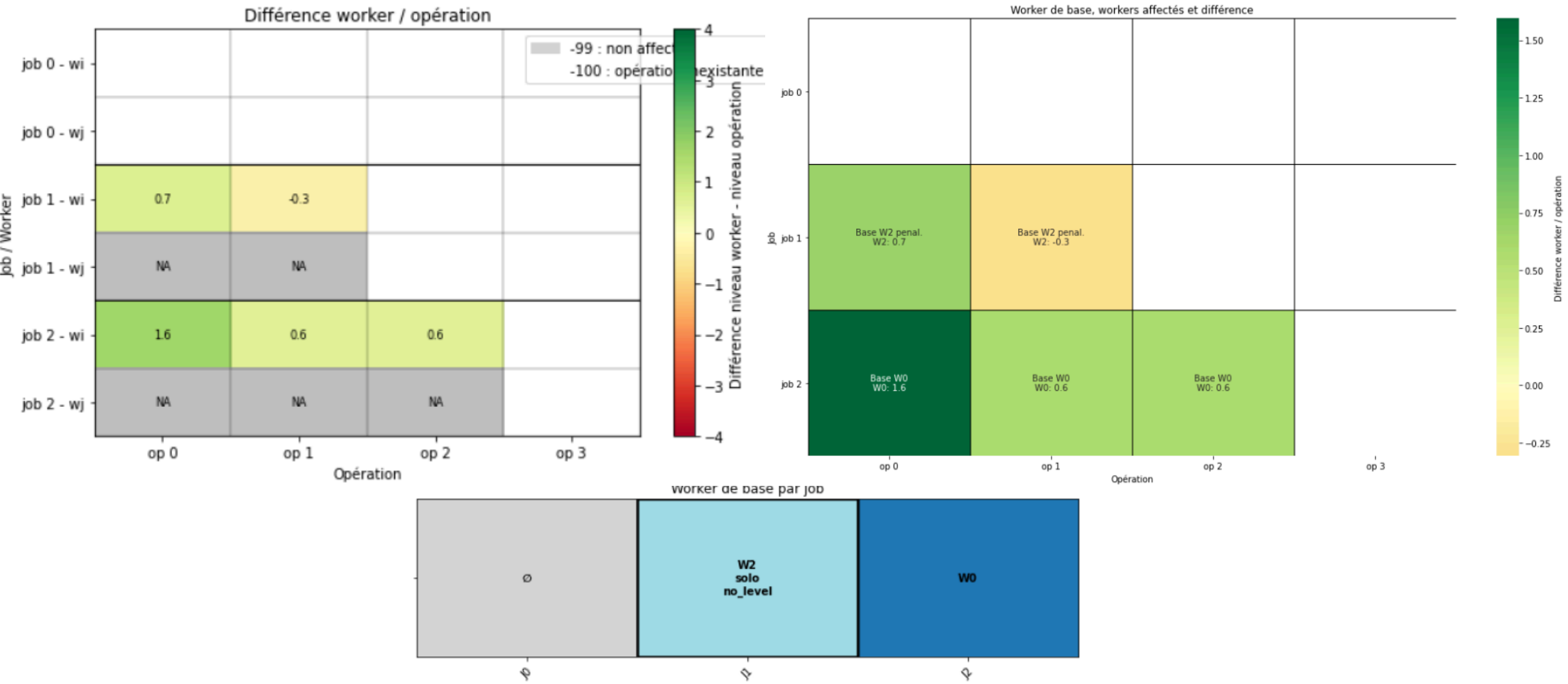
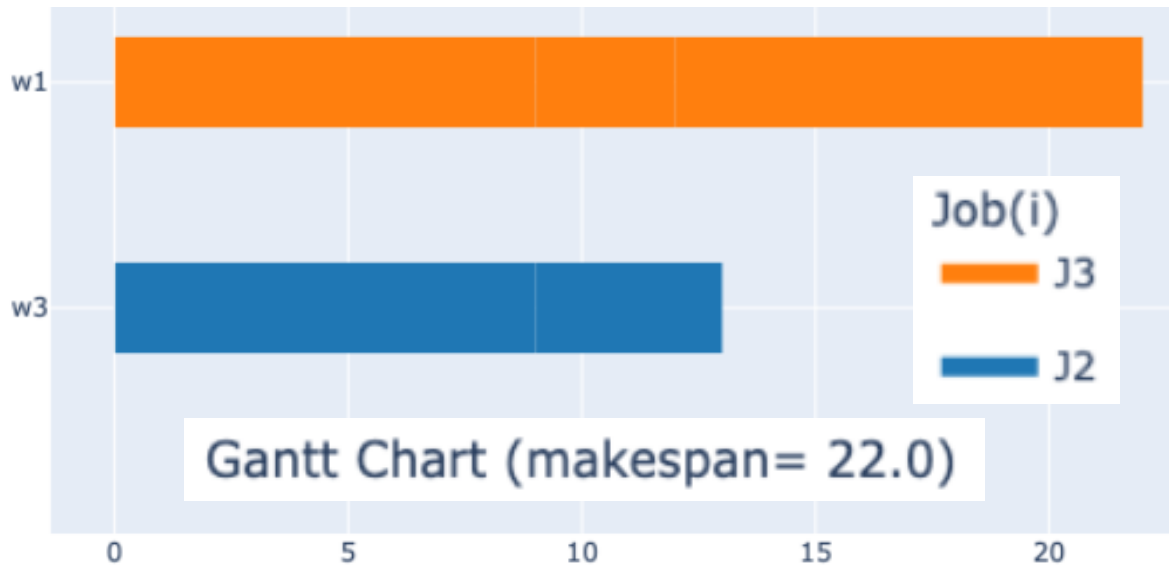
Max = 14456.37
Moy. fin = 10467.57
Gap max = 0.02 %
Gap moy fin = 27.60 %

Cumulative

Max = 14457.28
Moy. fin = 10476.75
Gap max = 0.01 %
Gap moy fin = 27.54 %

Alternating

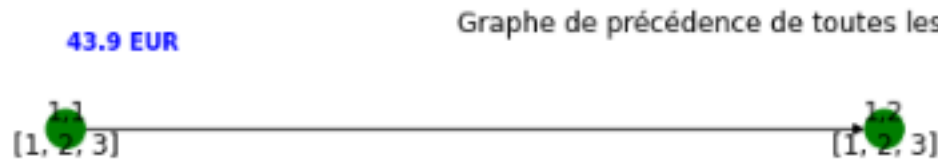
Max = 14458.86
Moy. fin = 12465.76
Gap max = 0.00 %
Gap moy fin = 13.78 %



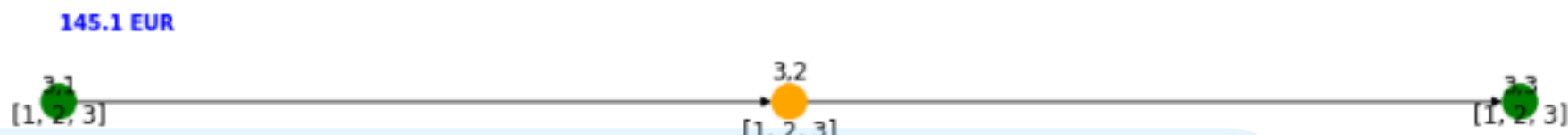
MILP MODEL

instance : 4
profit : 189.02667783439696
skills : 0.4732967096895724
cognitive_load : 0.0
Cmax : 32.0
18875.400750536595

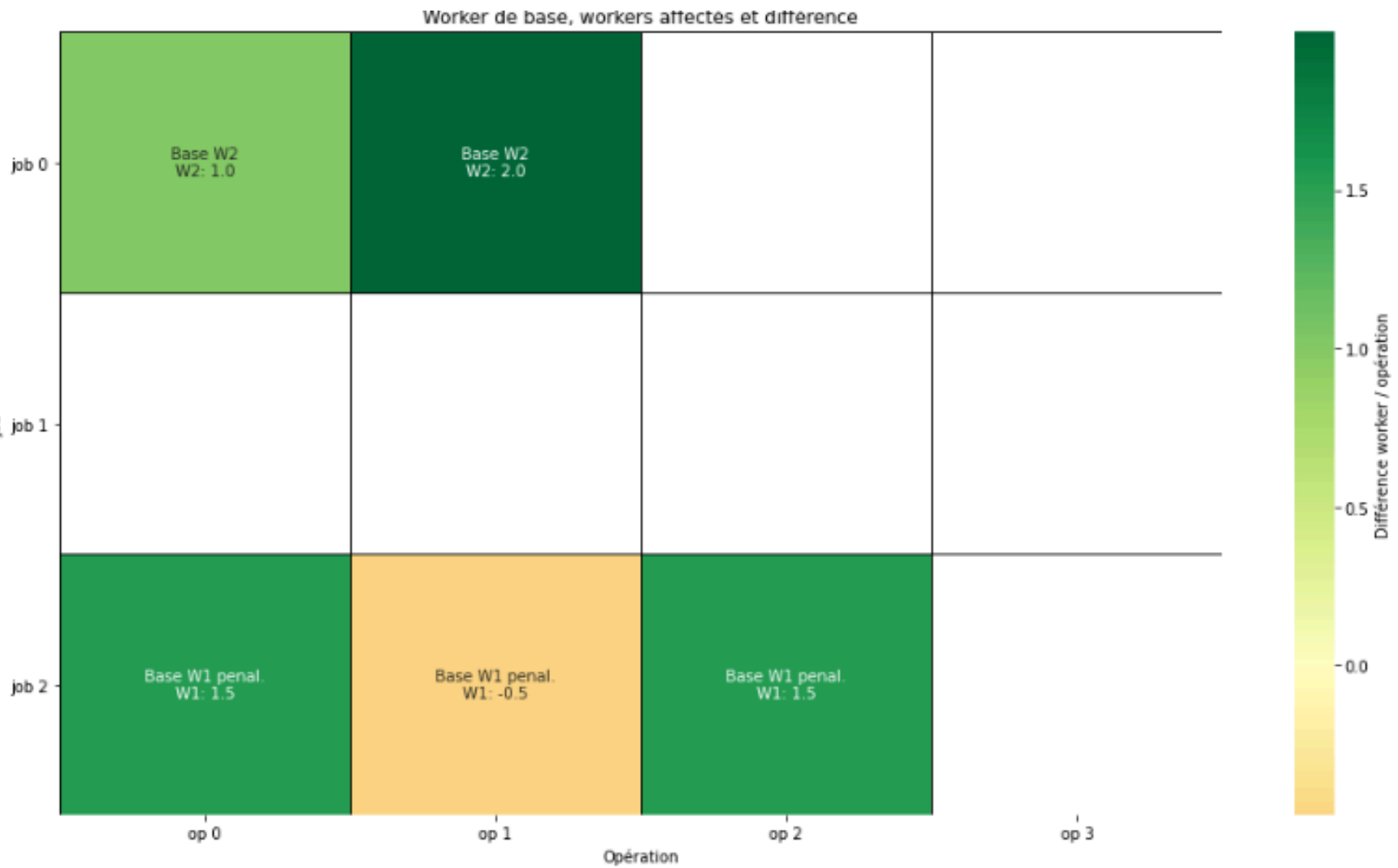
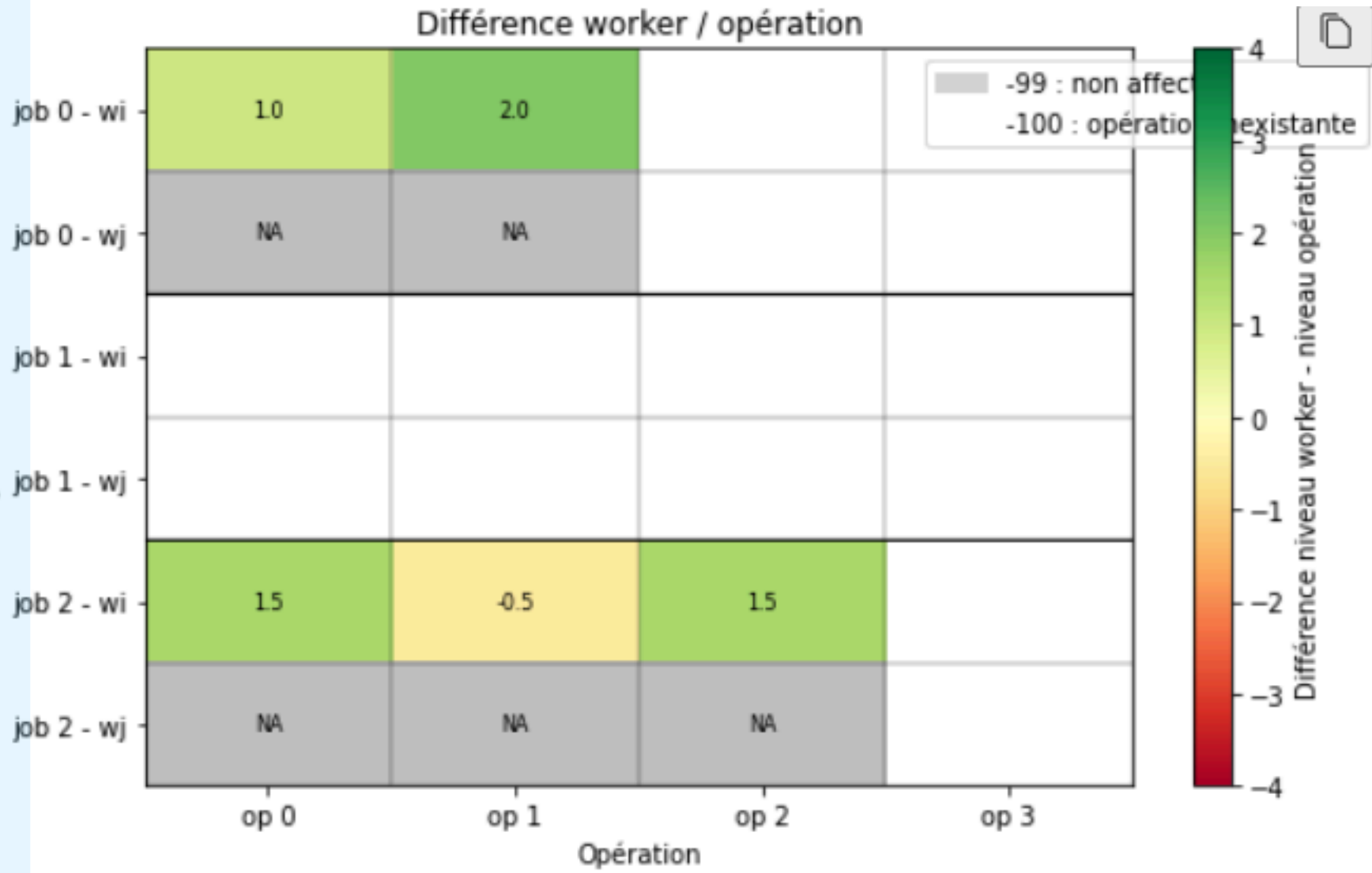
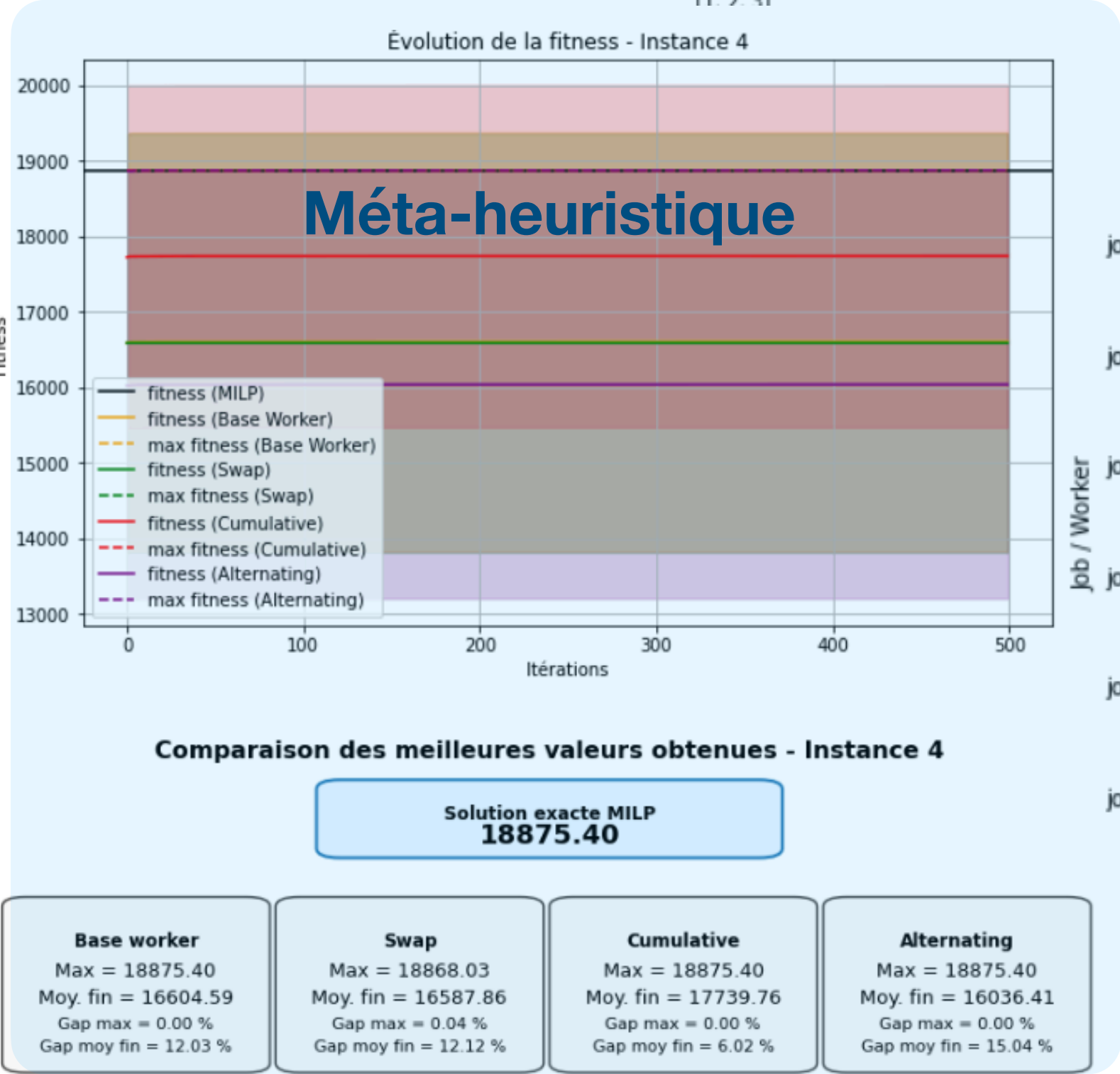
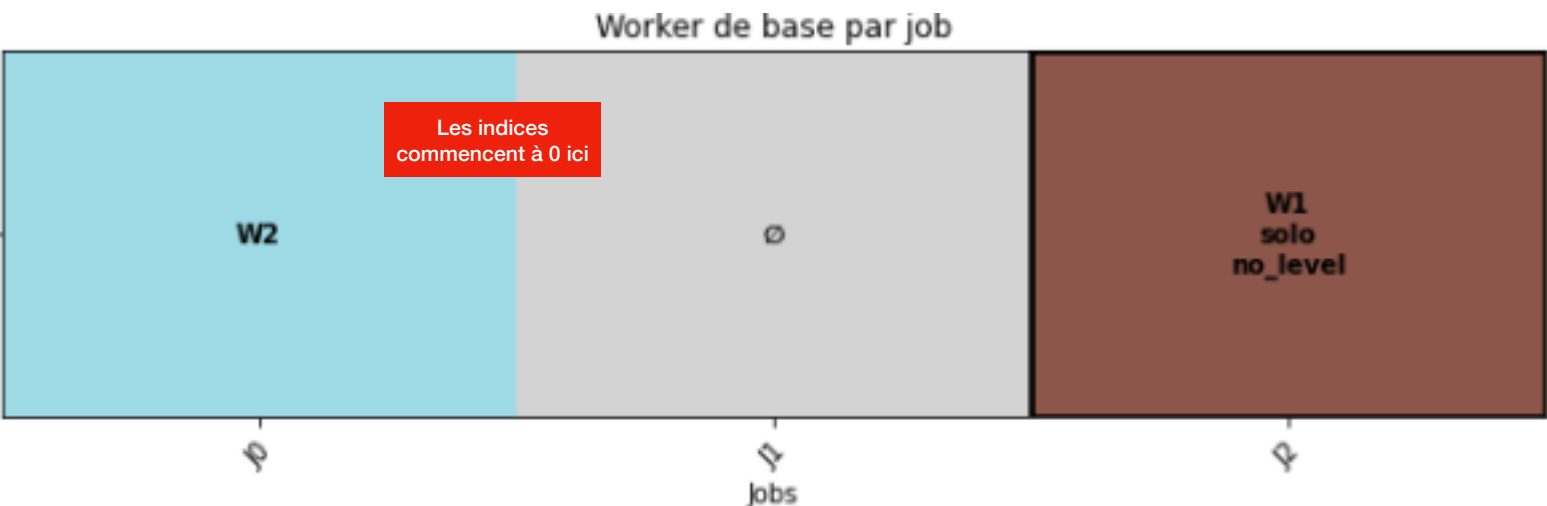
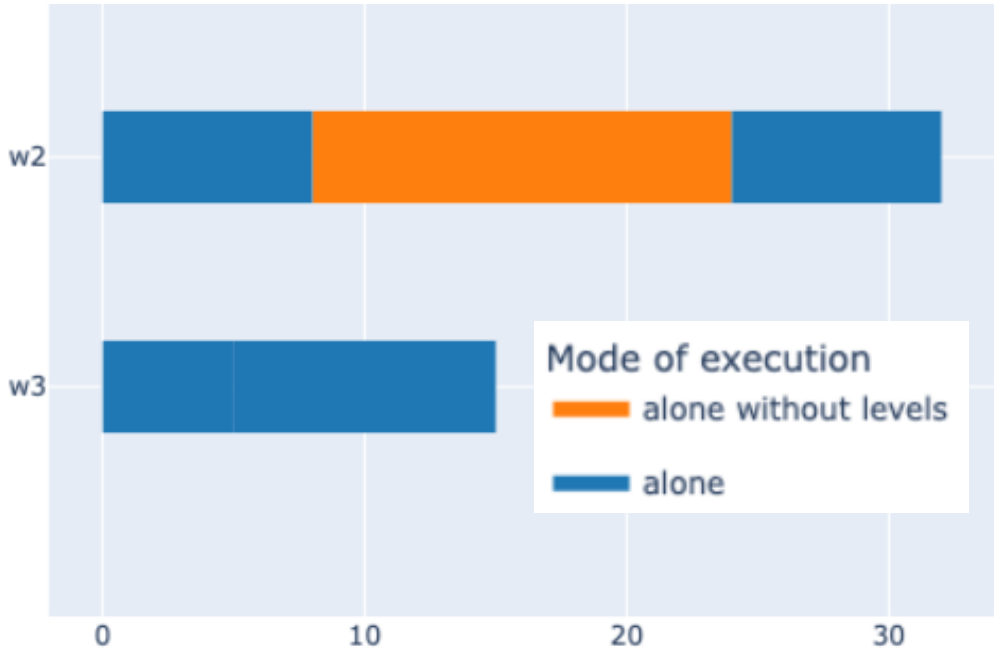
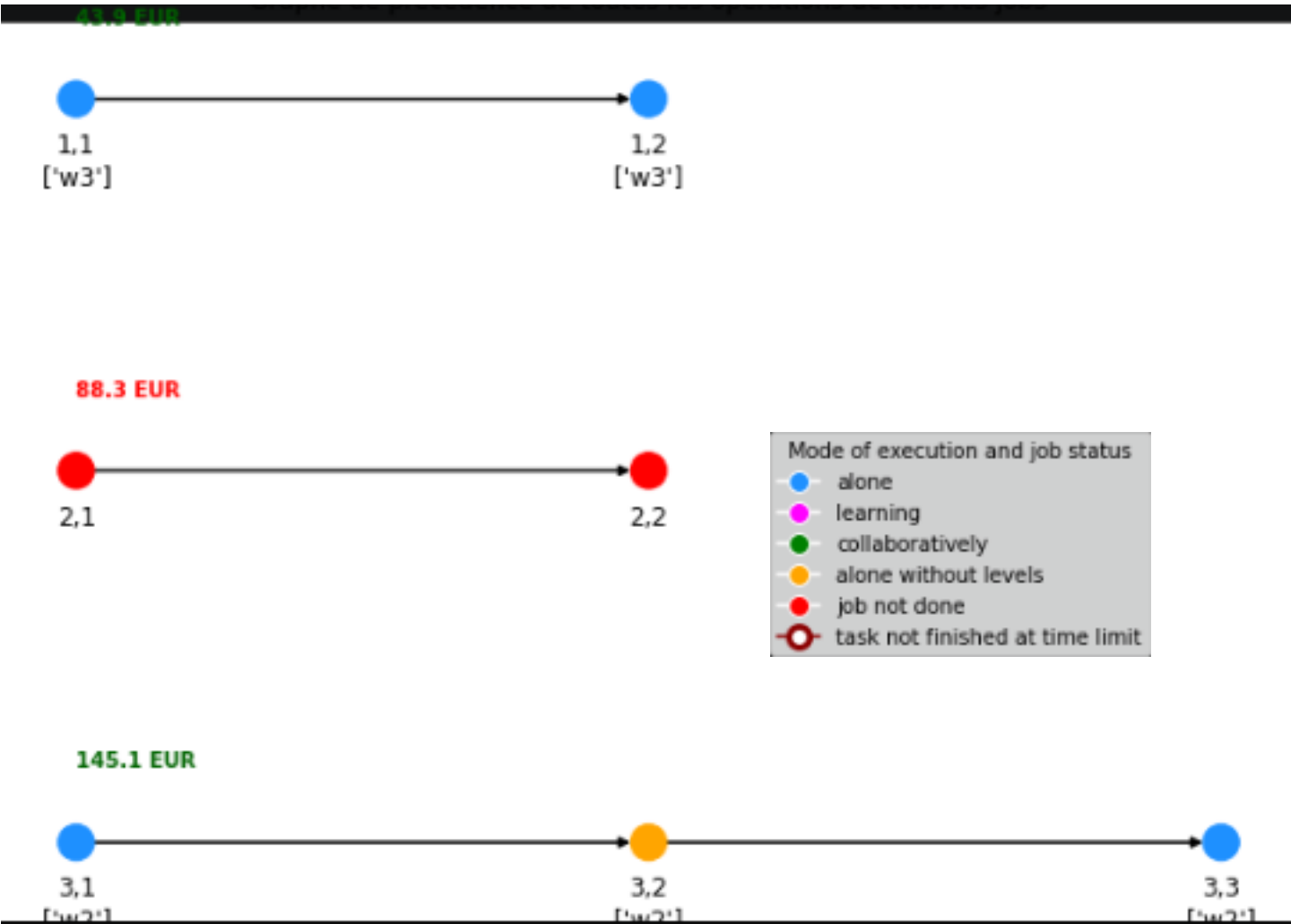
Instance 4



Graphe de précedence de toutes les opérations de tous les jobs



Qualification of workers for operations
● job with qualified workers
● job without qualified workers
● job have at most one level of difference



Comparaison des meilleures valeurs obtenues - Instance 4

Instance 4

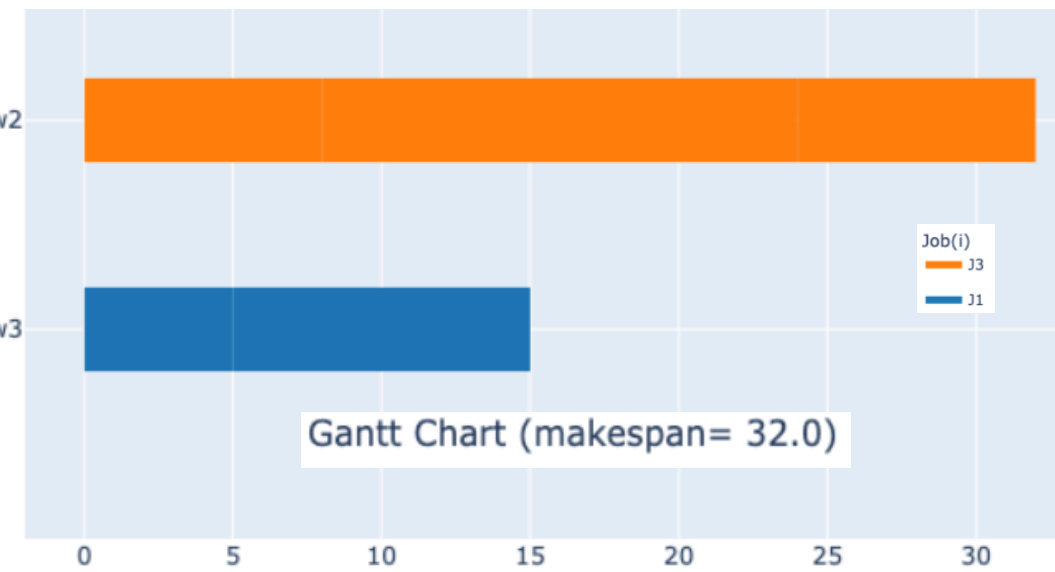
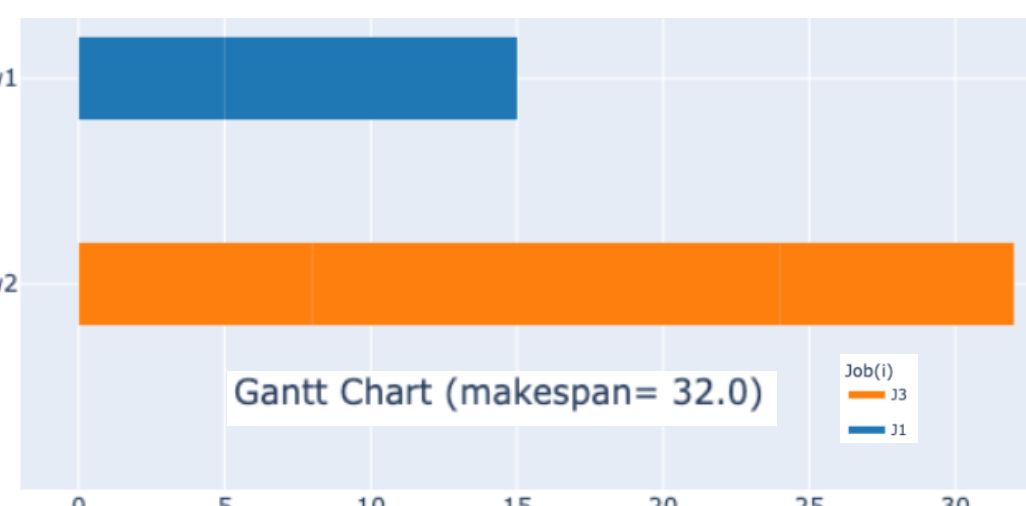
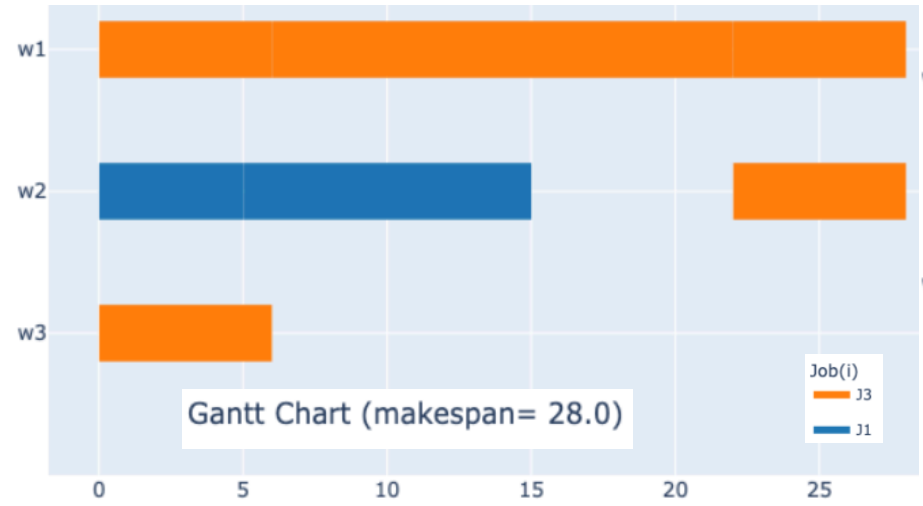
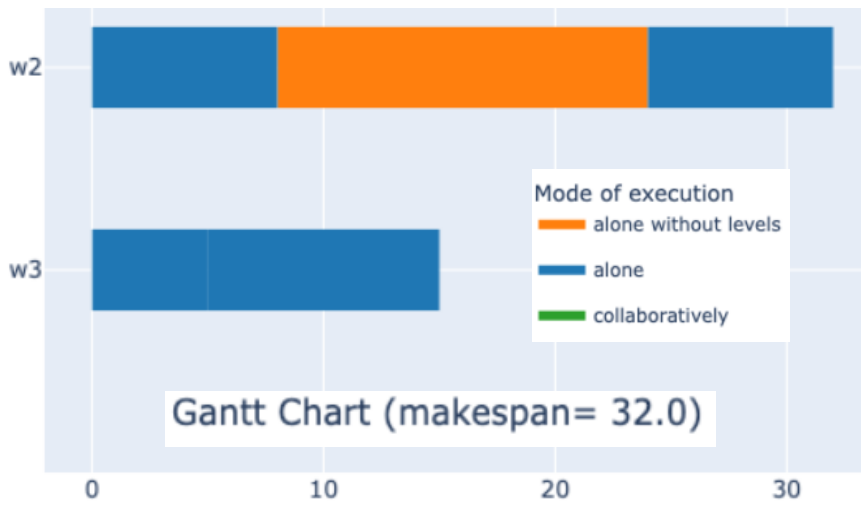
Solution exacte MILP
18875.40

Base worker
Max = 18875.40
Moy. fin = 16604.59
Gap max = 0.00 %
Gap moy fin = 12.03 %

Swap
Max = 18868.03
Moy. fin = 16587.86
Gap max = 0.04 %
Gap moy fin = 12.12 %

Cumulative
Max = 18875.40
Moy. fin = 17739.76
Gap max = 0.00 %
Gap moy fin = 6.02 %

Alternating
Max = 18875.40
Moy. fin = 16036.41
Gap max = 0.00 %
Gap moy fin = 15.04 %



Métaheuristique

Generator : *Instance, Encoding_Dico, Config*

generate_random(nb_job_no_skills) -> Chromosome(
 osc,
 fwac,
 swac,
 base_worker_assignment)

Evaluator : *Decoder, Weight, Obj_config*

evaluate_agg(chromosome) -> float

1. Appel **get_objectives** du *Decoder* pour récupérer la valeurs de chaque objectifs
2. Fais somme pondérer selon *Weight* et retranche le Cmax pour être en Accord avec le **MILP**
3. Affecte fitness et objectives au chromosome
4. Retourne la fitness

Utils

chromosome_to_x_variable(chromosome) -> np.ndarray(nb_jobs, max_op, nb_w)
assert_base_worker_is_respected(x, base_worker_assignment) -> bool

Local Search : *Generator, Evaluator, Neighborhood, Selector*

run(max_iter, verbose)
• **Generator.generate()**
• **Evaluator.evaluate()**
• For Loop : Neighborhood.generate() -> **Evaluator.evaluate()** -> Selector.select()

Chromosome

- osc
- fwac
- swac
- fitness
- objectives
- base_worker_Assignment

Decoder : *Instance, Encoding_Dico, Config*

decode_fast(chromosome) -> dico

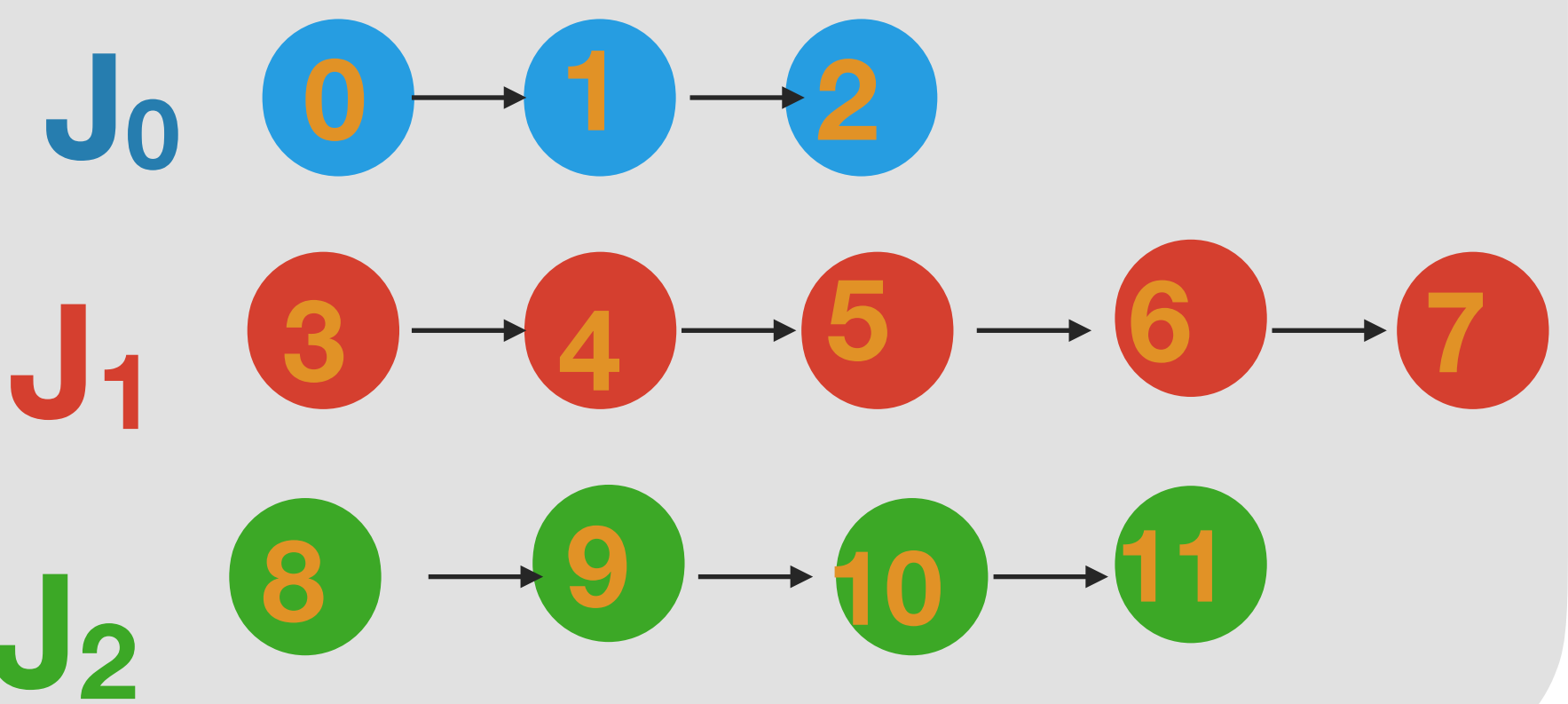
decode_solution(chromosome) -> Solution

- Appel **decode_fast(chromosome)** puis ajoute les variables importantes pour Solution

get_objectives(chromosome) -> list[obj1, obj2, ...]

- Récupère chaque objectif avec l'appel à **decode_fast(chromosome)**

id des opérations des jobs



Worker de base pour job i	2	0	-1
Penalité level pour worker de base	0	1	0

Quelle que soit la permutation des gènes de **OSC**, la séquence obtenue respecte un tris topologique des opérations

Operation Sequence Chromosome (OSC)

1	1	0	2	2	0	0	1	1	1	2	2
---	---	---	---	---	---	---	---	---	---	---	---

First Worker Assignment Chromosome (FWAC)

0	2	2	0	0	0	0	1	0	2	0	-1
id: 0	id: 1	2	3	4	5	6	7	8	9	10	id: 11

Second Worker Assignment Chromosome (SWAC)

2	-1	-1	-1	-1	-1	1	0	0	2	0	-1
id: 0	id: 1	2	3	4	5	6	7	8	9	10	id: 11

Mode opératoire est **déduit** avec le niveau de compétence des workers affectés

C	A	T	NA	C	C	AL	A	A	T	T	NA
---	---	---	----	---	---	----	---	---	---	---	----

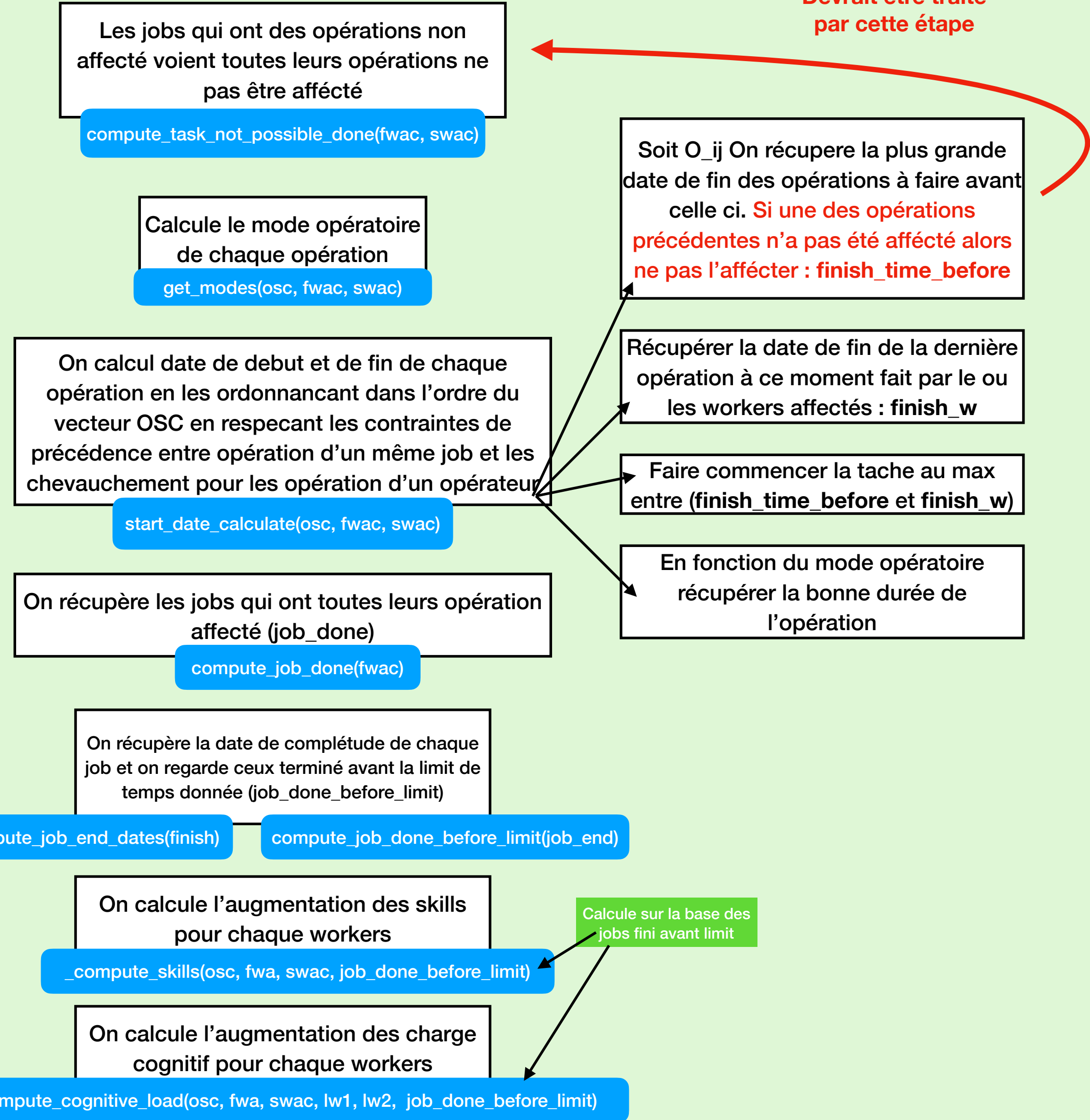
Besoin de ce vecteur pour calculer les compétences et charge cognitive des workers

A : Alone
C : Collab
T : Teaching
AL : Alone no Level
NA : Not Assigned

OSC	1	1	0	2	2	0	0	1	1	1	2	2
FWAC	0	2	2	-1	0	0	1	1	0	2	0	2
SWAC	0	2	2	-1	0	0	1	1	0	2	0	2
	id: 0	id: 1	2	3		5		8				id: 11

Flowchart de décodage

Devrait être traité par cette étape



Run Local Search

```
chromosome = generator.generate_random()
```

```
chromosome = evaluator.evaluate_agg(chromosome)
```

```
for loop in Range MAX_ITER
```

```
    voisins = neighborhood.generate(chromosome)
```

```
    fitness_voisins = evaluator.evaluate_agg(voisins)
```

```
    idx_voisins = selector.select(fitness_voisins)
```

```
    if voisins[idx_selected].fitness >= chromosome.fitness
```

```
        chromosome = voisins[idx_selected]
```

```
        chromosome.fitness = fitness_voisins[idx_selected]
```

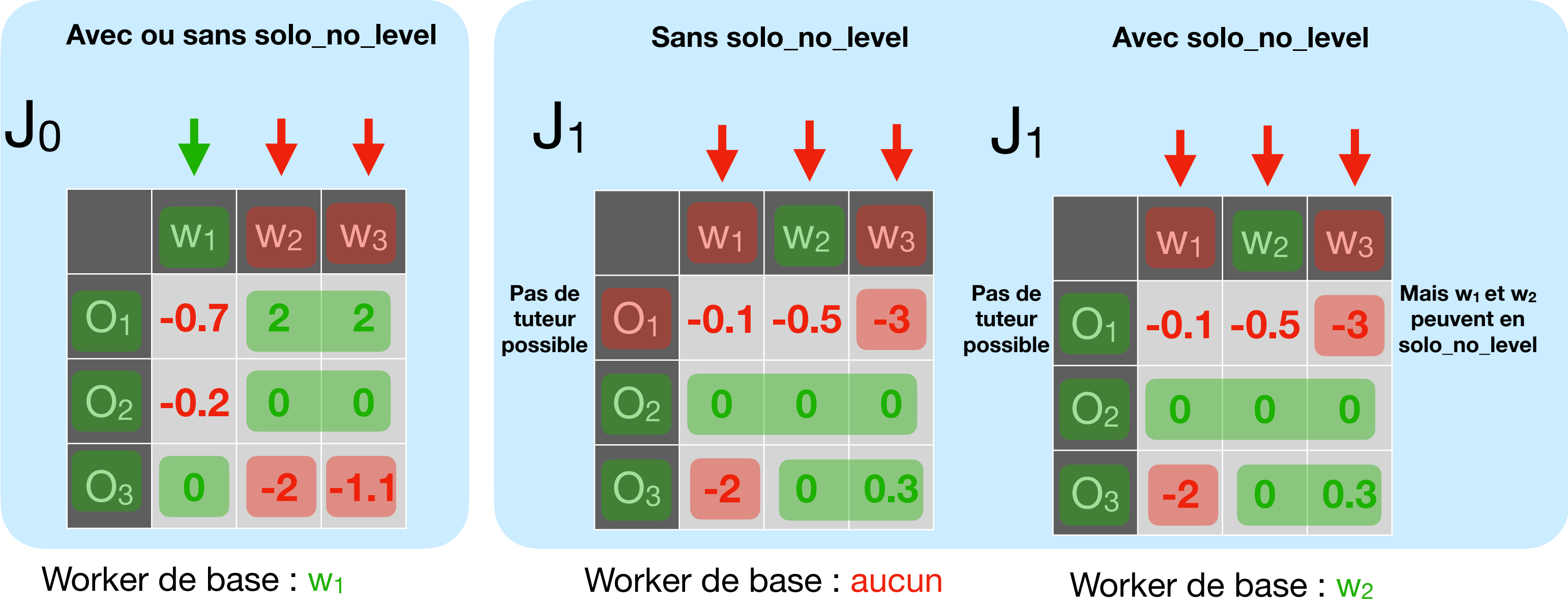
```
    all_viewed.append(chromosome)
```

Evaluate_agg

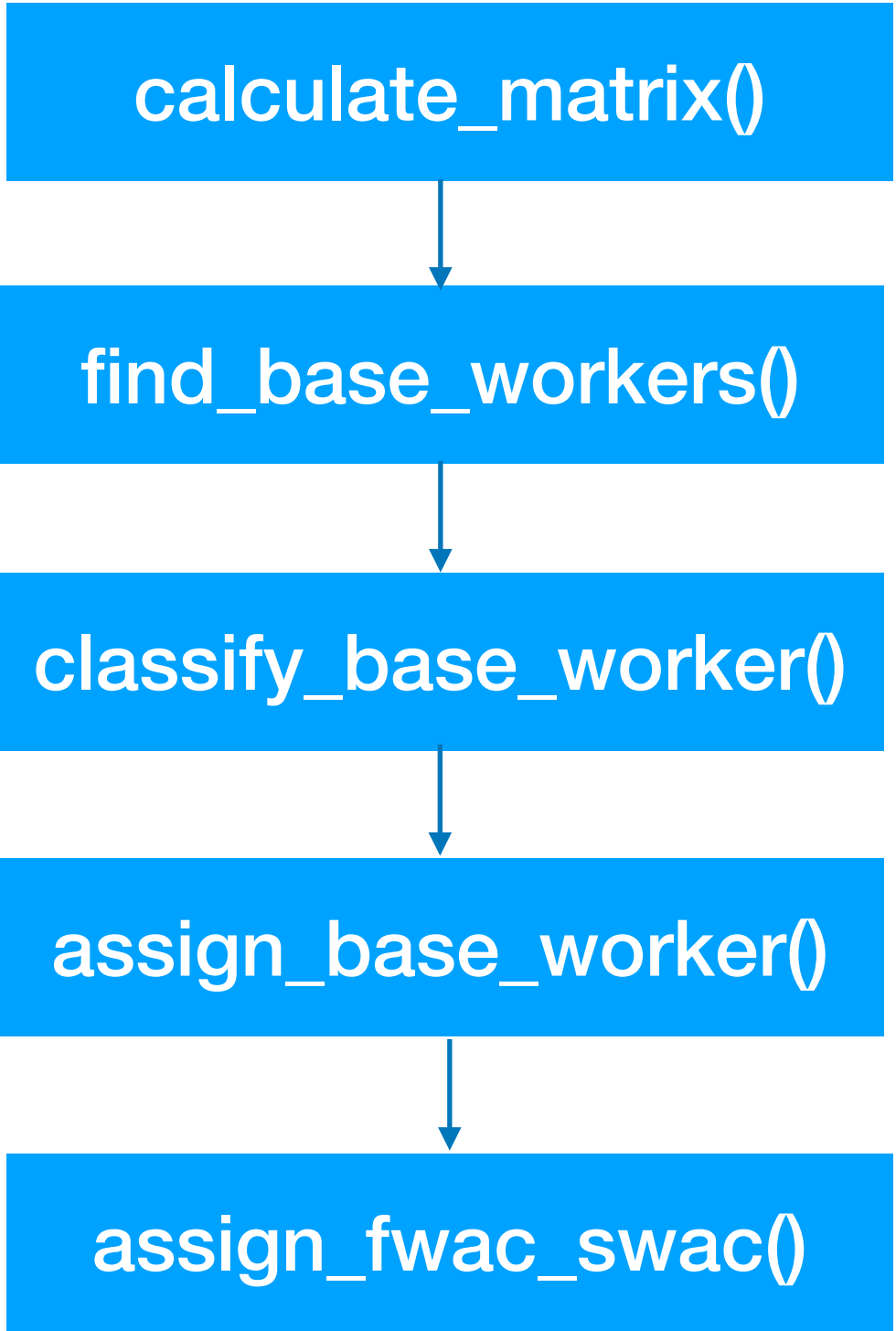
```
chromosome = generator.generate_random()
```

Modélisation worker de base :

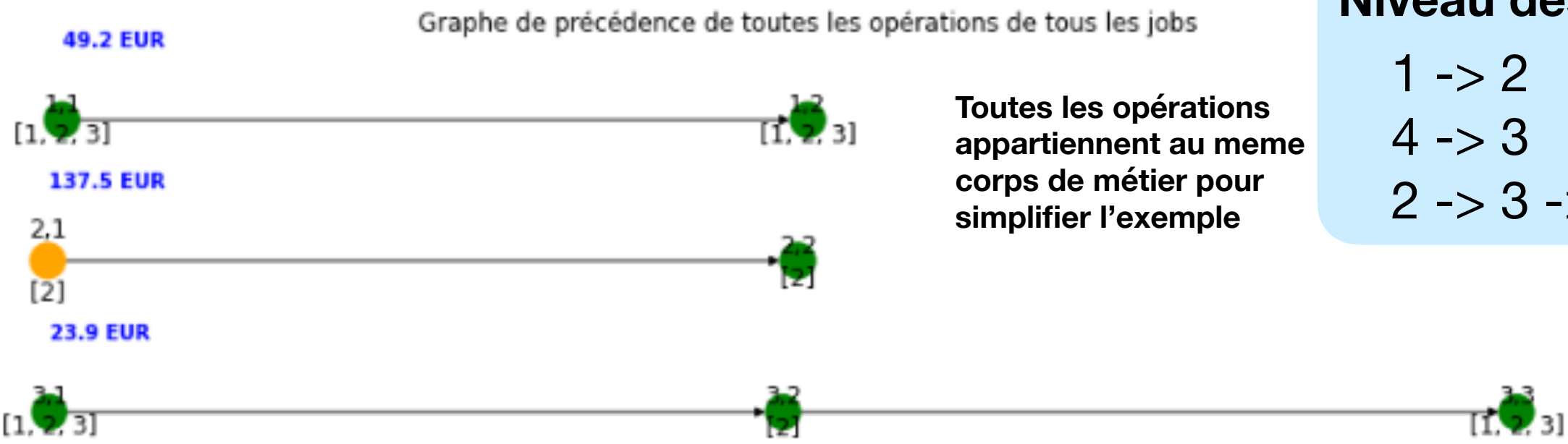
- Pour qu'un worker k soit worker de base d'un job J_i il faut que :
- 1. k ai au plus une difference de 1 avec le niveau de difficulté du job J_i (colonne)
 - 2. Pour chaque opération du job J_i il existe un worker k' de niveau requis pour chaque opération du job (ligne)
 - 3. Si solo sans level alors le autoriser maximum1 job a enfreindre la règle 2.



Limiter le nombre de job qui se voit attribuer des workers en solo no levels



2 1 0 ← worker de base du job i
0 1 0 ← 1 si solo_no_level, pour savoir quelle job est fait en apprentissage autonome et fixé un nombre limite de job fait ainsi

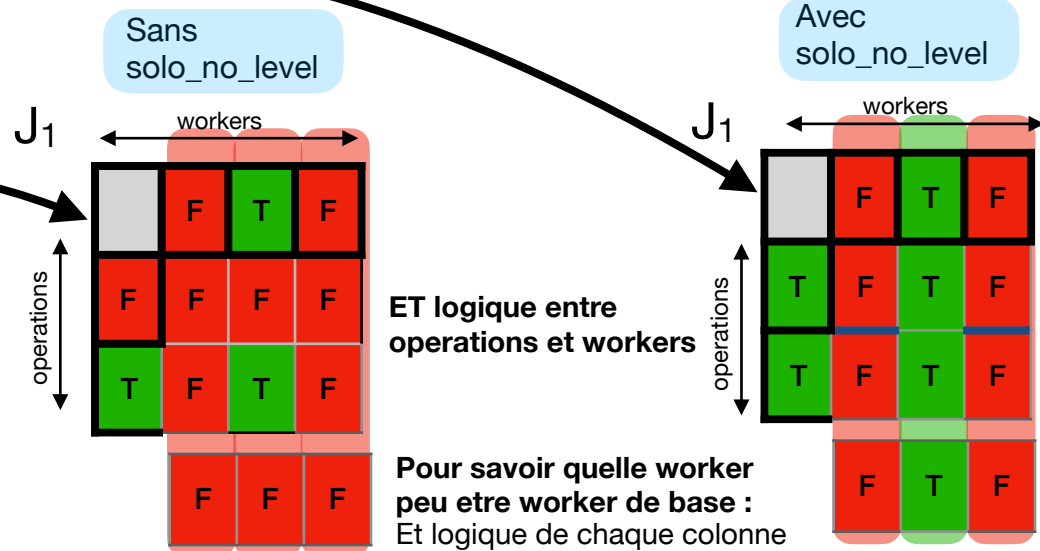


Niveau des taches

1 -> 2
4 -> 3
2 -> 3 -> 2

Niveau des workers

w1 : 2.45
w2 : 3.09
w1 : 2.11



Assign_fwac_swac algorithm

On parcourt les opérations selon l'ordre de OSC
Si le job de l'opération à un worker de base :

Si ce worker w_i de base n'est pas solo_no_levels :

Si ce worker w_i n'a pas le niveau pour cette opération :

w_j doit être un worker qualifié (**Teaching seulement**)

Sinon :

w_j doit être un worker avec au plus différence de 1 avec l'opération (**Teaching, collab ou solo**)

Mettre le worker le plus qualifié dans FWAC et l'autre dans SWAC

Si ce worker w_i de base est solo_no_levels :

Si ce worker w_i n'a pas le niveau pour cette opération :

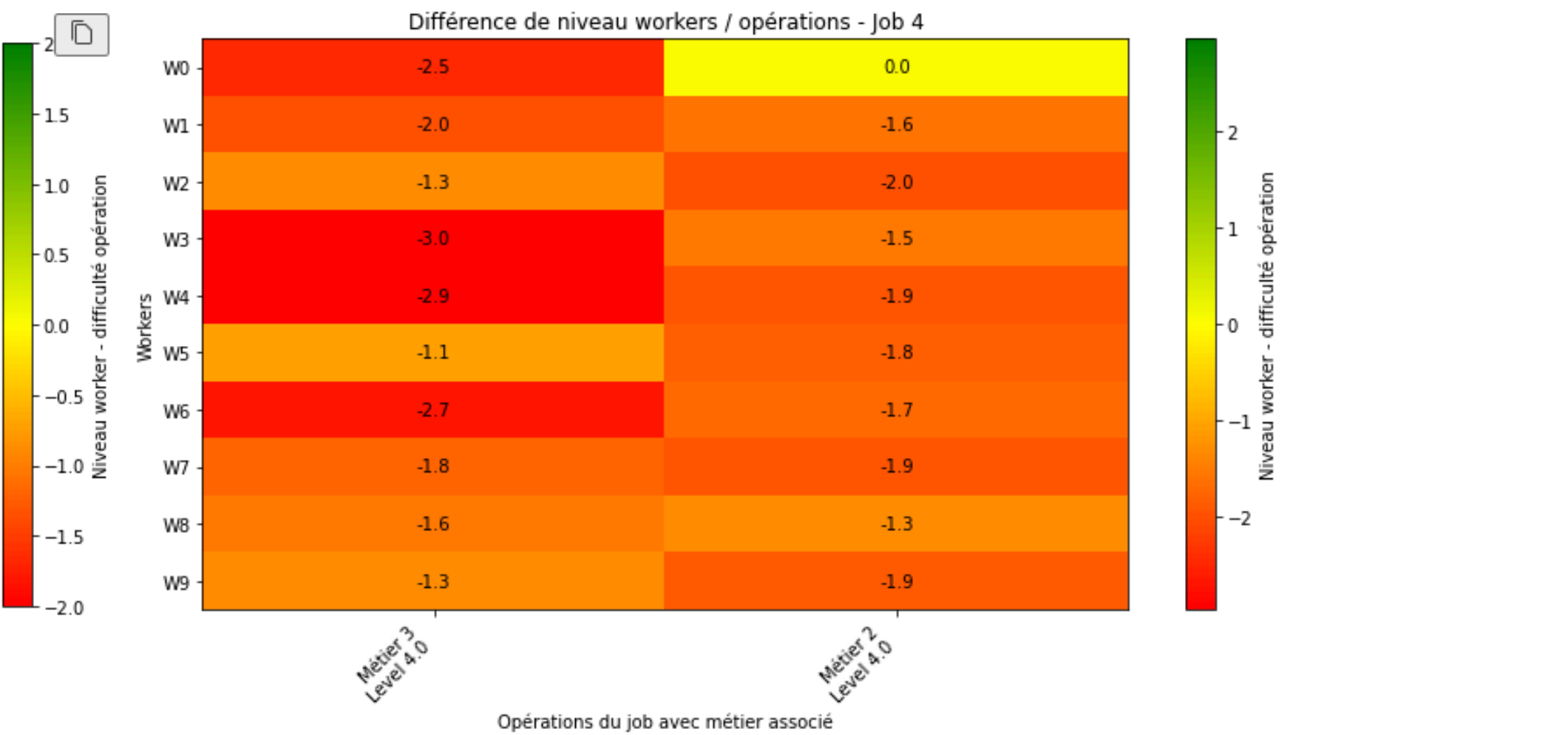
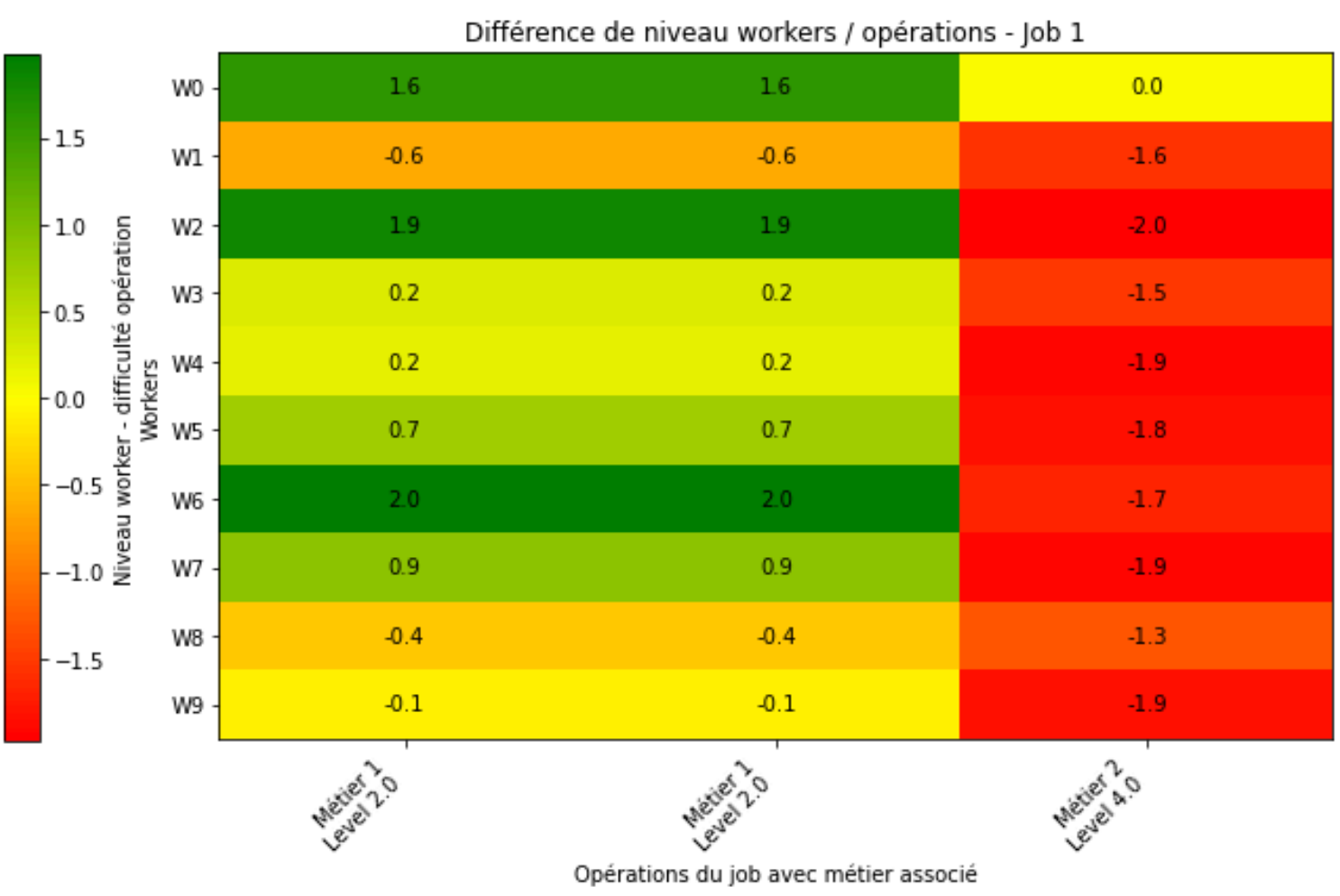
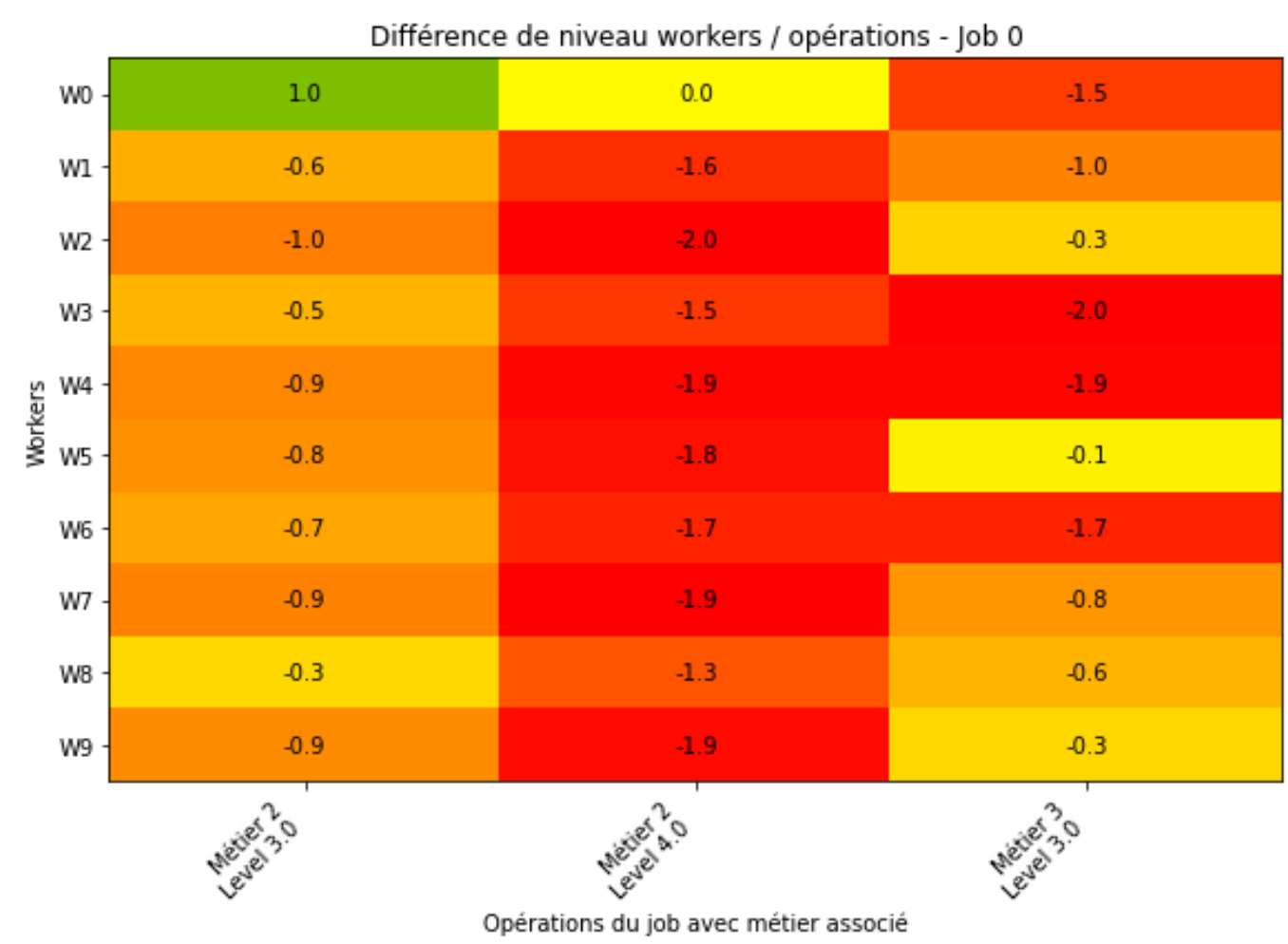
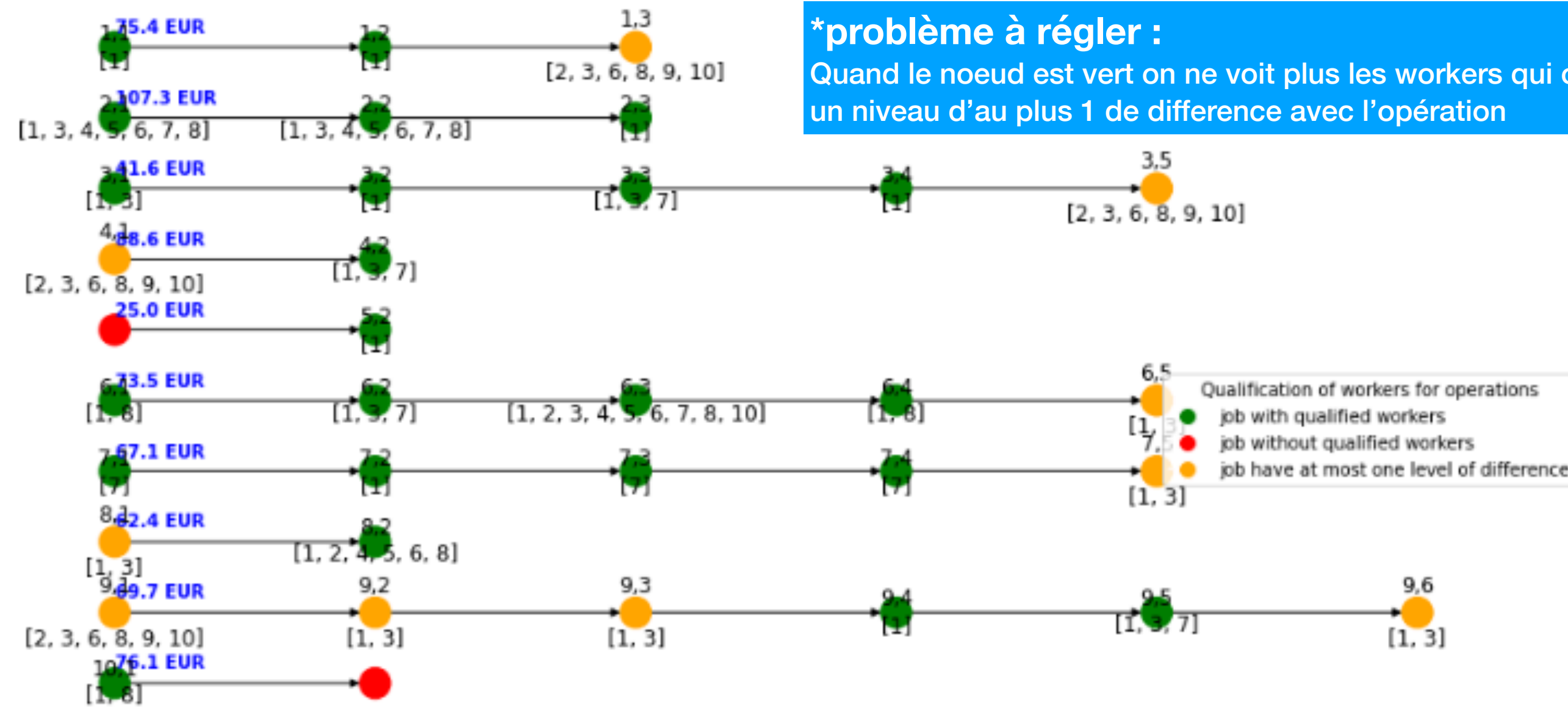
w_j doit être un worker qualifié ou personne (**Teaching ou solo_no_levels**)

Sinon :

w_j doit être un worker avec au plus différence de 1 avec l'opération (**Teaching, collab ou solo**)

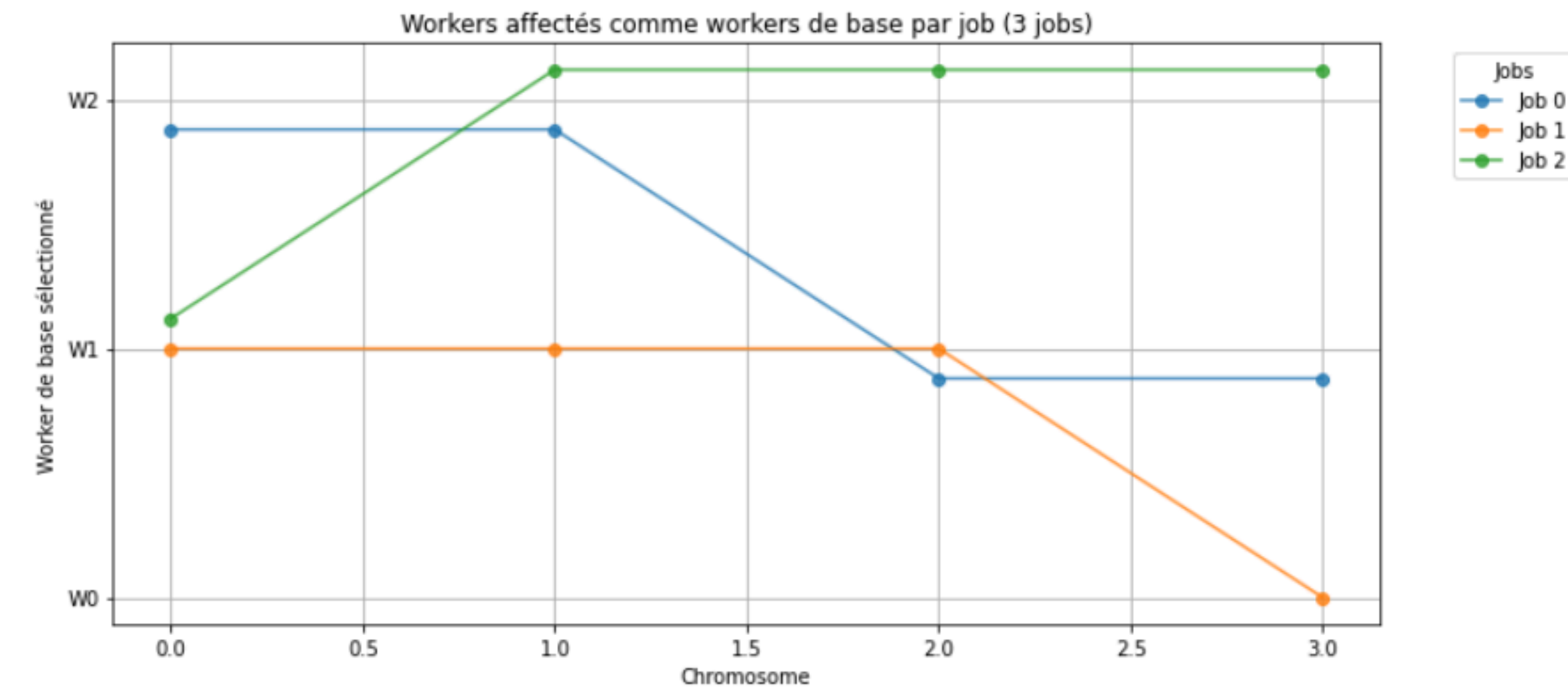
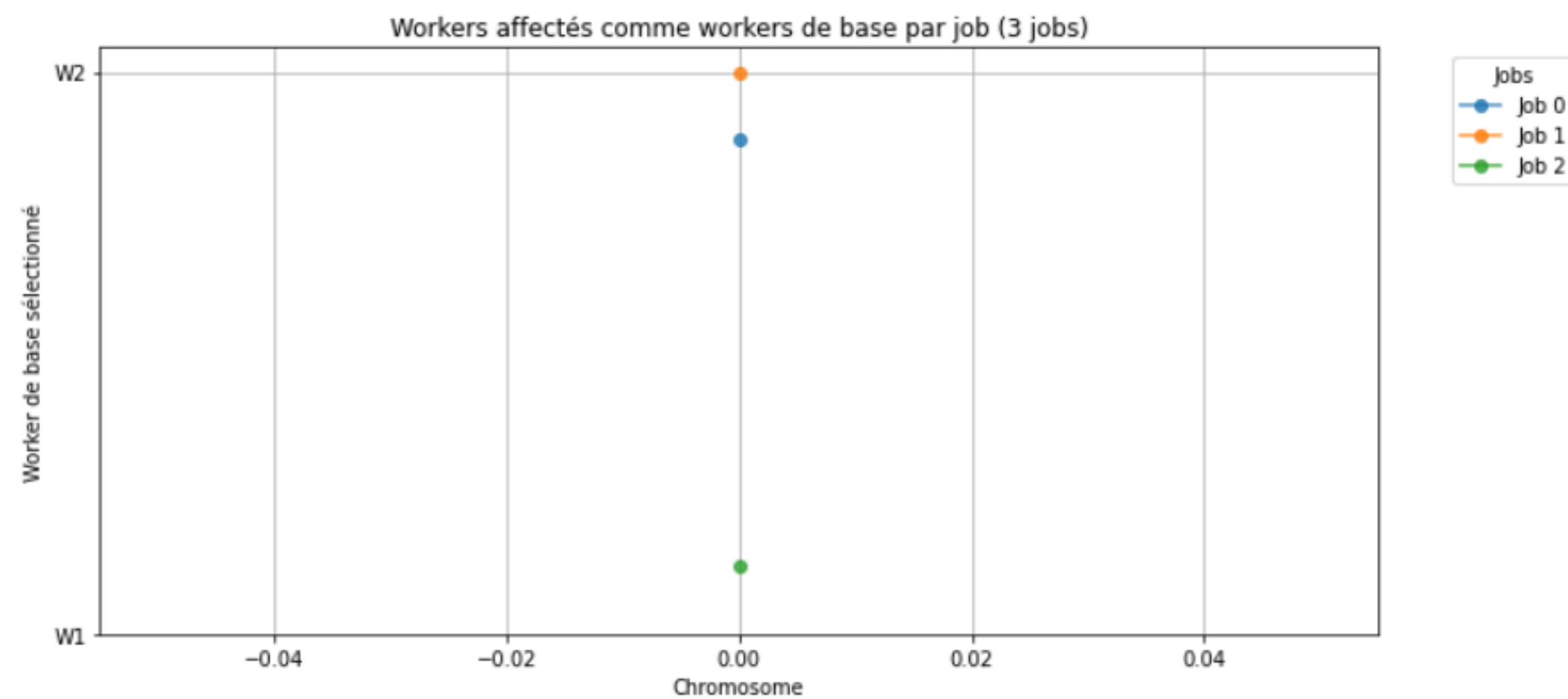
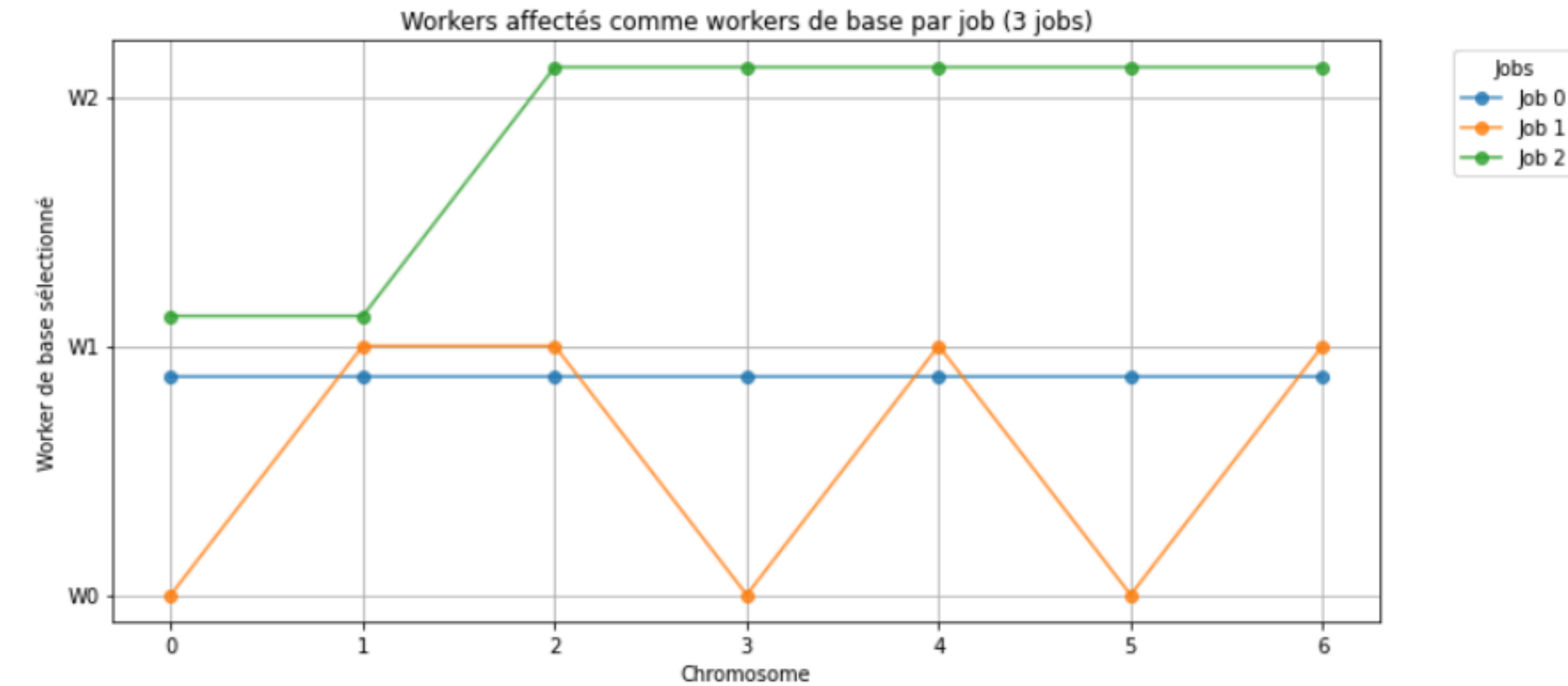
Mettre le worker le plus qualifié dans FWAC et l'autre dans SWAC

Graphe de précedence de toutes les opérations de tous les jobs



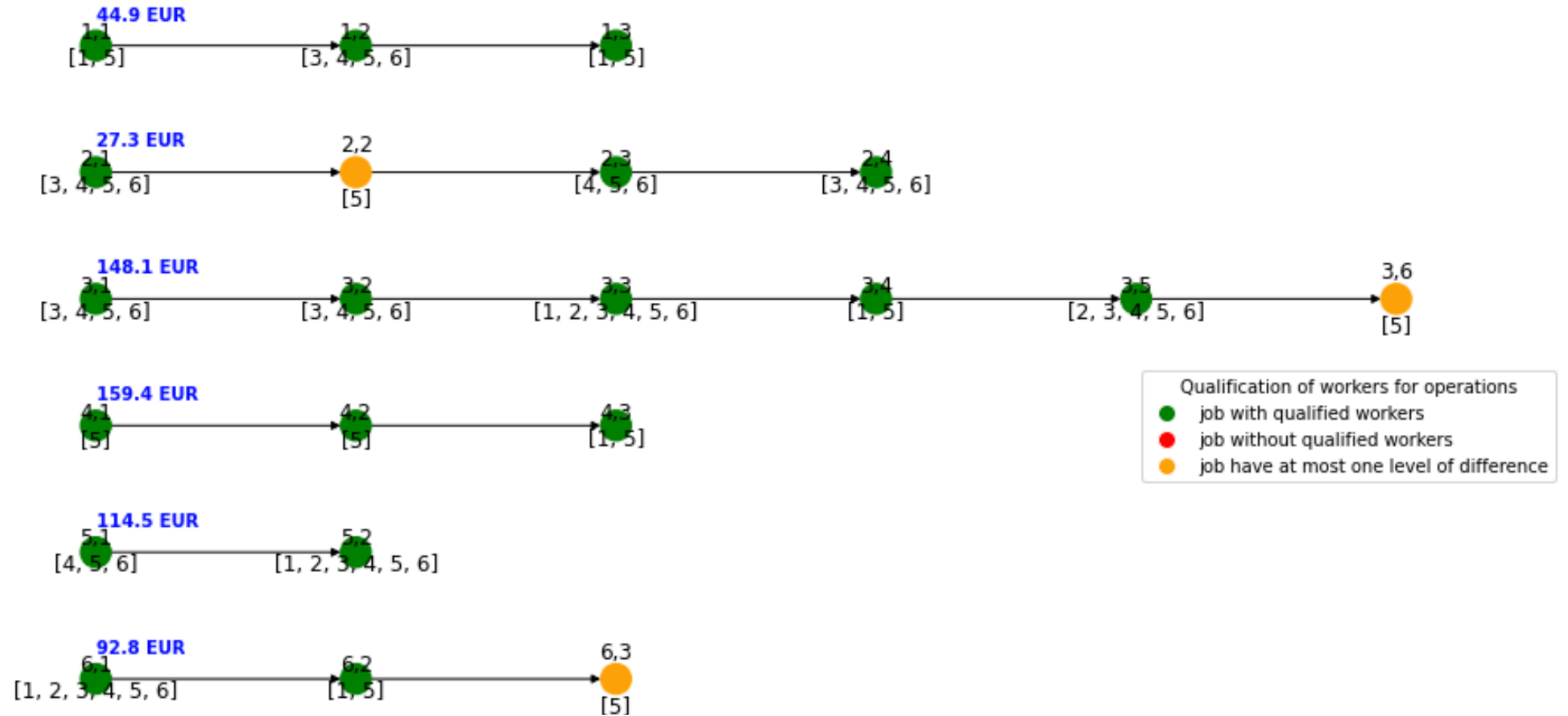
Variation des workers de base affecté aux jobs

En fonction du best chromosome courant lors des itérations



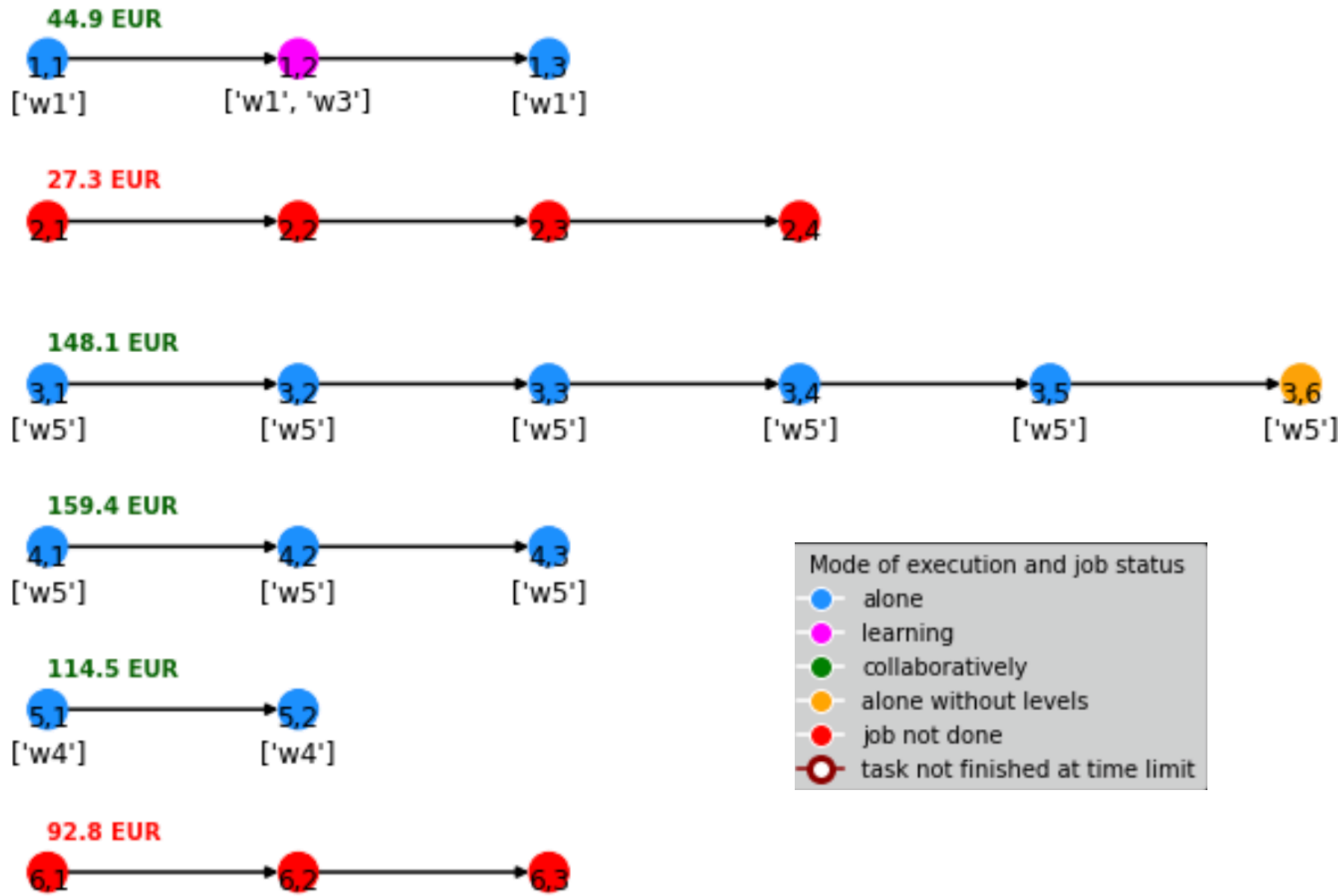
Grande Instance

Graphe de précedence de toutes les opérations de tous les jobs



MILP

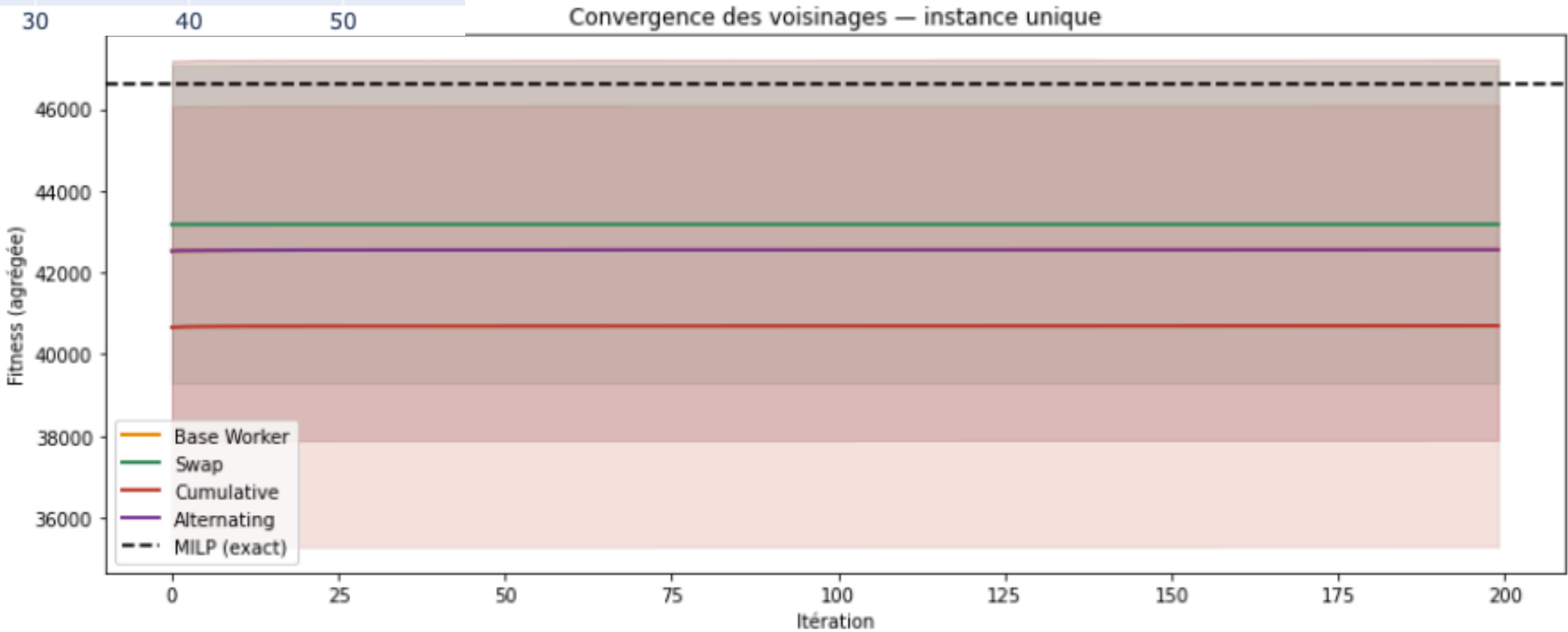
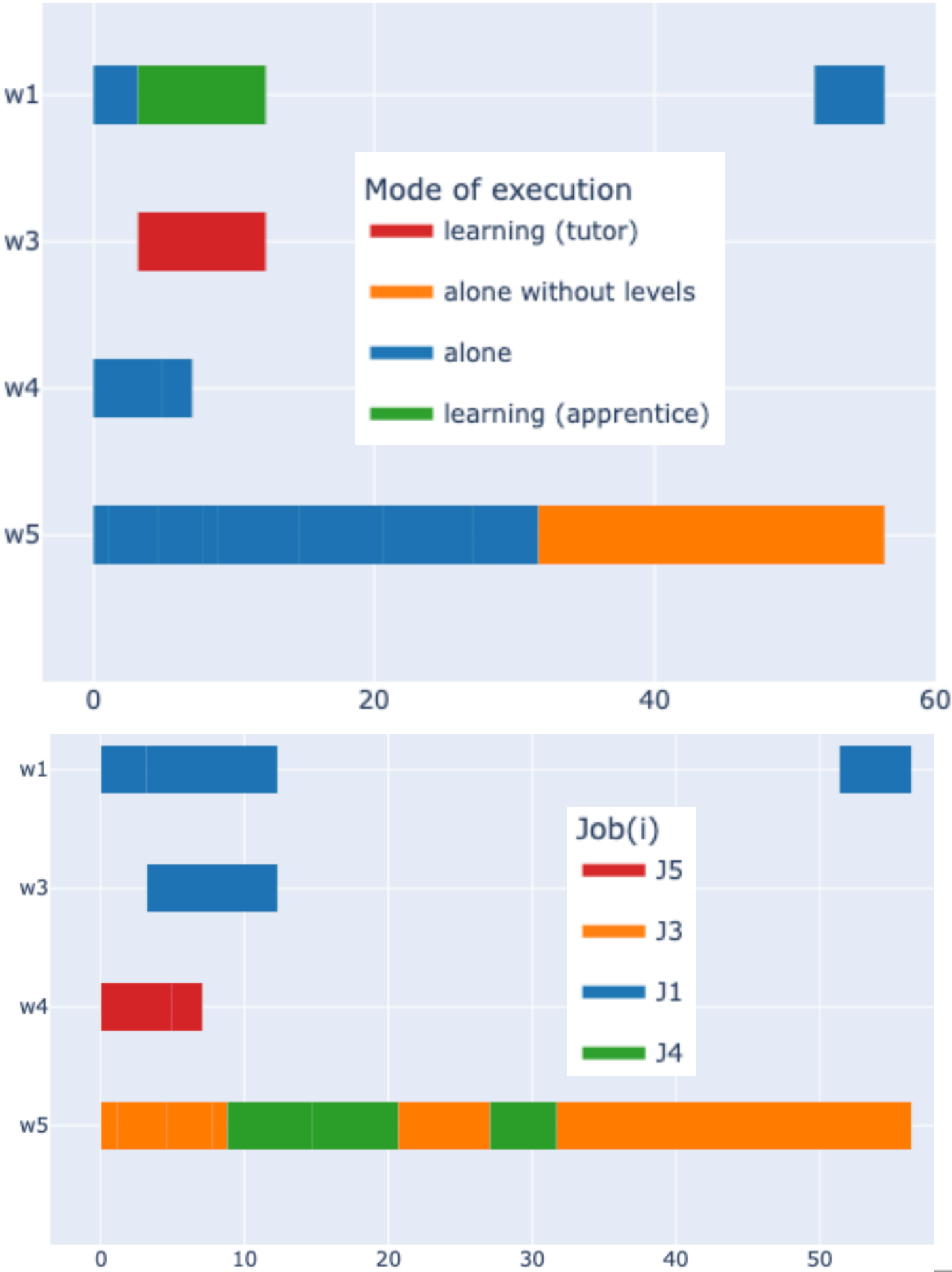
profit : 466.89075292987206
skills : 1.0
cognitive_load : 6.423149125240208
Cmax : 56.38540915152526
expected_fitness : 46636.266734710436



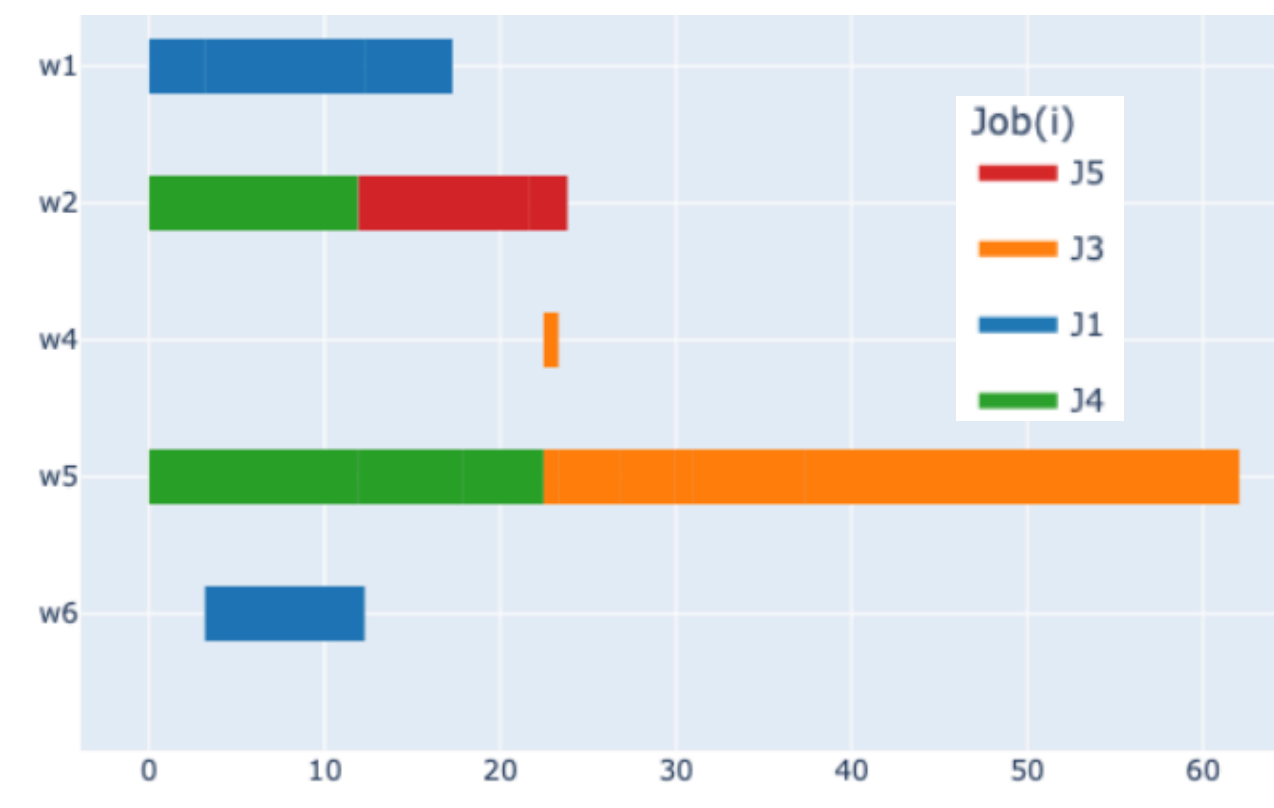
Comparaison des meilleures valeurs obtenues

Solution exacte MILP
46636.27

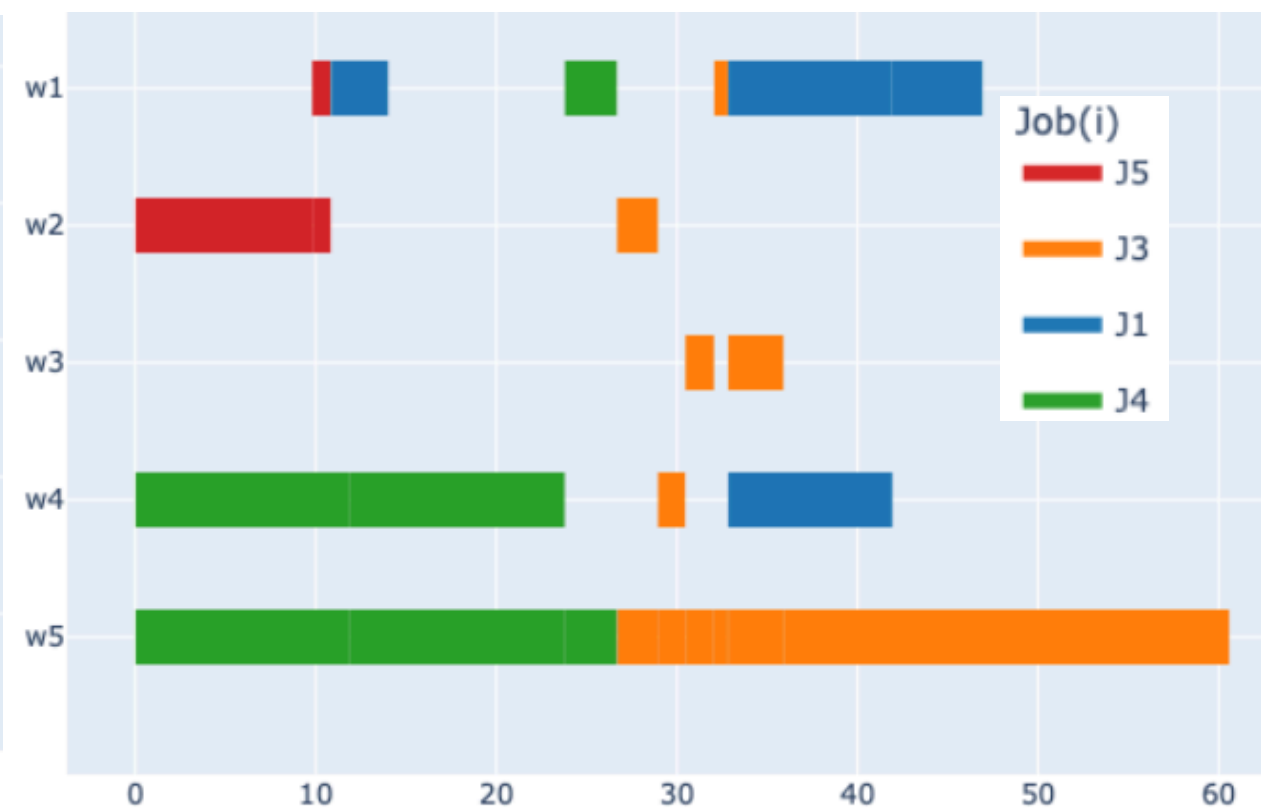
Base Worker	Swap	Cumulative	Alternating
Max = 46628.81	Max = 46597.71	Max = 46629.99	Max = 46630.90
Moy. fin = 42559.17	Moy. fin = 43180.15	Moy. fin = 40696.25	Moy. fin = 42558.84
Gap max = 0.02 %	Gap max = 0.08 %	Gap max = 0.01 %	Gap max = 0.01 %
Gap moy. = 8.74 %	Gap moy. = 7.41 %	Gap moy. = 12.74 %	Gap moy. = 8.74 %



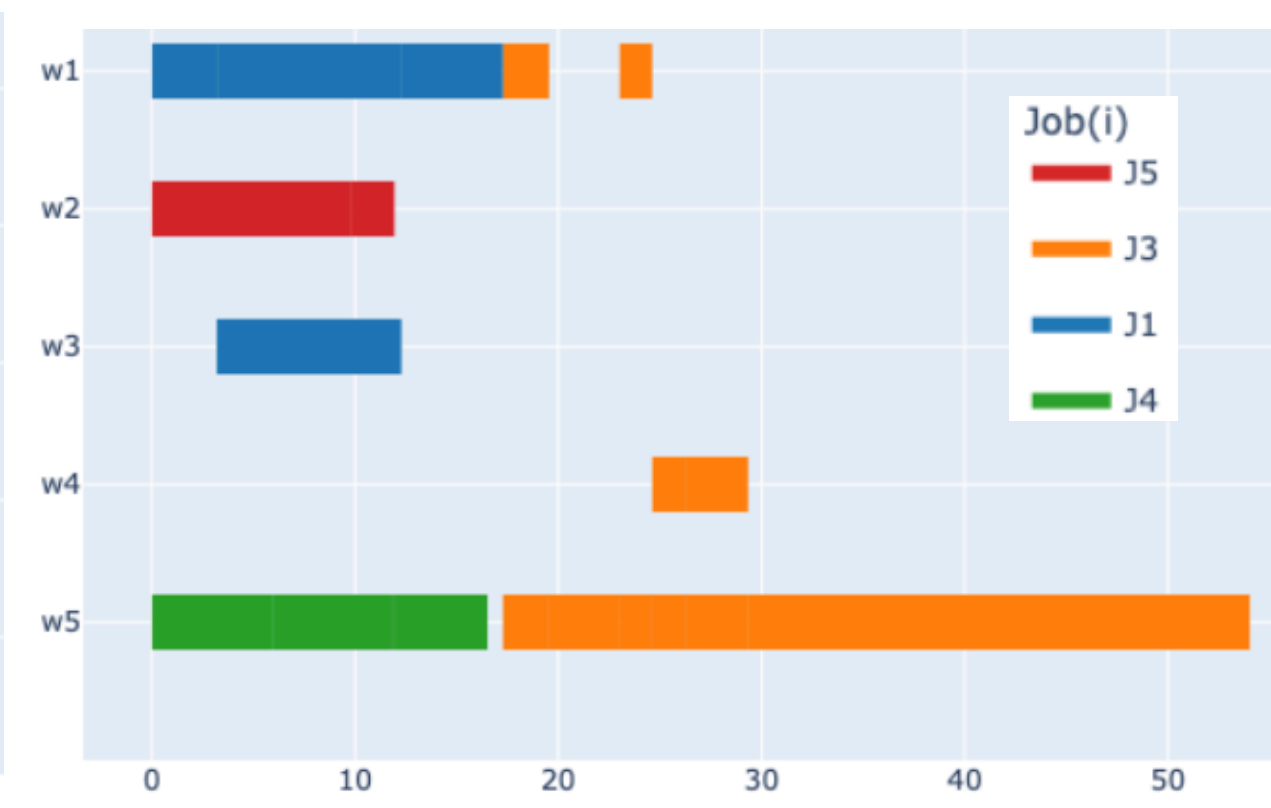
Voisinage base_worker



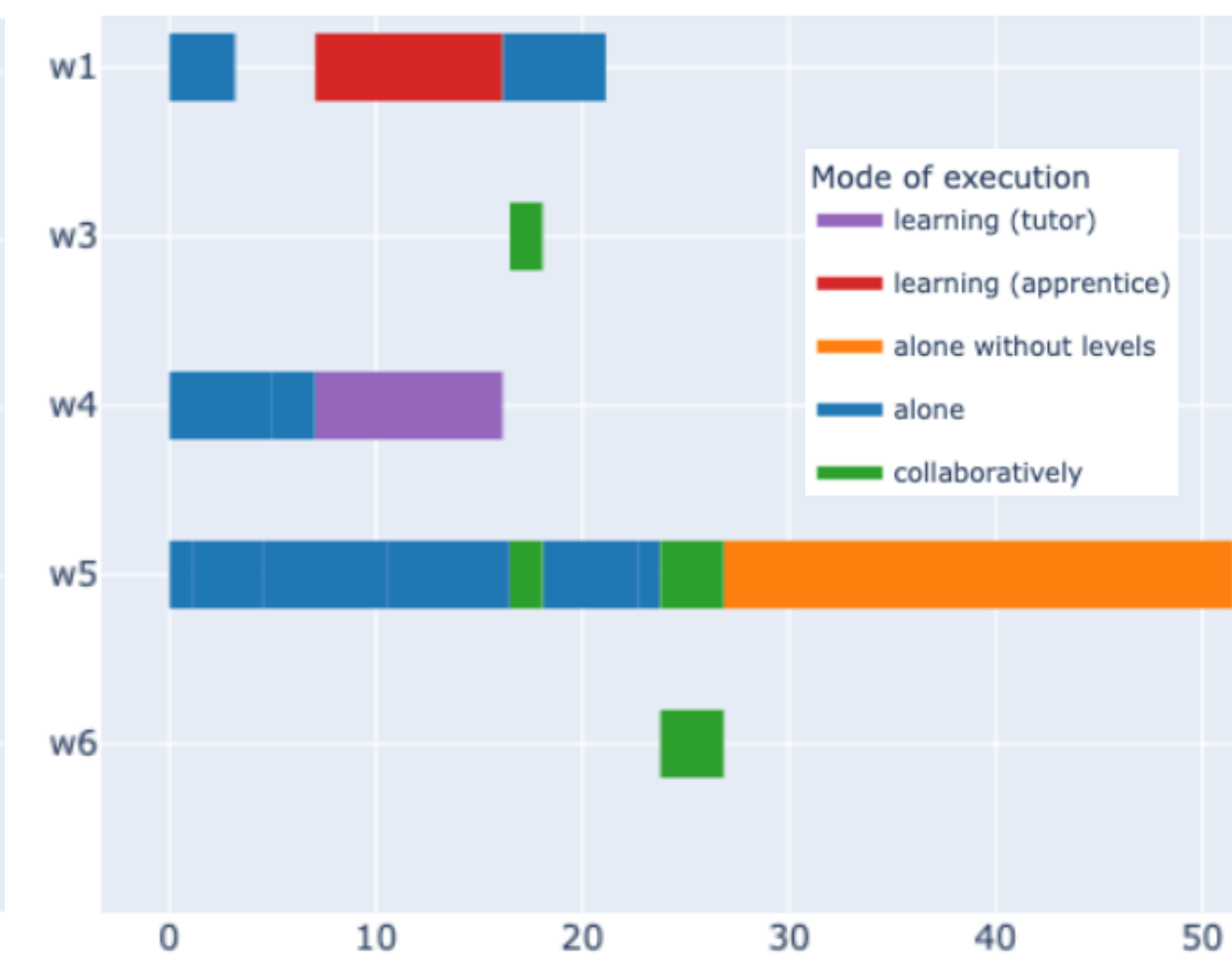
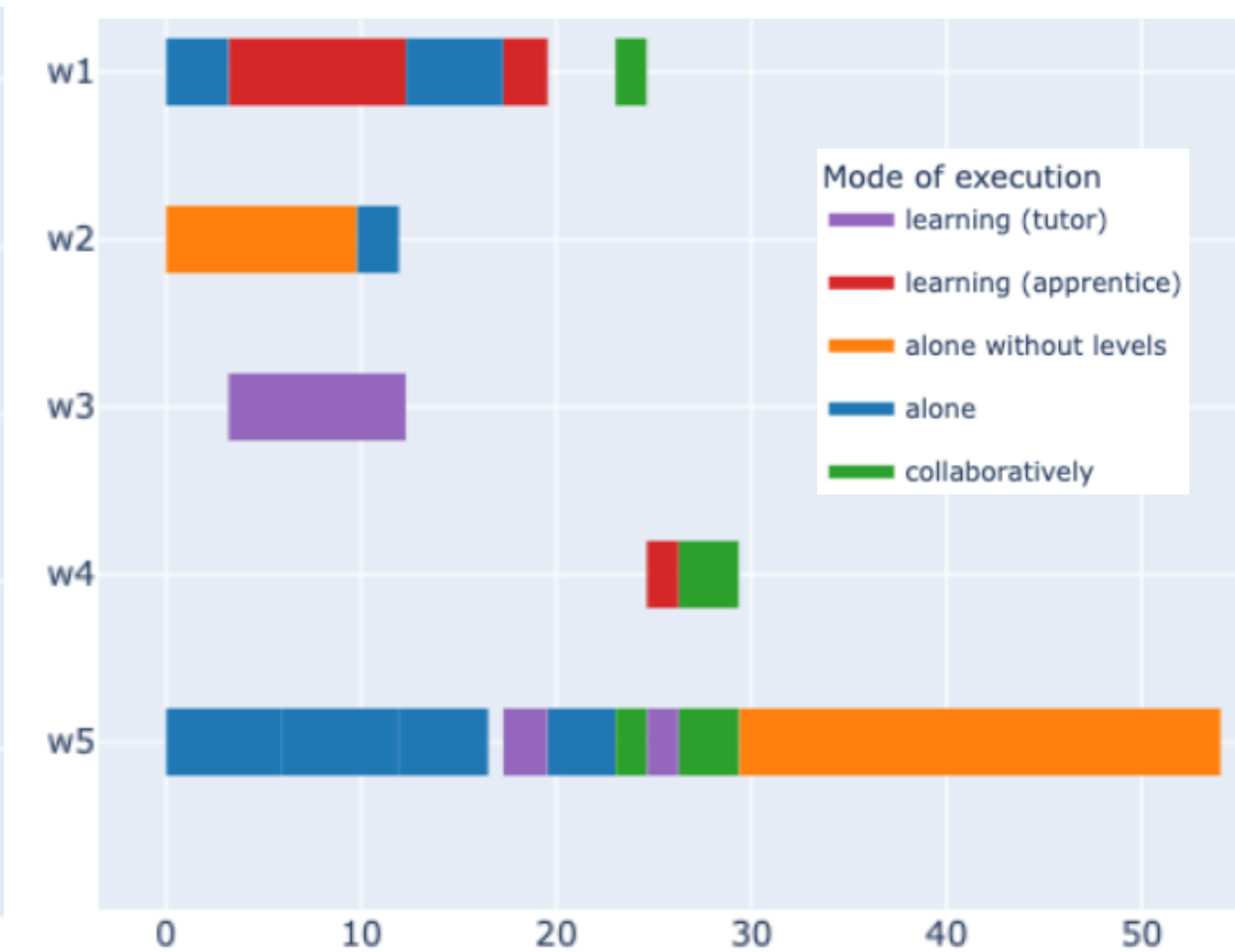
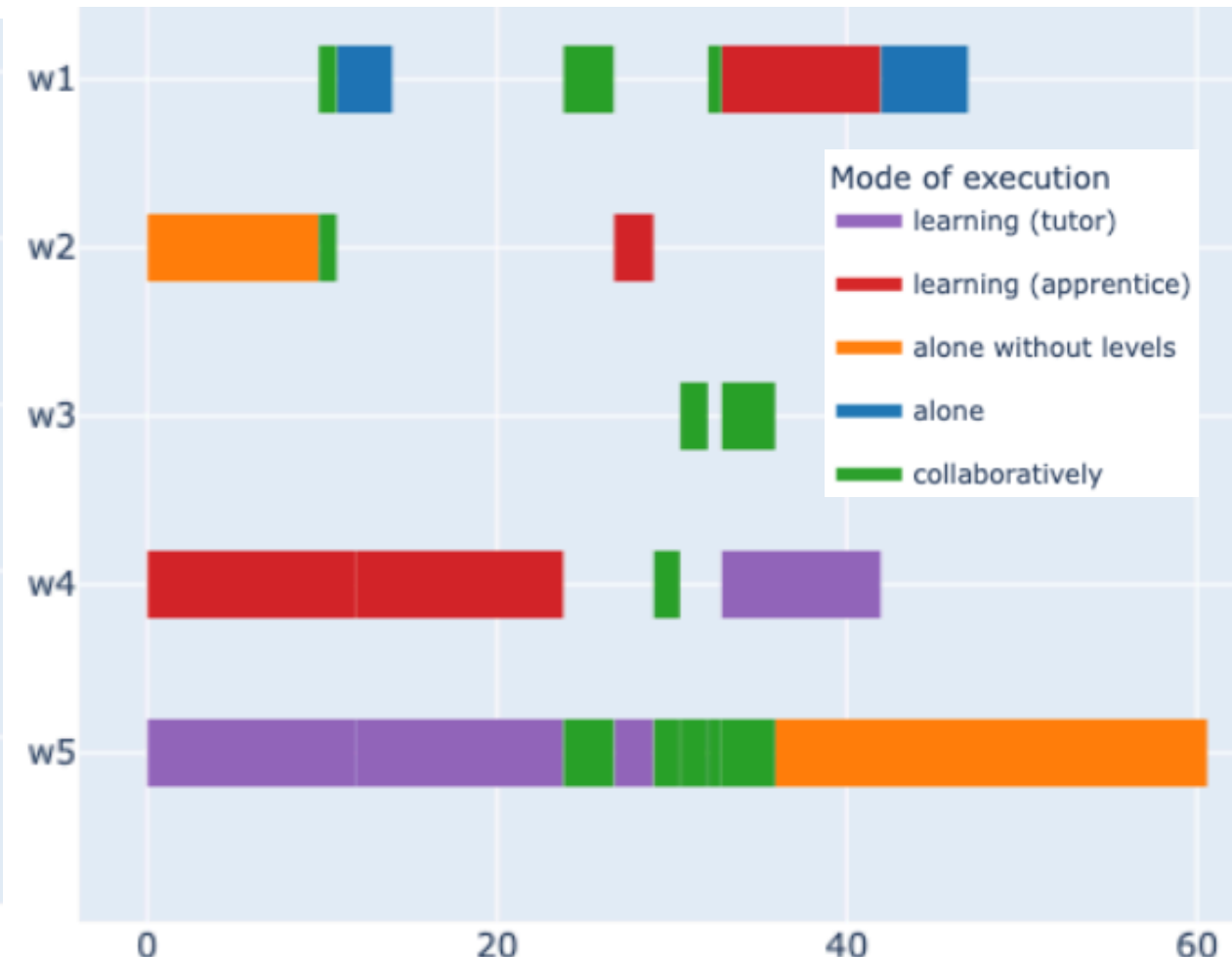
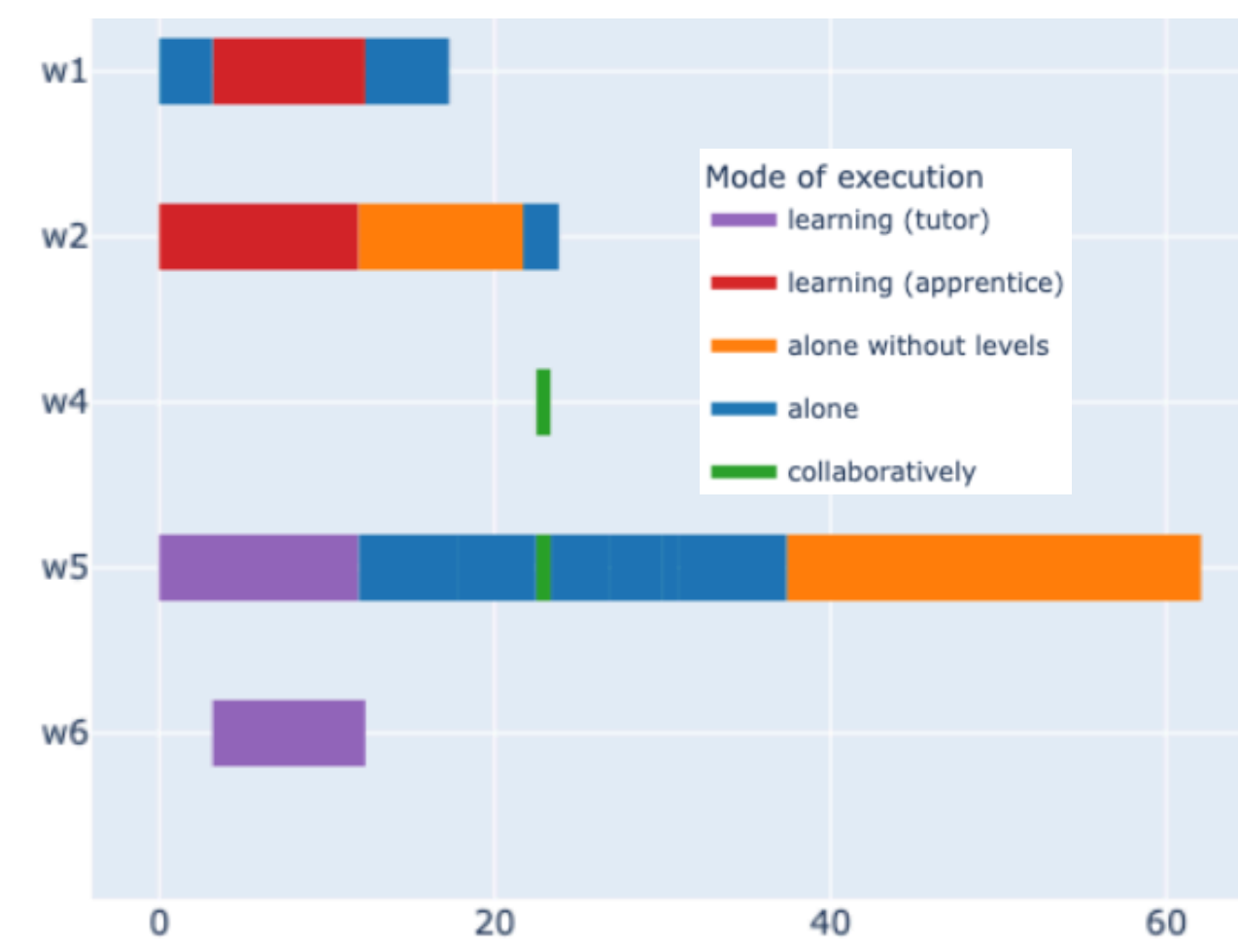
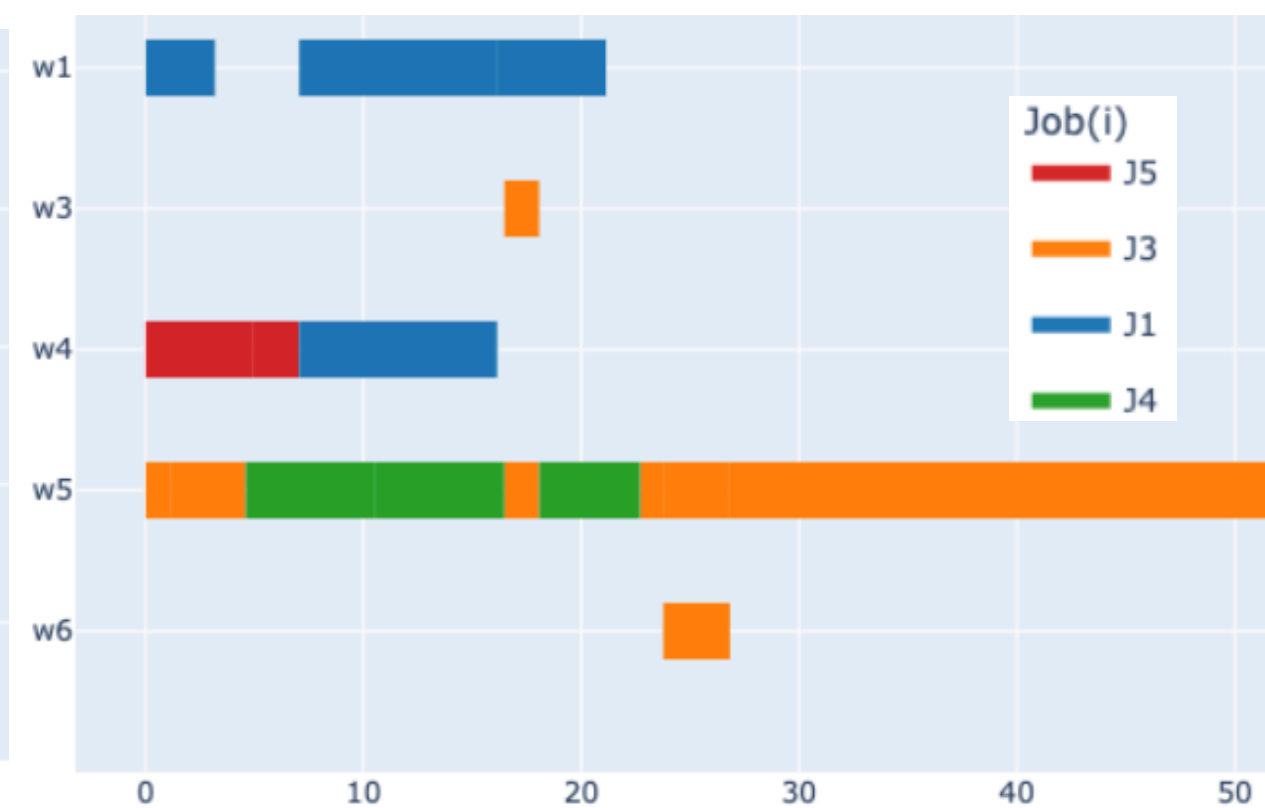
Voisinage swap



Voisinage cumulative



Voisinage alternating



Cmax = 62

fitness: 46628.81136315283
objectives: (466.89075292987206, 2.0, 18.198568847785296)

Cmax = 60.57

fitness: 46597.71005044271
objectives: (466.89075292987206, 3.0, 60.79187954918901)

Cmax = 54

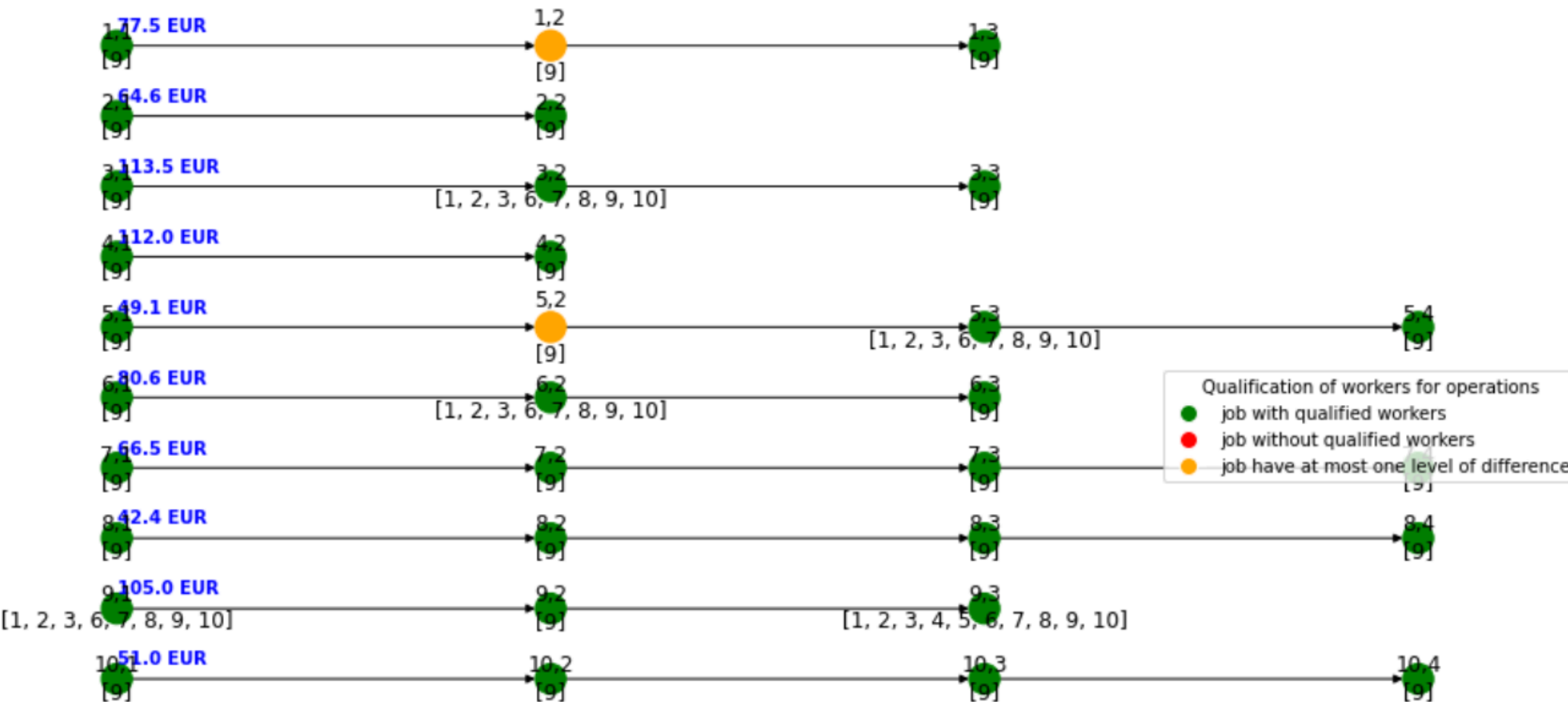
fitness: 46629.99069764843
objectives: (466.89075292987206, 2.5, 30.05865146414324)

Cmax = 51.5

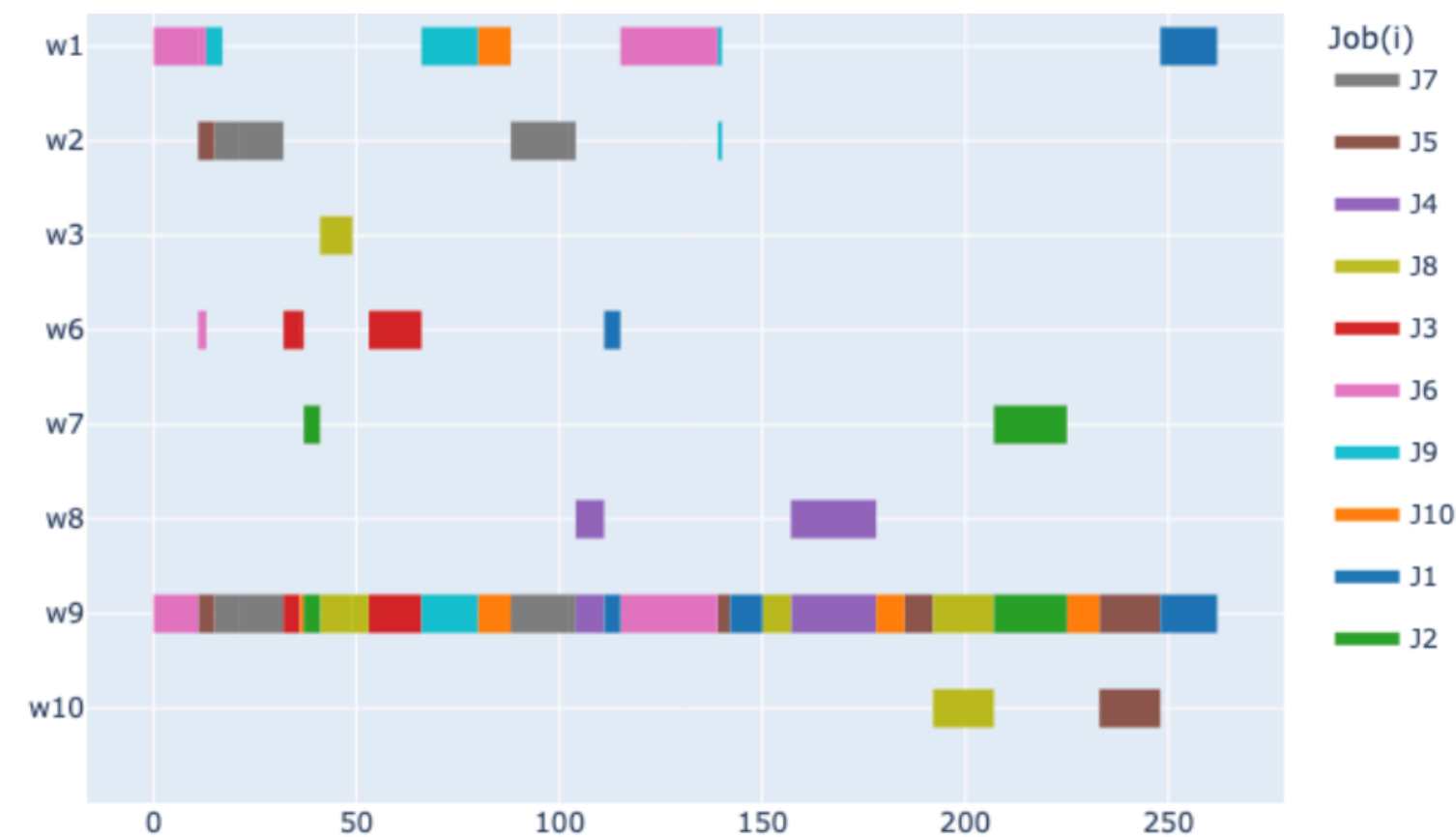
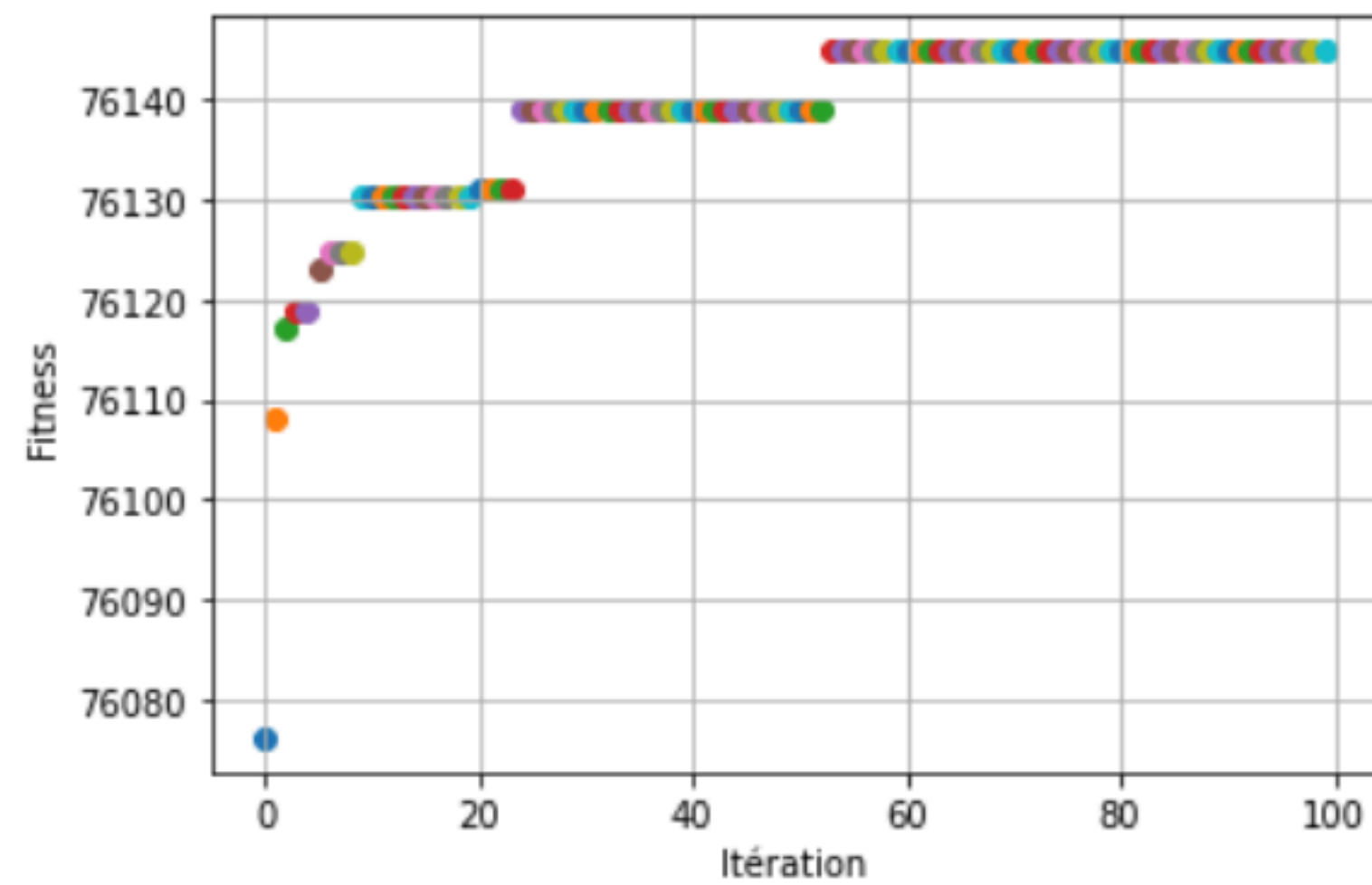
fitness: 46630.90280613877
objectives: (466.89075292987206, 1.0, 16.66011003893516)

Test voisinage worker de base

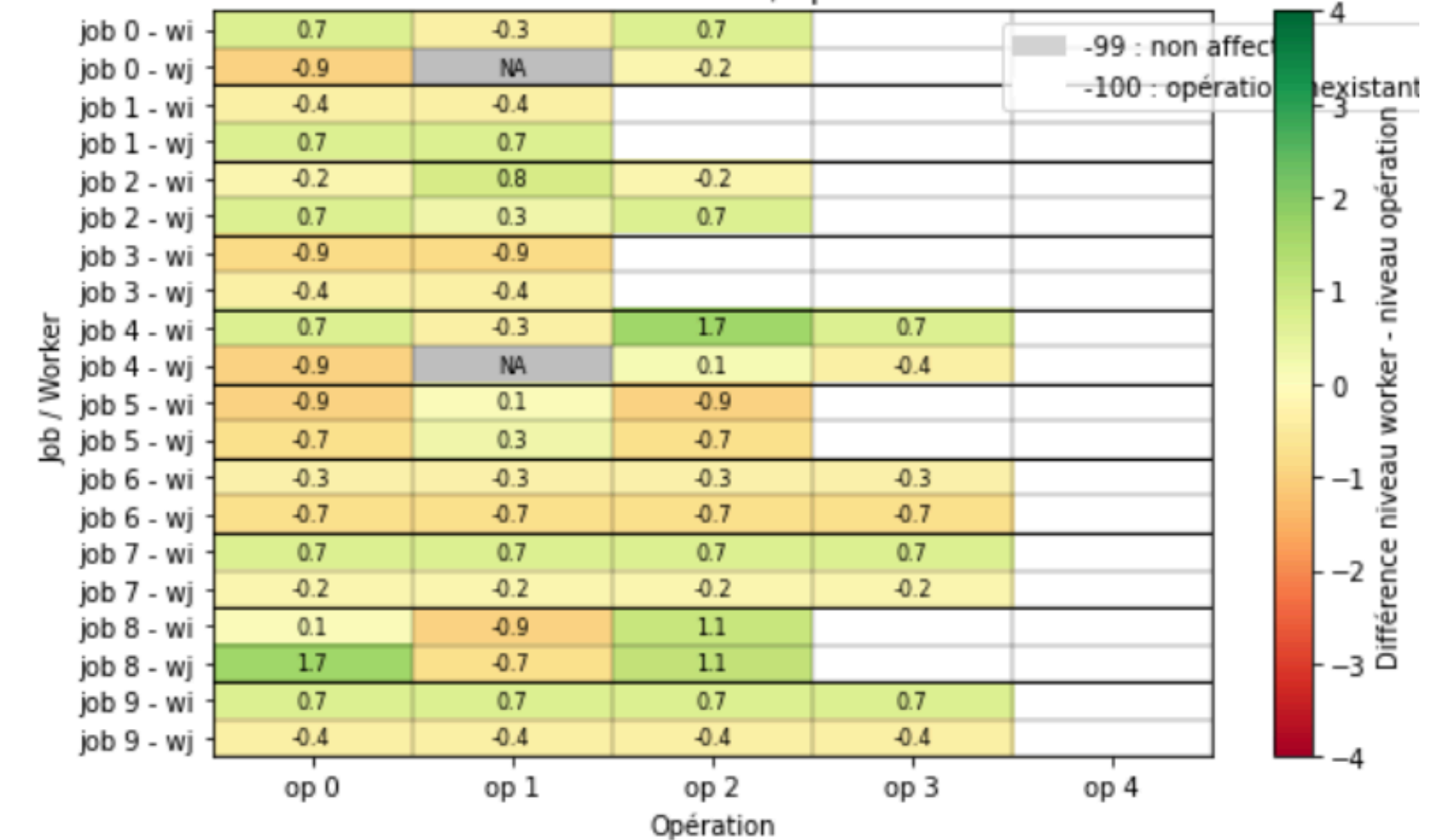
Graphe de précedence de toutes les opérations de tous les jobs



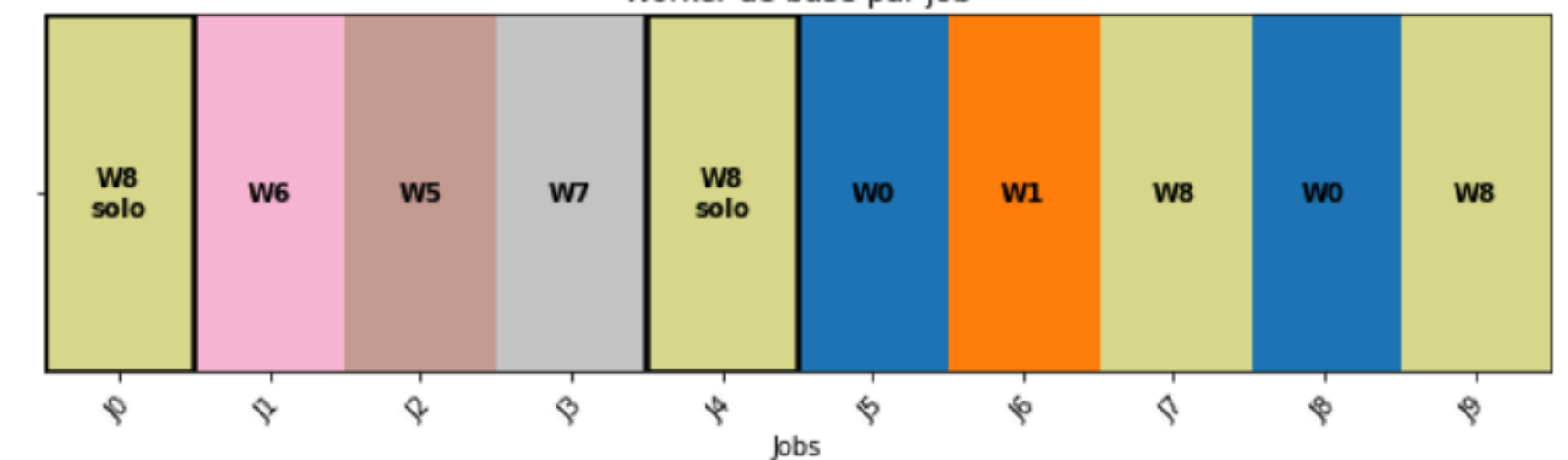
Évolution du fitness au cours des itérations



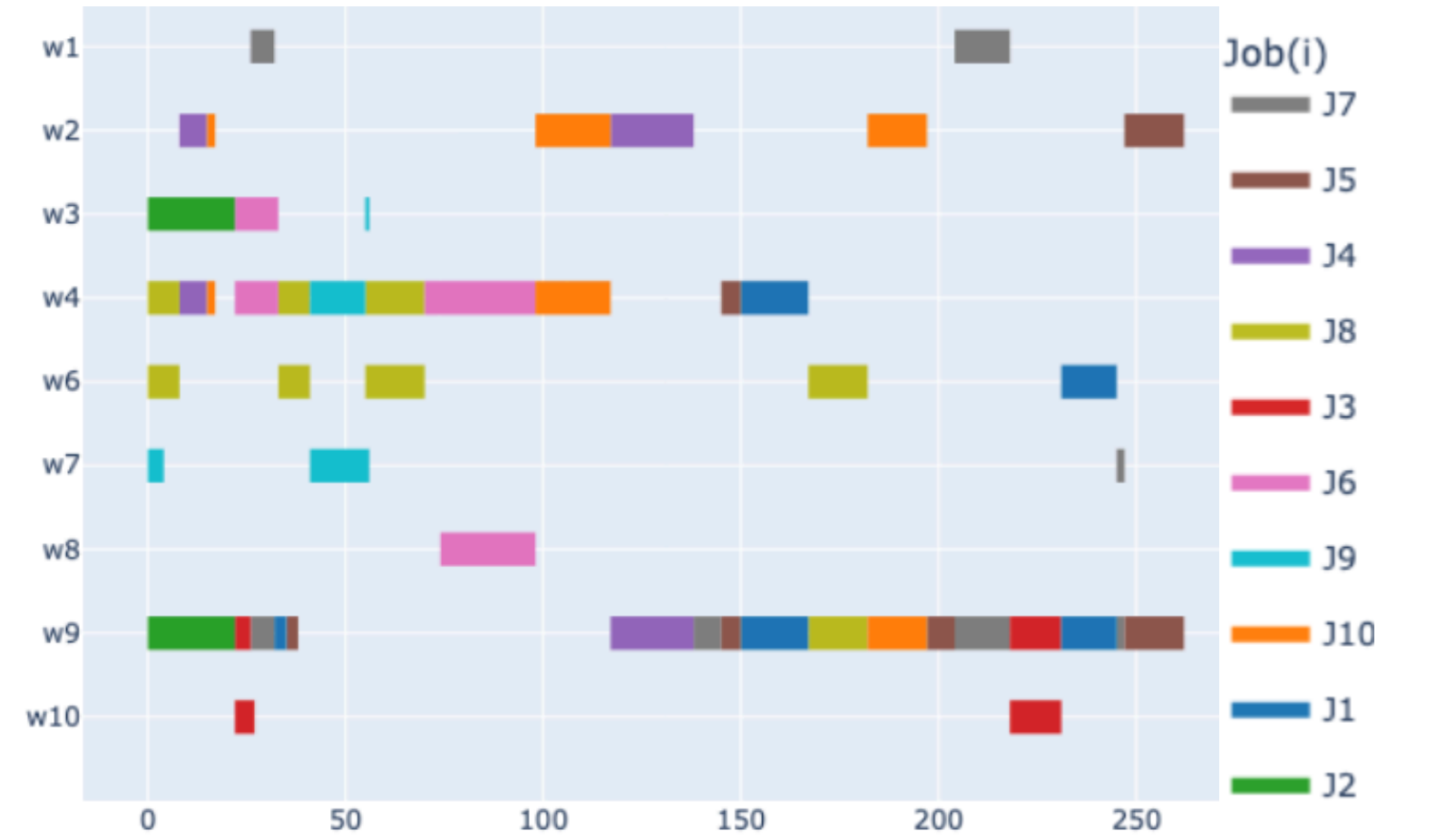
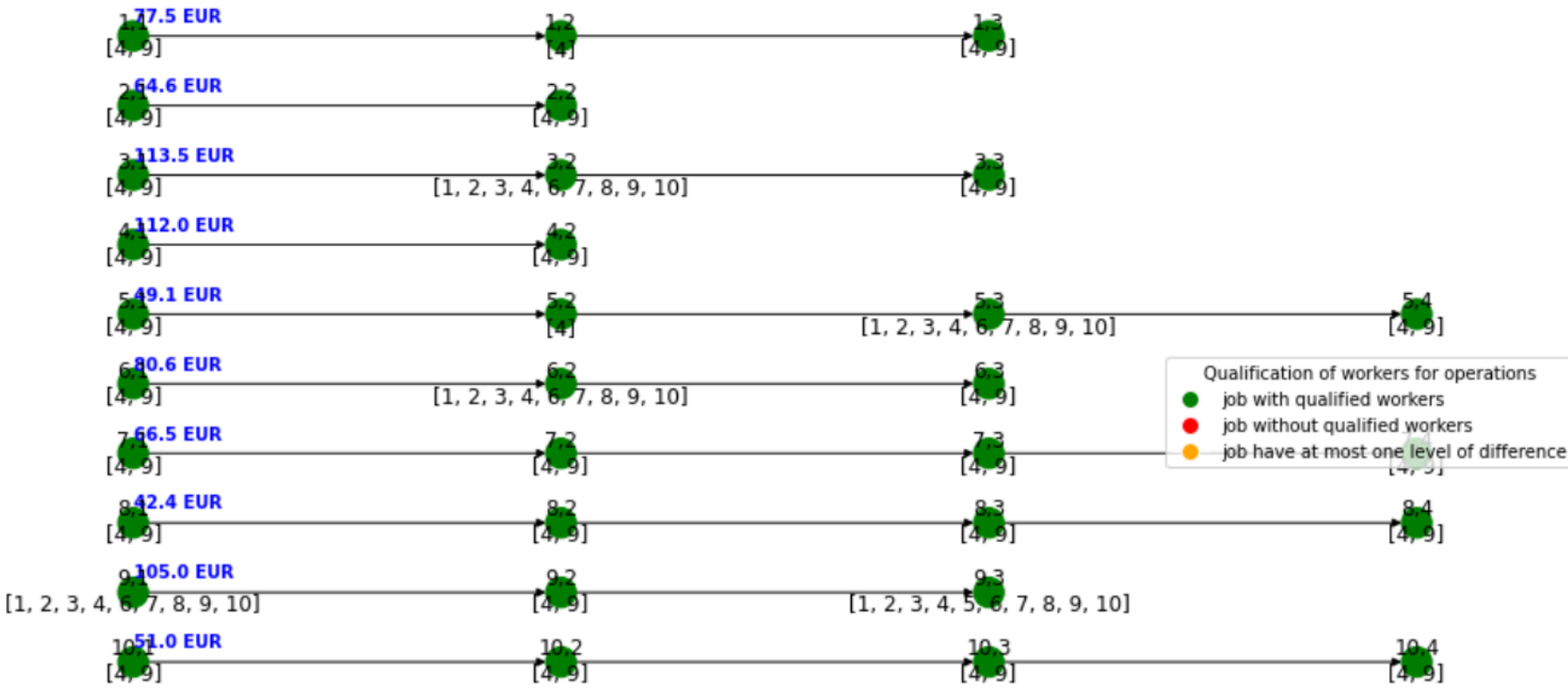
Différence worker / opération



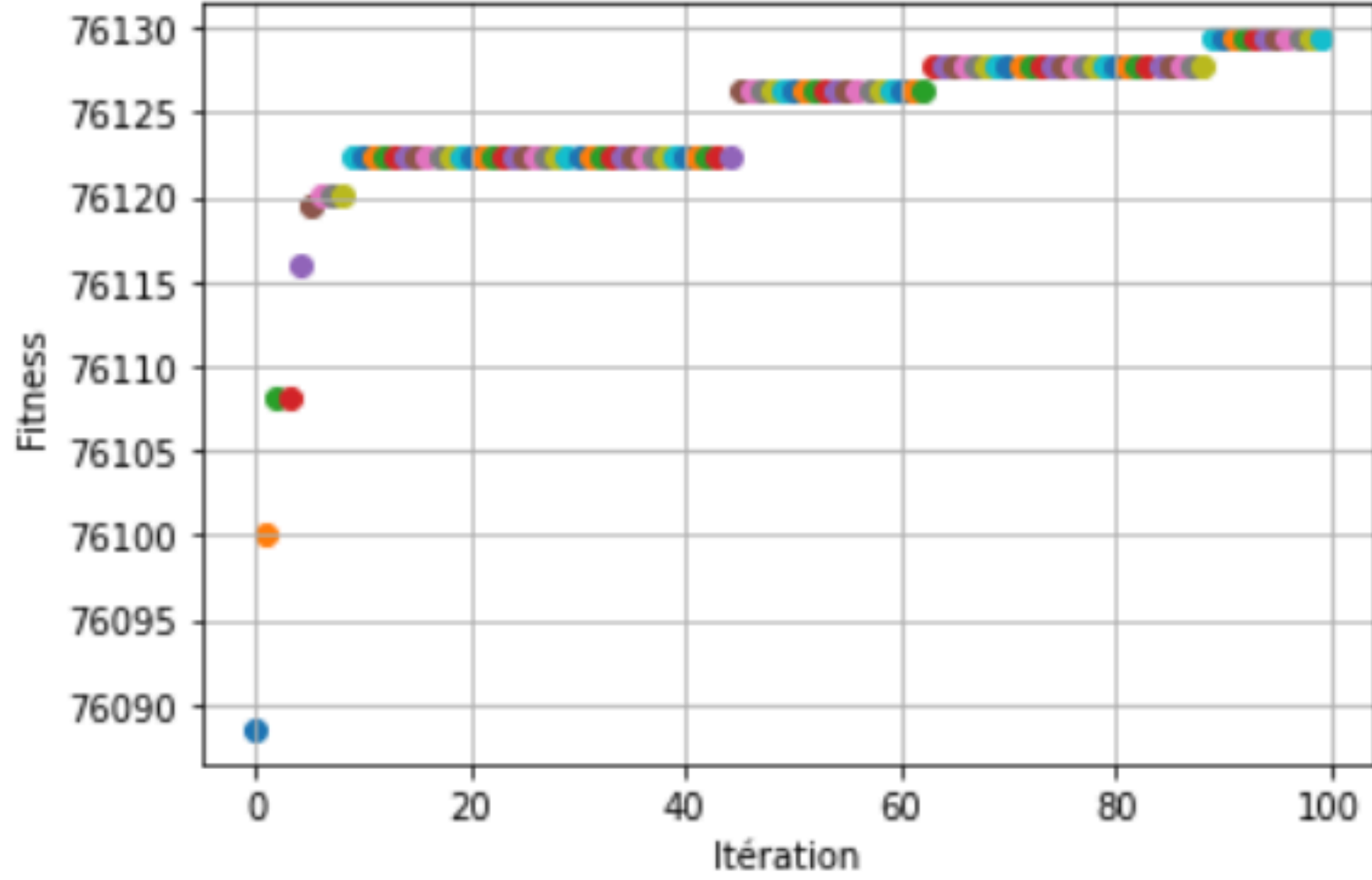
Worker de base par job



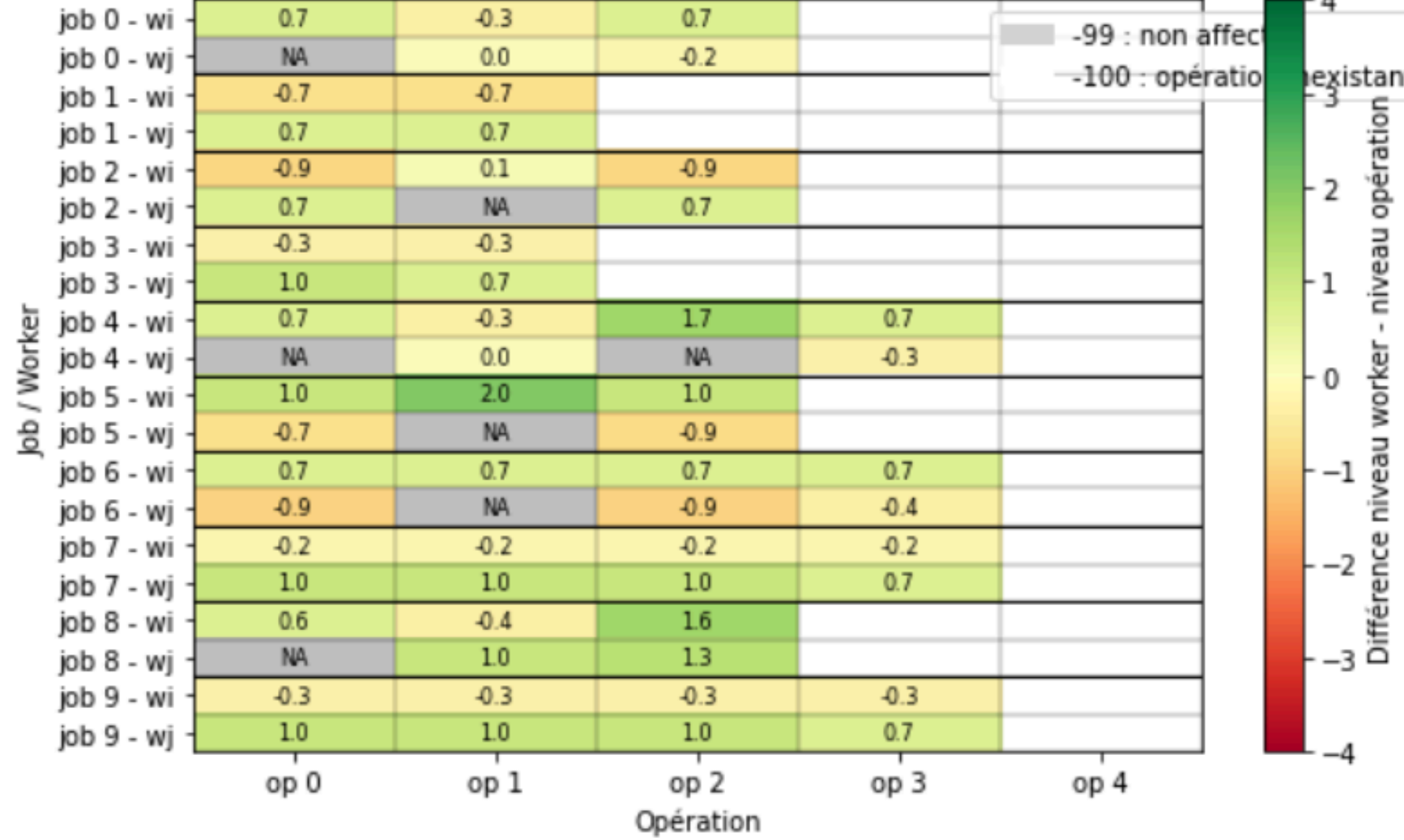
Graphe de précedence de toutes les opérations de tous les jobs



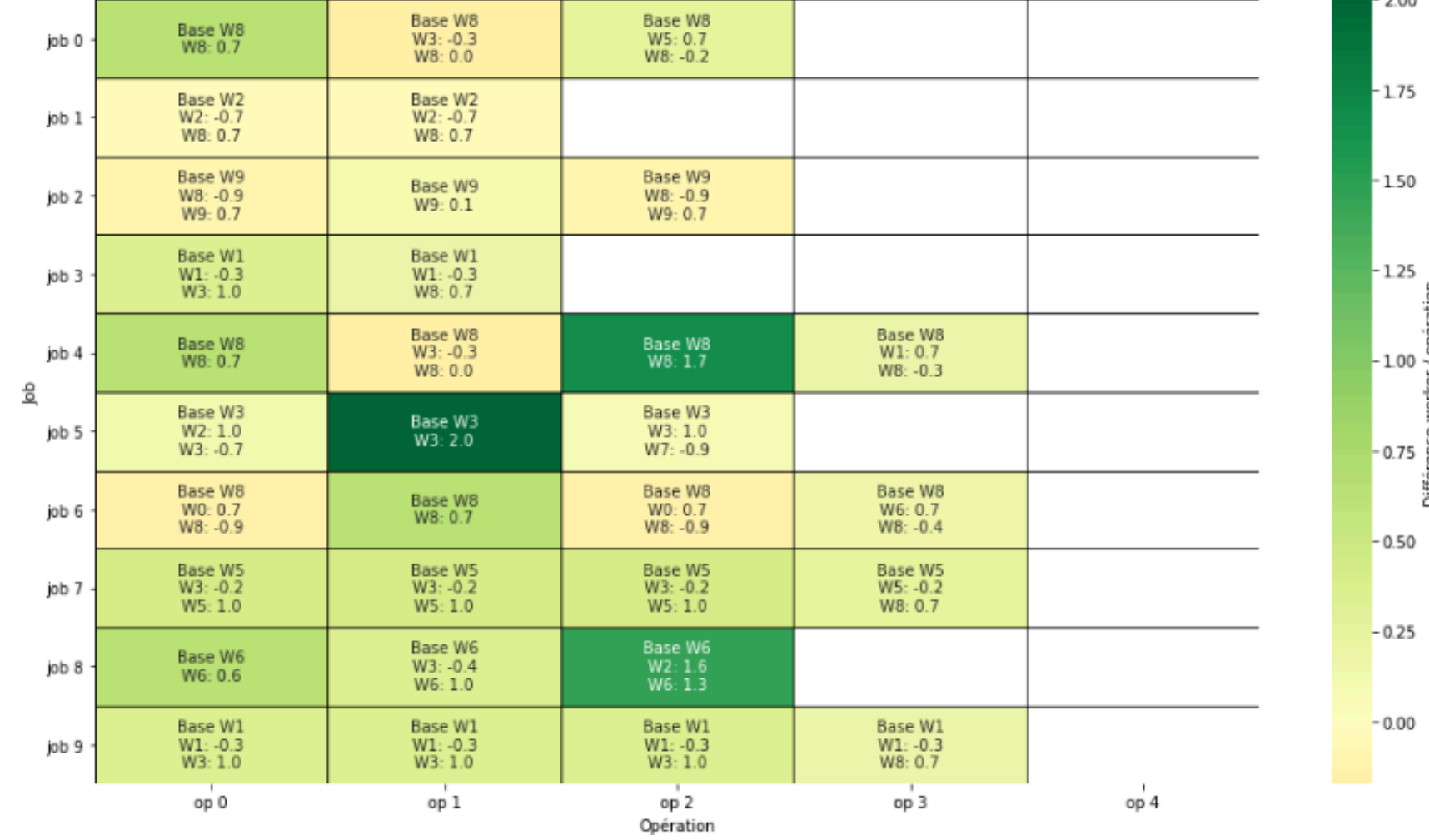
Évolution du fitness au cours des itérations



Différence worker / opération



Worker de base, workers affectés et différence



Worker de base par job

